หน้านี้ไม่ใช่ส่วนของรายงาน — ลบทิ้งก่อนส่ง

ไฟล์นี้สร้างอัตโนมัติจากไฟล์ร่าง markdown ในโฟลเดอร์ `md/` ด้วยสคริปต์ `tools/md2docx.py` รูปแบบหน้ากระดาษ ฟอนต์ ขนาดตัวอักษร ตำแหน่งเลขหน้า คำบรรยายรูปและตาราง ตั้งตามข้อกำหนดใน `report/exam_report/รูปเล่มสหกิจสำหรับคณะวิศวะ.pdf` แล้ว

สิ่งที่ต้องทำใน Word ก่อนส่ง

1. กด Ctrl+A แล้วกด F9 หนึ่งครั้ง เพื่อให้ Word สร้างสารบัญ สารบัญตาราง สารบัญรูป และเลขหน้าทั้งเล่ม (ถ้ามีกล่องถาม ให้เลือก Update entire table)
2. ลบ **กล่องพื้นสีเหลือง** ทุกกล่อง — เป็นหมายเหตุของร่าง บอกว่าตรงนั้นยังขาดข้อมูลอะไร หรือต้องถ่ายภาพอะไร ไม่ใช่เนื้อหารายงาน
3. กล่อง **พื้นสีเทา** เป็นหมายเหตุที่อยู่ในเนื้อหาได้ แต่ควรอ่านแล้วตัดสินใจว่าจะกลืนเข้าไปในย่อหน้าปกติหรือคงไว้
4. แทนที่ **กรอบเส้นประ** ทุกกรอบด้วยรูปจริง โดยวางรูปแบบ In Line with Text แล้วลบกรอบทิ้ง ชื่อไฟล์ที่ต้องถ่ายเขียนไว้ในกรอบแล้ว รายการถ่ายภาพอยู่ใน `md/ด่วน-ทำก่อนย้ายเครื่อง.md`
5. เติมช่อง `[...]` ทุกช่องในปก หน้าอนุมัติ และประวัติผู้จัดทำ
6. ตัดสินใจเรื่องชื่อโครงงาน แล้วแก้ให้เหมือนกันทุกตัวอักษรใน 5 ที่ คือ ปก · หน้าอนุมัติ · บทสรุป · Summary · สารบัญ
7. พิมพ์หัว **(ต่อ)** ให้ทุกส่วนที่ยาวข้ามหน้า — รูปเล่มจริงเขียนติดกัน ไม่เว้นวรรค คือ **สารบัญ(ต่อ)** · **สารบัญตาราง(ต่อ)** · **สารบัญรูป(ต่อ)** และ **เอกสารอ้างอิง(ต่อ)** ด้วย — Word ใส่ให้เองไม่ได้ ต้องพิมพ์เองหลังอัปเดตเลขหน้าเสร็จ
8. แทรก **รายงานปฏิบัติงานประจำสัปดาห์** ลงในภาคผนวก ก. ท้ายเล่มมีหน้าคั่นกับกล่องอธิบายวางไว้ให้แล้ว รูปเล่มจริงที่ใช้เทียบแนบไว้ 27 หน้า หน้าละสัปดาห์
9. หน้าในภาคผนวกไม่แสดงเลขหน้า ซึ่งตั้งไว้แล้ว แต่ Word จะนับหน้าต่อไปเรื่อย ๆ ทำให้หน้าประวัติผู้จัดทำได้เลขที่เลยมา ถ้าต้องการให้ต่อจากเอกสารอ้างอิงเหมือนรูปเล่มจริง ให้คลิกที่เลขหน้าของหน้าประวัติ แล้วสั่ง Format Page Numbers > Start at ใส่เลขถัดจากหน้าสุดท้ายของเอกสารอ้างอิง

10. ตรวจว่าไม่มีตารางหรือรูปใดถูกตัดคร่อมสองหน้า และไม่มีหัวข้อค้างอยู่บรรทัดสุดท้ายของหน้า

ข้อที่ยังไม่ตรงตามข้อกำหนด และเป็นการตัดสินใจของผู้เขียน

– ข้อความในตารางใช้ขนาด 14 ไม่ใช่ 16 เพราะบางตารางมีห้าคอลัมน์ ซึ่งที่ขนาด 16 จะล้นหน้ากระดาษ ถ้าอาจารย์กำหนดให้ใช้ 16 ทั้งเล่ม ให้แก้ค่า `SZ_TABLE` ในสคริปต์แล้วสร้างใหม่

– ตัวอย่างโปรแกรมใช้ฟอนต์ Consolas ขนาด 11 เพราะโปรแกรมภาษา C# ต้องอาศัยความกว้างตัวอักษรเท่ากันจึงจะอ่านการย่อหน้าได้ ข้อกำหนดระบุว่าให้ใช้ TH Sarabun New ทั้งเล่ม — ถ้าต้องยึดตามนั้นเคร่งครัด ให้แก้ค่า `MONO` ในสคริปต์

– สารบัญตารางและสารบัญรูปที่ Word สร้างจะขึ้นต้นด้วยคำว่า "ตารางที่" และ "รูปที่" ตามคำบรรยายจริง ขณะที่ตัวอย่างในแบบฟอร์มพิมพ์เฉพาะเลข — แก้ได้หลังกด F9

การพัฒนาเกมจำลองห้องปิดตายมุมมองบุคคลที่หนึ่ง ด้วยสถาปัตยกรรมซอฟต์แวร์แบบแยกส่วนบนเอนจิน Unity

Development of a First-Person Escape Room Simulator Using a Modular Software Architecture in Unity

[ชื่อ-นามสกุล พร้อมคำนำหน้า]

โครงงานสหกิจศึกษานี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตร
ปริญญาวิศวกรรมศาสตรบัณฑิต สาขา[ชื่อสาขาวิชา]
คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีไทย - ญี่ปุ่น
ปีการศึกษา 2569

ชื่อโครงงาน การพัฒนาเกมจำลองห้องปิดตายมุมมองบุคคลที่หนึ่งด้วยสถาปัตยกรรมซอฟต์แวร์แบบแยกส่วนบนเอนจิน Unity

Development of a First-Person Escape Room Simulator Using a Modular Software Architecture in Unity

ผู้เขียน [ชื่อ-นามสกุล]

สาขาวิชา [สาขาวิชา]

อาจารย์ที่ปรึกษาสหกิจศึกษา [ชื่อ-นามสกุล]

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีไทย-ญี่ปุ่น

อนุมัติให้โครงงานสหกิจศึกษาฉบับนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

..... ประธานสหกิจศึกษาสาขา

(.....)

วันที่ ..... เดือน ..... พ.ศ. .....

คณะกรรมการสอบโครงงานสหกิจ

..... ประธานกรรมการ

(.....)

..... กรรมการ

(.....)

..... อาจารย์ที่ปรึกษาสหกิจ

(.....)

ลิขสิทธิ์ของสถาบันเทคโนโลยีไทย – ญี่ปุ่น

ชื่อโครงงาน	การพัฒนาเกมจำลองห้องปิดตายมุมมองบุคคลที่หนึ่งด้วยสถาปัตยกรรมซอฟต์แวร์แบบแยกส่วนบนเอนจิน Unity
ผู้เขียน	[ชื่อ-นามสกุล]
คณะ	คณะวิศวกรรมศาสตร์
สาขาวิชา	[สาขาวิชา]
อาจารย์ที่ปรึกษาสหกิจศึกษา	[ชื่อ-นามสกุล]
พนักงานที่ปรึกษา	[ชื่อ-นามสกุล]
ชื่อบริษัท	บริษัท Lumas corp

บทสรุป

โครงงานนี้มีที่มาจากโจทย์ของพนักงานที่ปรึกษาให้พัฒนาเกมจำลองห้องปิดตายมุมมองบุคคลที่หนึ่งบนเอนจิน Unity โดยกำหนดปุ่มควบคุมและธีมของเกมมาให้ ส่วนปริศนาเป็นงานออกแบบของผู้เขียนเองภายในเวลาจำกัด ปัญหาที่แท้จริงจึงไม่ใช่การทำให้เกมทำงานได้ แต่เป็นการออกแบบโครงสร้างให้การเพิ่มวัตถุชนิดใหม่หรือการเปลี่ยนกฎของปริศนาไม่บังคับให้แก้โปรแกรมที่เขียนไว้แล้ว ผู้เขียนจึงประกอบวัตถุโต้ตอบทุกชิ้นจากคอมโพเนนต์ย่อย แทนการสืบทอดคลาส ให้คอมโพเนนต์ค้นหาผู้ร่วมงานผ่านส่วนต่อประสานตามบทบาท บังคับขอบเขตของโมดูลด้วยไฟล์ Assembly Definition ให้การพึ่งพาที่ผิดทิศเป็นข้อผิดพลาดตอนคอมไพล์ และเขียนชุดทดสอบยืนยันข้อกำหนดของสถาปัตยกรรมการวิเคราะห์ทั้งหมดวัดจากไฟล์ของโครงงานโดยตรง

ผลที่ได้คือเกมที่เล่นได้ครบห้าห้องและปริศนาห้าองก์ตามเส้นทางเดียว โปรแกรมมี 78 ไฟล์ 10,604 บรรทัด (นับบรรทัดที่ไม่ว่าง) ประกอบด้วย 51 คลาสที่สืบทอดจาก MonoBehaviour โดยตรงทั้งหมด ไม่สืบทอดระหว่างกัน ส่วนต่อประสาน 12 รายการกับการนำไปสร้างจริง 29 ความสัมพันธ์ ขอบเขตของโมดูลบังคับด้วยไฟล์ Assembly Definition 6 ไฟล์ และชุดทดสอบ 15 กรณี ที่ทำงานได้โดยไม่ต้องเปิดฉาก ประโยชน์ที่วัดได้ชัดที่สุดคือ นาฬิกาทั้ง 46 เรือนใช้หน้าปัดเดียวกัน และชนิดของนาฬิกาเกิดจากคอมโพเนนต์ที่ประกอบเข้าไป จึงเปลี่ยนกลไกของปริศนาระหว่างการพัฒนาได้โดยไม่ต้องแก้โปรแกรมส่วนอื่น ข้อเสนอแนะคือขยายชุดทดสอบให้ครอบคลุมโมดูลปริศนา ย้ายข้อความออกเป็นไฟล์ข้อมูล และกำหนดข้อตกลงเรื่องช่วงเวลาที่เรียกใช้ส่วนต่อประสานให้ชัดเจน

Title of Project	Development of a First-Person Escape Room Simulator Using a Modular Software Architecture in Unity
Writer / Submitted by	[ชื่อ-นามสกุล (อังกฤษ)]
Faculty	Faculty of Engineering
Major	[Major]
Advisor's	[Advisor (อังกฤษ)]
Job Supervisor	[พี่เลี้ยง (อังกฤษ)]
Company's name	Lumas Corp.

Summary

The supervisor's assignment was a first-person escape room simulator in Unity, with the controls and genre fixed and the puzzle design left to the author on a limited schedule. The engineering problem was therefore not making the game run but structuring it so that adding a new kind of interactive object, or changing a puzzle rule, would not force edits to code already written. Objects are composed from small components instead of subclassed, and each finds its collaborators through role-named interfaces rather than concrete classes. Assembly Definition files enforce the module boundaries, making a wrong-way dependency a compile error rather than a matter of discipline, and a test suite asserts that those constraints hold. Every figure below is measured from the project's own files.

The result is a playable game of five rooms and five puzzle acts on a single route: 78 source files, 10,604 non-blank lines, 51 MonoBehaviour classes all deriving directly from MonoBehaviour with no inheritance between them, 12 interfaces with 29 implementations, 6 Assembly Definition files, and 15 test cases that run without a scene or a player object. All 46 clocks share one dial implementation, and each clock's kind is decided by the components attached to it rather than by a class of its own, which let a puzzle mechanism be replaced mid-development without altering the rest of the program. The recommendations are to extend the suite to the puzzle module, move all in-game text into data files, and state when each interface may be called.

กิตติกรรมประกาศ

รายงานการปฏิบัติงานสหกิจศึกษาฉบับนี้สำเร็จลงได้ด้วยความอนุเคราะห์จากบริษัท Lumas corp ที่ให้โอกาสผู้เขียนเข้าปฏิบัติงานในตำแหน่งนักพัฒนาเกม ผู้เขียนขอขอบพระคุณ [**ชื่อพี่เลี้ยง**] [**ตำแหน่ง**] พนักงานที่ปรึกษา ที่มอบหมายโจทย์โครงงาน ให้คำแนะนำในการปฏิบัติงาน และให้อิสระในการออกแบบส่วนที่เป็นงานสร้างสรรค์ของโครงงาน และขอขอบพระคุณพนักงานทุกท่านในบริษัทที่ให้ความช่วยเหลือและคำแนะนำระหว่างการปฏิบัติงาน

ผู้เขียนขอขอบพระคุณ [**ชื่ออาจารย์ที่ปรึกษา**] อาจารย์ที่ปรึกษาสหกิจศึกษา ที่ให้คำแนะนำในการจัดทำรายงาน และคณาจารย์คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีไทย-ญี่ปุ่นทุกท่าน ที่ประสิทธิ์ประสาทความรู้อันเป็นพื้นฐานของการปฏิบัติงาน ครั้งนี้ ขอขอบคุณเพื่อนนักศึกษาที่ช่วยทดสอบการเล่นเกมและให้ความเห็นที่เป็นประโยชน์ ต่อการปรับปรุงปริศนา และขอขอบพระคุณครอบครัวที่ให้การสนับสนุนตลอดระยะเวลาการศึกษา

[ชื่อ-นามสกุล]

[เดือน พ.ศ.]

สารบัญ

	หน้า
บทสรุปภาษาไทย	____
บทสรุปภาษาอังกฤษ	____
กิตติกรรมประกาศ	____
สารบัญ	____
สารบัญตาราง	____
สารบัญรูป	____

บทที่

บทที่ 1 บทนำ	1
1.1 ชื่อและที่ตั้งของสถานประกอบการ	1
1.2 ลักษณะธุรกิจของสถานประกอบการ หรือการให้บริการหลักขององค์กร	2
1.3 รูปแบบการจัดองค์กรและการบริหารองค์กร	2
1.4 ตำแหน่งและหน้าที่งานที่นักศึกษาได้รับมอบหมาย	3
1.5 พนักงานที่ปรึกษา และ ตำแหน่งของพนักงานที่ปรึกษา	4
1.6 ระยะเวลาที่ปฏิบัติงาน	5
1.7 ที่มาและความสำคัญของปัญหา	6
1.8 วัตถุประสงค์หรือจุดมุ่งหมายของโครงงาน	8
1.9 ผลที่คาดว่าจะได้รับจากการปฏิบัติงานหรือโครงงานที่ได้รับมอบหมาย	8
1.10 นิยามศัพท์เฉพาะ	9
บทที่ 2 ทฤษฎีที่เกี่ยวข้อง	12
2.1 ทฤษฎีการออกแบบซอฟต์แวร์เชิงโมดูล	12
2.2 หลักการออกแบบเชิงวัตถุที่ใช้ในโครงงาน	18
2.3 ทฤษฎีความถูกต้องของการนับเวลาในเกม	26
2.4 ทฤษฎีการทดสอบซอฟต์แวร์	29
2.5 เทคโนโลยีที่ใช้ในการปฏิบัติงาน	32
บทที่ 3 ขั้นตอนการดำเนินงาน	42
3.1 แผนงานการทำโครงงาน	42
3.2 รายละเอียดขั้นตอนการทำโครงงาน	46

3.3 ขั้นตอนการดำเนินงานที่นักศึกษาปฏิบัติงานหรือโครงงาน 50
บทที่ 4 ผลการดำเนินงาน 64
4.1 ผลการดำเนินงาน 64
4.2 ผลการวิเคราะห์โครงสร้างซอฟต์แวร์ 72
4.3 ผลการทดสอบ 84
4.4 สรุปผลและข้อจำกัดที่พบ 91
บทที่ 5 บทสรุปและข้อเสนอแนะ 97
5.1 สรุปผลการดำเนินงาน 97
5.2 ปัญหาและแนวทางการแก้ไข 98
5.3 ข้อเสนอแนะจากการดำเนินโครงงาน 104
เอกสารอ้างอิง 108
ประวัติผู้จัดทำโครงงาน 114

สารบัญตาราง

ตารางที่	หน้า
ตารางที่ 1.1 ข้อกำหนดการควบคุมตามโจทย์ เทียบกับความสามารถของซอฟต์แวร์ที่ส่งมอบ	3
ตารางที่ 2.1 โมดูลของโครงงาน จำนวนไฟล์ และขอบเขตระดับการคอมไพล์	12
ตารางที่ 2.2 ทิศทางการพึ่งพาระหว่างโมดูลที่วัดได้จากซอร์สโค้ด	14
ตารางที่ 2.3 ตัวชี้วัดที่ยืนยันการใช้ Composition แทน Inheritance	17
ตารางที่ 2.4 Interface ที่ประกาศในโครงงานและจำนวนผู้ implement	19
ตารางที่ 2.5 ผลการประเมินตามหลักการเปิด–ปิด	22
ตารางที่ 2.6 การใช้ Observer Pattern ในโครงงาน	24
ตารางที่ 2.7 ระยะห่างระหว่างเลขที่แทนได้ ณ ค่าสะสม 280 นาที (เวลา 4:40)	27
ตารางที่ 2.8 ไฟล์ทดสอบและคุณลักษณะเชิงสถาปัตยกรรมที่แต่ละไฟล์พิสูจน์	30
ตารางที่ 2.9 ข้อตกลงการอ่านข้อมูลนำเข้าตามชนิดของค่า	38
ตารางที่ 2.10 เครื่องมือสร้างเนื้อหาด้วยกระบวนวิธีในโครงงาน	40
ตารางที่ 3.1 แผนการปฏิบัติงานและการทำโครงงาน	43
ตารางที่ 3.2 จำนวนไฟล์ที่ถูกเขียนครั้งล่าสุดในแต่ละวันของการพัฒนา	44
ตารางที่ 3.3 ข้อกำหนดจากโจทย์และส่วนของโปรแกรมที่รองรับ	46
ตารางที่ 3.4 ขอบเขตเนื้อหาของเกมที่กำหนดขึ้นเอง	47
ตารางที่ 3.5 เครื่องมือที่ใช้ในการพัฒนาโครงงาน	48
ตารางที่ 3.6 ทรัพยากรที่นำเข้าจากภายนอกโครงงาน	49
ตารางที่ 3.7 ลำดับขั้นการดำเนินงานทั้งแปดขั้น	50
ตารางที่ 3.8 ไฟล์ที่ถูกแก้ไขพร้อมกันในวันที่ 2 กันยายน 2569 เวลา 14:05:12 น.	55
ตารางที่ 3.9 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา	62
ตารางที่ 4.1 ปริมาณผลลัพธ์ของโครงงานที่วัดได้จากไฟล์ในโปรเจกต์	64
ตารางที่ 4.2 คำตอบของปริศนาแต่ละองก์และหลักฐานที่ยืนยัน	67
ตารางที่ 4.3 กลไกที่แต่ละองก์ใช้ร่วมกันและส่วนที่เขียนขึ้นเฉพาะ	69
ตารางที่ 4.4 จำนวนบรรทัดของโปรแกรมแยกตามบทบาท	71
ตารางที่ 4.5 โมดูลทั้งเก้ากลุ่มและขนาดของแต่ละกลุ่ม	73
ตารางที่ 4.6 การพึ่งพาระหว่างโมดูลที่วัดได้จากโปรแกรม	74
ตารางที่ 4.7 การเทียบขอบเขตที่ประกาศไว้กับที่โปรแกรมใช้จริง	76
ตารางที่ 4.8 ส่วนต่อประสานทั้งสิบสองรายการและจำนวนคลาสที่นำไปใช้	77

ตารางที่ 4.9 หลักฐานการนำกลับมาใช้ที่วัดได้ 79
ตารางที่ 4.10 ผลการนับรูปแบบการเขียนโปรแกรมในโปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์ 80
ตารางที่ 4.11 ผลการประเมินความสามารถในการขยายเก้าสถานการณ์ 82
ตารางที่ 4.12 ชุดทดสอบทั้ง 15 กรณีและสิ่งที่แต่ละกรณียืนยัน 84
ตารางที่ 4.13 ความครอบคลุมของชุดทดสอบแยกตามโมดูล 87
ตารางที่ 4.14 สถานะของโปรเจกต์และข้อความที่พบใน Console 89
ตารางที่ 4.15 ผลการประเมินคุณภาพของโครงสร้างแปดเกณฑ์ 92
ตารางที่ 4.16 ข้อจำกัดที่พบ เรียงตามผลกระทบ 93
ตารางที่ 4.17 จุดที่เอกสารประกอบภายในโปรเจกต์ไม่ตรงกับไฟล์จริง 94
ตารางที่ 4.18 ผลเทียบกับขอบเขตที่กำหนดไว้ในหัวข้อ 3.2.1 95
ตารางที่ 5.1 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา 99
ตารางที่ 5.2 ลำดับความสำคัญของข้อเสนอแนะ 107

สารบัญรูป

รูปที่	หน้า
รูปที่ 1.1 แผนที่บริษัท Lumas corp	1
รูปที่ 1.2 ผังการจัดองค์กรของบริษัท Lumas corp	2
รูปที่ 1.3 โจทย์โครงงานที่ได้รับจากพนักงานที่ปรึกษา	4
รูปที่ 2.1 ทิศทางการพึ่งพาระหว่างโมดูลของโครงงาน	15
รูปที่ 2.2 ส่วนติดต่อผู้ใช้ของ Unity Editor ในโครงงาน	33
รูปที่ 2.3 ข้อมูลผังบ้านที่เก็บในรูปแอสเซ็ต ScriptableObject	35
รูปที่ 2.4 การประกาศขอบเขตการอ้างอิงของโมดูลนาฬิกาด้วยไฟล์ Assembly Definition	37
รูปที่ 2.5 ผลการรันชุดทดสอบ PlayMode ทั้ง 15 กรณีด้วย Unity Test Framework	39
รูปที่ 3.1 โจทย์ของโครงงานที่ได้รับจากพนักงานที่ปรึกษา	42
รูปที่ 3.2 ลำดับขั้นการดำเนินงานทั้งแปดขั้นเรียงตามเวลา	52
รูปที่ 3.3 เมนูเครื่องมือสร้างเนื้อหาที่พัฒนาขึ้นในโครงงาน	58
รูปที่ 3.4 ผังบ้านทั้งห้าห้องที่สร้างจากไฟล์ HousePlan.asset	58
รูปที่ 4.1 ห้องหน้าร้าน องก์ที่หนึ่ง	65
รูปที่ 4.2 ห้องช่าง องก์ที่สอง	65
รูปที่ 4.3 ห้องนั่งเล่น องก์ที่สาม	66
รูปที่ 4.4 ห้องนอน องก์ที่สี่	66
รูปที่ 4.5 ห้องหลังตู้นาฬิกา องก์ที่ห้า	66
รูปที่ 4.6 ผนังนาฬิกาเจ็ดเรือนขององก์ที่หนึ่ง ให้เห็นลูกตุ้มที่แกว่งและที่หยุด	70
รูปที่ 4.7 การยกนาฬิกาขึ้นดูและแป้นเลขที่ปรากฏเมื่อไขลาน	70
รูปที่ 4.8 นาฬิกาพกและนาฬิกาทรายที่ชูขึ้นข้างกันขณะจับเวลาในองก์ที่สอง	70
รูปที่ 4.9 ภาพถ่ายเก่าและป้ายเลขหลังของในองก์ที่สี่	71
รูปที่ 4.10 จุดแสงหกจุดบนจอก่อนและหลังหมุนปริซึมถูกต้อง ในองก์ที่ห้า	71
รูปที่ 4.11 ผังบ้านทั้งห้าห้องมองจากด้านบนใน Scene View	72
รูปที่ 4.12 แผนภาพการพึ่งพาระหว่างโมดูลทั้งแปดกลุ่ม	75
รูปที่ 4.13 ไฟล์ PlankClock.asset ใน Inspector ให้เห็นรายการ Fittings ที่ไม่แยกส่วนแสดงเวลาจากชิ้นส่วนกลไก	80
รูปที่ 4.14 ผลการรันชุดทดสอบใน Test Runner แสดงทั้ง 15 กรณีและเวลาที่ใช้	86
รูปที่ 4.15 หน้าต่าง Console ของ Unity แสดงสถานะการคอมไพล์และข้อความที่พบ	91

รูปที่ 5.1 ส่วนของโปรแกรมที่แก้ปัญหาการนับเวลา คือการประกาศตัวแปรสะสมเป็น double และการคิดเศษของเฟรมสุดท้าย 100
รูปที่ 5.2 ข้อความข้อยกเว้นและลำดับการเรียกที่ปรากฏใน Unity Console 104

บทที่ 1
บทนำ

1.1 ชื่อและที่ตั้งของสถานประกอบการ

บริษัท **Lumas corp** [จากผู้เขียน]

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ส่วนที่ยังเติมไม่ได้** ที่ตั้งของสถานประกอบการ เลขที่ ถนน แขวง เขต จังหวัด รหัสไปรษณีย์ และประวัติความเป็นมาโดยย่อ **ไม่มีอยู่ในโปรเจกต์** จึงเขียนแทนผู้เขียนไม่ได้ ตัวอย่างในแบบฟอร์มของคณะเขียนส่วนนี้ประมาณหนึ่งย่อหน้า ความยาวราว 10–12 บรรทัด ประกอบด้วยชื่อเต็มขององค์กร ปีที่ก่อตั้ง วัตถุประสงค์ในการก่อตั้ง และขอบเขตการดำเนินงานในปัจจุบัน แล้วปิดท้ายด้วยรูปแผนที่
> **โครงประโยคที่ใช้เติมได้** — "บริษัท Lumas corp ตั้งอยู่เลขที่ [ต้องเติม] ถนน [ต้องเติม] แขวง [ต้องเติม] เขต [ต้องเติม] จังหวัด [ต้องเติม] รหัสไปรษณีย์ [ต้องเติม] ก่อตั้งเมื่อ [ต้องเติม] ดำเนินกิจการด้าน [ต้องเติม] ..."

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📷 **รูปที่ 1.1 แผนที่บริษัท Lumas corp** — แคปจากแผนที่ออนไลน์ หรือขอจากฝ่ายบุคคลของบริษัท
> แบบฟอร์มของคณะกำหนดให้ส่วนนี้มีรูปแผนที่หนึ่งรูป วางแบบ In Line with Text และคำบรรยายอยู่ใต้รูป **คำบรรยายห้ามขึ้นต้นด้วยคำว่า "แสดง"** ให้พิมพ์ว่าสิ่งนั้นคืออะไร

[วางรูปที่ 1.1 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-1-01.png

รูปที่ 1.1 แผนที่บริษัท Lumas corp

1.2 ลักษณะธุรกิจของสถานประกอบการ หรือการให้บริการหลักขององค์กร

> [หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ส่วนนี้ยังเขียนไม่ได้** ลักษณะธุรกิจ สินค้าหรือบริการหลัก กลุ่มลูกค้า และขอบเขตการให้บริการของ Lumas corp **ไม่ปรากฏที่ใดในโปรเจกต์** และห้ามอนุมานจากชื่อบริษัทหรือจากลักษณะของงานที่ได้รับมอบหมาย
> **สิ่งเดียวที่อนุมานได้อย่างมีหลักฐาน** คือบริษัทมีงานด้านการพัฒนาเกมอยู่จริง เพราะตำแหน่งที่มอบหมายให้นักศึกษาคือ Game Developer และโจทย์ที่ได้รับเป็นโจทย์พัฒนาเกม (หัวข้อ 1.4) แต่ข้อมูลเท่านี้ไม่พอจะเขียนเป็นหัวข้อ และการเขียนเกินจากนี้จะกลายเป็นการแต่งข้อมูลทันที

1.3 รูปแบบการจัดองค์กรและการบริหารองค์กร

> [หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ส่วนนี้ยังเขียนไม่ได้** ต้องใช้ผังองค์กร (Organization Chart) ของบริษัท ซึ่งต้องขอจากพนักงานที่ปรึกษาหรือฝ่ายบุคคล
> ตัวอย่างรูปเล่มของคณะใช้ผังองค์กรหนึ่งรูปพร้อมคำอธิบายสั้น ๆ ว่าแผนกที่นักศึกษาสังกัดอยู่ตรงไหนของผัง
> **สิ่งที่ควรระบุเพิ่มเมื่อมีผังแล้ว** คือชื่อแผนกหรือทีมที่นักศึกษาสังกัด และจำนวนบุคลากรในทีมนั้น ซึ่งปัจจุบันยังไม่ทราบ [ต้องเติม]

> [หมายเหตุของร่าง — ลบก่อนส่ง] 📷 **รูปที่ 1.2 ผังการจัดองค์กรของบริษัท Lumas corp** — ขอจากบริษัท หรือวาดขึ้นใหม่ตามที่พนักงานที่ปรึกษาอธิบาย

[วางรูปที่ 1.2 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-1-02.png

รูปที่ 1.2 ผังการจัดองค์กรของบริษัท Lumas corp

1.4 ตำแหน่งและหน้าที่งานที่นักศึกษาได้รับมอบหมาย

ตำแหน่งที่ได้รับมอบหมายคือ **Game Developer** [จากผู้เขียน] หน้าที่หลักคือการพัฒนาเกมตามโจทย์ที่พนักงานที่ปรึกษากำหนดให้ โดยโจทย์ที่ใช้เป็นโครงงานของรายงานฉบับนี้คือโจทย์ลำดับที่สาม ชื่อ **Project 3 : Escape Room Simulator**

โจทย์กำหนดขอบเขตของงานไว้สามระดับ คือ **ประเภทของเกม** ที่ต้องทำ **ข้อกำหนดการควบคุม** ที่ต้องทำตามทุกข้อ และ **เงื่อนไขของผลงาน** ซึ่งระบุว่างานนี้จะนับเป็น Capstone Project ก็ต่อเมื่อปริศนาในเกมเป็นการออกแบบขึ้นเองทั้งหมด (original puzzle design) ส่วนการจัดวางฉากโจทย์ระบุให้เป็นหน้าที่ของผู้พัฒนาเอง และปริศนาทั้งห้าองก์ในเกมที่ส่งมอบเป็นการออกแบบขึ้นเองทั้งหมด [จากผู้เขียน]

หน้าที่งานที่ได้รับมอบหมายจึงไม่ใช่การเพิ่มด่านให้เกมที่มีอยู่แล้ว แต่เป็นการพัฒนาเกมทั้งเกมขึ้นมาใหม่ ตั้งแต่ระบบผู้เล่น ระบบการโต้ตอบกับวัตถุ ระบบปริศนา การจัดวางฉาก ไปจนถึงการสร้างแอสเซ็ตที่ใช้ในฉาก ซึ่งโจทย์ระบุเหตุผลไว้ว่าต้องการให้ทำทั้งเกมเพื่อการฝึก

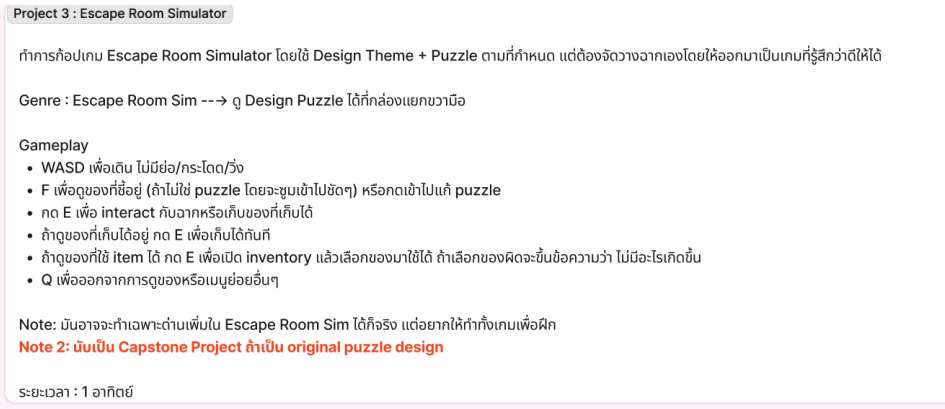
ข้อกำหนดการควบคุมที่โจทย์ระบุไว้ และผลการตรวจสอบว่าซอฟต์แวร์ที่ส่งมอบทำได้ครบหรือไม่ แสดงดังตารางที่ 1.1

ตารางที่ 1.1 ข้อกำหนดการควบคุมตามโจทย์ เทียบกับความสามารถของซอฟต์แวร์ที่ส่งมอบ

#	ข้อกำหนดในโจทย์	ตำแหน่งในซอฟต์แวร์ที่ทำตามข้อกำหนด	ผล
1	ใช้ปุ่ม WASD เพื่อเดิน ไม่มีย่อ กระโดด หรือวิ่ง	Assets/Scripts/Player/PlayerMovement.cs มีเฉพาะการเดินและแรงโน้มถ่วง ไม่มีฟังก์ชันกระโดด ย่อ หรือวิ่ง	ครบ
2	ปุ่ม F เพื่อดูวัตถุที่เล็งอยู่ ถ้าไม่ใช่ปริศนาให้ซูมเข้าไปดูชัด ๆ หรือกดเข้าไปแก้ปริศนา	Assets/Scripts/Interaction/InspectStyle.cs:3 ประกาศรูปแบบการดูสามแบบ คือ Hold Turn และ Zoom และ Inspectable เป็นคอมโพเนนต์ที่ระบุว่าวัตถุชิ้นใดใช้แบบใด · PlayerInteractor.cs:130-133 ส่งการกด F ไปยังวัตถุที่เล็งอยู่	ครบ
3	ปุ่ม E เพื่อโต้ตอบกับฉากหรือเก็บของที่เก็บได้	PlayerInteractor.cs:127 ส่งการกด E ไปยัง IInteractable ของวัตถุที่เล็งอยู่ ส่วนข้อความ [E] ... เป็นของวัตถุแต่ละชิ้นเอง เช่น Openable.cs:49	ครบ

#	ข้อกำหนดในโจทย์	ตำแหน่งในซอฟต์แวร์ที่ทำตามข้อกำหนด	ผล
4	ถ้าเป็นของที่เก็บได้ กด E เพื่อเก็บได้ทันที	Assets/Scripts/Interaction/PickupItem.cs:10 ออกข้อความ [E] เก็บ{ชื่อไอเทม}	ครบ
5	ถ้าเป็นของที่ใช้ไอเทมได้ กด E เพื่อเปิดกระเป๋าแล้วเลือกของมาใช้ ถ้าเลือกผิดให้ขึ้นข้อความว่า "ไม่มีอะไรเกิดขึ้น"	Assets/Scripts/UI/InventoryUI.cs:134 ออกข้อความนี้เมื่อ IItemReceiver.Give คืนค่า false	ครบ
6	ปุ่ม Q เพื่อออกจากการดูของหรือเมนูย่อยอื่น ๆ	PlayerModeController.SwitchTo() สลับชุดการกดปุ่มสามชุด คือ Player Inspect และ Inventory	ครบ

[หมายเหตุของร่าง — ลบก่อนส่ง] **รูปที่ 1.3 โจทย์โครงงานที่ได้รับจากพนักงานที่ปรึกษา** — ใช้ไฟล์ EscapeReport/picture/Design.png ได้ทันที
แต่ต้องขออนุญาตบริษัทก่อนนำภาพโจทย์ลงในรายงาน ถ้าบริษัทไม่อนุญาต ให้ตัดรูปนี้ออกแล้วใช้ตารางที่ 1.1 เล่าข้อกำหนดแทน ซึ่งเพียงพอในตัวเอง



รูปที่ 1.3 โจทย์โครงงานที่ได้รับจากพนักงานที่ปรึกษา

1.5 พนักงานที่ปรึกษา และ ตำแหน่งของพนักงานที่ปรึกษา

[หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ส่วนนี้ยังเขียนไม่ได้** สิ่งที่ยืนยันได้มีเพียงว่า **พนักงานที่ปรึกษาเป็นผู้กำหนดโจทย์โครงงานนี้** [จากผู้เขียน] ส่วนชื่อ–นามสกุล ตำแหน่ง และแผนกที่สังกัด **ไม่มีอยู่ในโปรเจกต์** และห้ามเดา เพราะชื่อบุคคลที่ผิดในหน้าอนุมัติจะทำให้เอกสารใช้ไม่ได้
โครงประโยคที่ใช้เติมได้ — "พนักงานที่ปรึกษาคือ [ต้องเติม] ตำแหน่ง [ต้องเติม] สังกัด [ต้องเติม]"
หมายเหตุ ชื่อเดียวกันนี้ต้องปรากฏในหน้าอนุมัติของส่วนหน้าเล่มด้วย ควรเก็บพร้อมกันในครั้งเดียว

1.6 ระยะเวลาที่ปฏิบัติงาน

ระยะเวลาปฏิบัติงานสหกิจศึกษา คือเดือน **กรกฎาคม ถึง ตุลาคม พ.ศ. 2569** [จากผู้เขียน] รวมประมาณสี่เดือน [ต้องเติม — วันที่เริ่มและวันที่สิ้นสุดที่แน่นอน]

ในช่วงเวลาดังกล่าว โครงงานที่รายงานในเอกสารฉบับนี้ใช้เวลาพัฒนา **15 วันปฏิทิน** คือระหว่างวันที่ **25 สิงหาคม ถึง 8 กันยายน พ.ศ. 2569** โดยมีการเขียนหรือแก้ไขไฟล์ในโครงงานจริง **10 วันทำงาน** ตัวเลขทั้งสองได้จากวันเวลาของไฟล์ในระบบไฟล์ โดยไฟล์แรกที่ผู้เขียนสร้างในโครงงานคือ .mcp.json ที่รากของโปรเจกต์ เมื่อวันที่ 25 สิงหาคม เวลา 07:20 น. ไฟล์โปรแกรมไฟล์แรกคือ Assets/Scripts/Player/InputReader.cs เมื่อวันที่ 26 สิงหาคม เวลา 10:30 น. และไฟล์สุดท้ายที่ถูกแก้ไขคือ Assets/Scripts/Editor/PuzzleBuilder.cs เมื่อวันที่ 8 กันยายน เวลา 07:18 น. รายละเอียดของลำดับงานรายวันอยู่ในบทที่ 3 หัวข้อ 3.1

ข้อสังเกตที่ควรบันทึกไว้ตามความเป็นจริง คือ **โจทย์ที่ได้รับระบุระยะเวลาไว้ 1 สัปดาห์ แต่ระยะเวลาที่ใช้จริงตามวันเวลาของไฟล์คือ 15 วันปฏิทิน หรือ 10 วันทำงาน**

> **แก้เมื่อ 8 พฤศจิกายน 2569** ร่างแรกของหัวข้อนี้ระบุว่า 14 วัน คือ 26 สิงหาคม ถึง 8 กันยายน **ตัวเลขที่ถูกคือ 15 วันปฏิทิน เริ่มวันที่ 25 สิงหาคม** วันที่ 25 ตกไปจากการนับครั้งก่อน เพราะไฟล์ของวันนั้นส่วนใหญ่เป็นไฟล์วัสดุที่ Unity เขียนขึ้นตอนนำเข้าแพ็กโมเดลตัวละคร ไม่ใช่ไฟล์โปรแกรม แต่เป็นวันที่โปรเจกต์ถูกตั้งขึ้นจริงจึงต้องนับ ข้อมูลดิบอยู่ที่ evidence/03-timeline-by-day.csv

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อควรระวังเรื่องความน่าเชื่อถือของหลักฐาน** โครงงานนี้**ไม่มีระบบควบคุมเวอร์ชัน** (ตรวจแล้วไม่พบโฟลเดอร์ .git) หลักฐานเรื่องเวลาทั้งหมดจึงมาจากวันเวลาของไฟล์ในระบบไฟล์ ซึ่งอ่อนกว่าประวัติการแก้ไขของระบบควบคุมเวอร์ชัน เพราะไฟล์ที่ถูกคัดลอกหรือย้ายจะได้วันที่ใหม่ **ในรายงานให้เขียนว่า "จากวันเวลาของไฟล์ในระบบ" เสมอ ห้ามเขียนว่า "จากประวัติการแก้ไข"** **สิ่งที่ผู้เขียนต้องตัดสินใจ** ว่าจะอธิบายส่วนต่างระหว่าง 1 สัปดาห์ในโจทย์กับ 10 วันทำงานที่ใช้จริงอย่างไร **การสำรวจเมื่อ 8 พฤศจิกายน 2569 ให้คำตอบที่หลักฐานรองรับไว้แล้ว** คือส่วนที่เป็นตัวเกมตามขอบเขตของโจทย์เสร็จภายใน 25 สิงหาคม ถึง 1 กันยายน ซึ่งเป็น 5 วันทำงาน ตรงตามที่โจทย์กำหนด ส่วนงานในช่วง 2 ถึง 8 กันยายน เป็นงานที่ขยายขอบเขตจากโจทย์โดยผู้เขียนเอง คือการปรับโครงสร้างเป็นโมดูล ชุดทดสอบ และการย้ายข้อมูลออกเป็นไฟล์

> **ยังต้องยืนยันว่าทำเองหรือพี่เลี้ยงอนุมัติให้ขยายเวลา** รายละเอียดอยู่ในข้อ ค ของ คำตอบที่-Claude-คิดเอง.md

1.7 ที่มาและความสำคัญของปัญหา

โจทย์ที่ได้รับมอบหมายกำหนดให้พัฒนาเกม Escape Room Simulator **ทั้งเกม** ไม่ใช่การเพิ่มด่านให้เกมที่มีอยู่เดิม และกำหนดเพิ่มเติมว่าผลงานจะนับเป็น Capstone Project ก็ต่อเมื่อปริศนาที่ใช้เป็นการออกแบบขึ้นเองทั้งหมด
เงื่อนไขสองข้อนี้ทำให้ลักษณะของปัญหาทางวิศวกรรมในโครงงานนี้ต่างจากการเขียนเกมขนาดเล็กที่มีปริศนาคงที่เพียงไม่กี่อย่าง

เกมที่ส่งมอบประกอบด้วยห้องห้าห้องที่เดินต่อกันเป็นเส้นเดียว
แต่ละห้องมีปริศนาหนึ่งองก์ และภายในฉากมีวัตถุที่ผู้เล่นโต้ตอบได้จำนวนมาก ได้แก่ นาฬิกา 46 เรือน ประตูและลิ้นชักที่เปิดได้ กลอนหลายชนิด ไอเทมที่เก็บใส่กระเป๋าได้ แผ่นกระดาษที่อ่านได้ และกลไกเฉพาะของแต่ละองก์ เช่น นาฬิกาทรายจับเวลา กลไกนาฬิกาบนโต๊ะช่าง และเครื่องมือทางดาราศาสตร์ในห้องสุดท้าย

หากเขียนโปรแกรมด้วยวิธีตรงไปตรงมา คือสร้างคลาสหนึ่งคลาสต่อวัตถุหนึ่งชนิด จำนวนคลาสจะเพิ่มขึ้นตามจำนวนชนิดของวัตถุ
และเมื่อใดที่ต้องการให้วัตถุชนิดใหม่ทำสิ่งที่วัตถุเดิมทำได้อยู่แล้ว
จะต้องย้อนกลับไปแก้ไขโปรแกรมที่เขียนไว้แล้ว
ปัญหานี้ทวีความสำคัญขึ้นด้วยเหตุผลสองข้อซึ่งเป็นเงื่อนไขเฉพาะของโครงงานนี้

ข้อแรก ระยะเวลาที่สั้น โจทย์กำหนดระยะเวลาไว้ 1 สัปดาห์ และระยะเวลาที่ใช้จริงคือ 10 วันทำงานใน 15 วันปฏิทิน ภายใต้กรอบเวลาเช่นนี้ ต้นทุนที่แพงที่สุดไม่ใช่การเขียนโปรแกรมชิ้นใหม่
แต่เป็นการต้องย้อนกลับไปแก้โปรแกรมเดิมทุกครั้งที่เพิ่มของใหม่
เพราะการแก้โปรแกรมเดิมมีความเสี่ยงที่จะทำให้สิ่งที่เคยทำงานได้เสียไป และต้องทดสอบซ้ำทั้งหมด

ข้อที่สอง ปริศนาต้องเป็นของที่ออกแบบขึ้นเอง จึงเปลี่ยนแปลงได้ระหว่างการพัฒนา เมื่อปริศนาไม่ได้ถูกกำหนดมาให้ตายตัวตั้งแต่ต้น การออกแบบจึงเกิดขึ้นพร้อมกับการพัฒนา และมีการเปลี่ยนแปลงเกิดขึ้นจริงในโครงงานนี้ โดยเอกสารประกอบที่อยู่ในโปรเจกต์ (CLAUDE.md) บันทึกไว้ว่าปริศนาขององก์ที่สี่เคยใช้นาฬิกาปลุกสิบสองเรือนเป็นคำใบ้
แล้วเปลี่ยนไปใช้ภาพถ่ายเก่าแทน ซึ่งเป็นการเปลี่ยนกลไกของปริศนาทั้งองก์

ร่องรอยของการเปลี่ยนครั้งนี้ยังปรากฏในฉากจริง และตรวจสอบยืนยันได้ คือนาฬิกาปลุกยังอยู่ครบสิบสองเรือนในห้องนอน แต่ **ทั้งสิบสองเรือนถูกตั้งค่าเหมือนกันทุกประการ** คือหน้าปัดหยุดที่ 6:40 (`startHour` 6, `startMinute` 40) สภาพเป็น `Stopped` และเข็มปลุกตั้งไว้ที่ 6:00 พร้อมคันโยกที่ง้างไว้เหมือนกันทุกเรือน จึงไม่มีเรือนใดต่างจากเรือนอื่นและอ่านเป็นคำใบ้ไม่ได้อีกต่อไป (ตรวจจากค่าในฉากที่เปิดอยู่ใน Unity Editor ทั้งสิบสองเรือน เมื่อ 6 พฤศจิกายน 2569)

เมื่อรวมสองข้อเข้าด้วยกัน **ปัญหาที่แท้จริงของโครงงานนี้จึงไม่ใช่ "จะเขียนเกมห้าห้องได้อย่างไร" แต่เป็น "จะออกแบบโครงสร้างซอฟต์แวร์อย่างไร ให้การเพิ่มวัตถุชนิดใหม่หรือการเปลี่ยนปริศนา ไม่บังคับให้ต้องแก้ไขโปรแกรมที่เขียนไว้แล้ว"** ซึ่งตรงกับหลักการออกแบบซอฟต์แวร์เรื่องการแบ่งโมดูล การลดการเกาะเกี่ยวระหว่างส่วน และหลักการเปิด–ปิด ที่อธิบายไว้ในบทที่ 2

นอกจากปัญหาด้านโครงสร้างแล้ว
ยังมีปัญหาด้านความถูกต้องของการทำงานอีกเรื่องหนึ่งซึ่งเป็นผลโดยตรงจากการออกแบบปริศนา
คือปริศนาขององค์ที่สองเป็น **การทดสอบความเที่ยงของนาฬิกาด้วยการจับเวลา**
ผู้เล่นต้องเริ่มเดินกลไกของนาฬิกาพกพร้อมกับพลิกนาฬิกาทราย
แล้วเปรียบเทียบว่าเข็มเดินไปถึงไหนเมื่อทรายหมด
ปริศนาลักษณะนี้บังคับว่าการนับเวลาในเกมต้องให้ผลเท่ากันทุกเครื่องที่นำไปเล่น
มิฉะนั้นคำตอบของปริศนาจะเปลี่ยนไปตามเครื่อง

ในการพัฒนาช่วงแรก การนับเวลาผูกอยู่กับความเร็วในการประมวลผลของเครื่อง `[จากผู้เขียน]` ทำให้เครื่องที่เร็วกว่านับเวลาได้เร็วกว่า และเครื่องที่ช้ากว่านับได้ช้ากว่า ทั้งที่นาฬิกาทรายควรจับเวลาได้สิบวินาทีเท่ากันทุกเครื่อง จึงต้องแก้ไขวิธีนับเวลาใหม่ทั้งระบบ ปัญหานี้เป็นปัญหาหลักของโครงงานตามที่รายงานไว้ในบทที่ 5 หัวข้อ 5.2.1 และวิธีแก้ประกอบด้วยกลไกสี่อย่างที่แยกจากกัน ตามที่อธิบายไว้ในบทที่ 3 หัวข้อ 3.3.8

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อจำกัดของหลักฐาน**
> เนื่องจากโครงงานไม่มีระบบควบคุมเวอร์ชัน จึงเห็นเฉพาะโปรแกรมชุดที่แก้แล้ว
> **ไม่สามารถยืนยันรูปแบบของโปรแกรมชุดเดิมได้**
> อาการของปัญหาเดิมในย่อหน้าข้างต้นจึงเป็นคำบอกเล่าของผู้เขียน ส่วนโปรแกรมปัจจุบันใช้เป็นหลักฐานของ *วิธีแก้* เท่านั้น อย่าเขียนว่าโปรแกรมชุดเดิมเขียนไว้อย่างไร

1.8 วัตถุประสงค์หรือจุดมุ่งหมายของโครงงาน

1. เพื่อพัฒนาเกมแนว Escape Room Simulator มุมมองบุคคลที่หนึ่งให้สมบูรณ์ทั้งเกมตามข้อกำหนดการควบคุมที่ได้รับมอบหมายครบทุกข้อ
2. เพื่อออกแบบโครงสร้างซอฟต์แวร์แบบประกอบคอมโพเนนต์ (Component Composition) ที่ทำให้การเพิ่มวัตถุชนิดใหม่เข้าไปในเกมทำได้โดยไม่ต้องแก้ไขโปรแกรมที่เขียนไว้แล้ว
3. เพื่อออกแบบปริศนาขึ้นเองทั้งห้าองก์ให้ใช้กลไกร่วมกันได้โดยไม่ต้องเขียนโปรแกรมเฉพาะกิจแยกสำหรับแต่ละองก์
4. เพื่อให้การนับเวลาภายในเกมให้ผลลัพธ์เท่ากันทุกเครื่องที่นำไปเล่นโดยไม่ขึ้นกับความเร็วในการประมวลผลของเครื่องนั้น

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อนี้ต้องให้ผู้เขียนยืนยันก่อนส่ง** วัตถุประสงค์ที่ตั้งไว้ *ตอนเริ่มโครงงาน* เป็นสิ่งที่อ่านจากไฟล์ในโปรเจกต์ไม่ได้ รายการสี่ข้อข้างต้นเป็นการ**เรียบเรียงย้อนกลับจากข้อกำหนดในโจทย์ และจากสิ่งที่ซอฟต์แวร์ทำได้จริง** ไม่ใช่สิ่งที่ผู้เขียนเขียนไว้ก่อนลงมือ
>
> - ข้อ 1 มาจากข้อกำหนดในโจทย์โดยตรง (ตารางที่ 1.1) — มั่นใจสูง
> - ข้อ 2 และข้อ 3 มาจากเงื่อนไข original puzzle design ในโจทย์ ประกอบกับโครงสร้างที่พบในซอฟต์แวร์จริง (บทที่ 2 และ 4) — มั่นใจปานกลาง **ถ้าตอนเริ่มยังไม่ได้ตั้งใจเรื่องโครงสร้างไว้ล่วงหน้า ให้ย้ายสองข้อนี้ไปเป็นผลที่ได้รับในบทที่ 5 แทน และอย่าเขียนไว้ที่นี่**
> - ข้อ 4 มาจากคำบอกเล่าของผู้เขียนว่าพบปัญหาแล้วจึงสั่งให้แก้ — **ถ้าปัญหานี้พบระหว่างทาง ไม่ได้ตั้งเป็นวัตถุประสงค์ไว้ตั้งแต่ต้น ให้ตัดออกจากหัวข้อนี้** แล้วคงไว้เฉพาะในหัวข้อ 1.7 และบทที่ 5 ซึ่งเป็นที่ของปัญหาที่พบระหว่างดำเนินงาน
>
> แบบฟอร์มของคณะวางช่องไว้สามข้อเป็นตัวอย่าง ไม่ได้บังคับจำนวน ถ้าต้องการให้เหลือสามข้อพอดี ให้ตัดข้อที่ผู้เขียนไม่ได้ตั้งใจไว้ตั้งแต่ต้นออก

1.9 ผลที่คาดว่าจะได้รับจากการปฏิบัติงานหรือโครงงานที่ได้รับมอบหมาย

1. ได้ซอฟต์แวร์เกม Escape Room Simulator ที่เล่นได้จบตั้งแต่ห้องแรกถึงห้องสุดท้าย พร้อมปริศนาที่ออกแบบขึ้นเองห้าองก์ ซึ่งเป็นเงื่อนไขที่โจทย์กำหนดไว้สำหรับการนับเป็น Capstone Project

2. ได้โครงสร้างซอฟต์แวร์ที่แยกส่วนตามหน้าที่ และนำส่วนประกอบกลับมาใช้ซ้ำข้ามปริศนาได้ ซึ่งทำให้เพิ่มวัตถุหรือกลไกใหม่ได้โดยแก้โปรแกรมเดิมน้อยที่สุด

3. ได้ความเข้าใจและประสบการณ์ในการนำหลักการออกแบบซอฟต์แวร์เชิงโมดูล การลดการเกาะเกี่ยวระหว่างส่วน และการทดสอบระดับสถาปัตยกรรม ไปใช้กับงานพัฒนาเกมจริงภายใต้ข้อจำกัดด้านเวลา

4. ได้แนวทางปฏิบัติสำหรับการนับเวลาในเกมที่ให้ผลเท่ากันทุกเครื่อง ซึ่งนำไปใช้กับงานอื่นที่ต้องการความถูกต้องของเวลาได้

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ต้องให้ผู้เขียนยืนยันเช่นเดียวกับหัวข้อ 1.8** และต้องให้แต่ละข้อสอดคล้องกับวัตถุประสงค์ในหัวข้อ 1.8 ตามลำดับ ถ้าตัดวัตถุประสงค์ข้อใดออก ต้องตัดผลที่คาดว่าจะได้รับข้อเดียวกันออกด้วย

1.10 นิยามศัพท์เฉพาะ

1. **เกมห้องปิดตาย (Escape Room Simulator)** — เกมที่ผู้เล่นอยู่ในพื้นที่ปิด ต้องสำรวจสิ่งของในฉาก เก็บเบาะแส และแก้ปริศนาตามลำดับเพื่อเปิดทางออกไปยังพื้นที่ถัดไป ในโครงงานนี้มีห้าห้องต่อกันเป็นเส้นเดียว ห้องละหนึ่งปริศนา

2. **องก์ (Act)** — คำที่ใช้ในรายงานฉบับนี้เรียกหน่วยของปริศนาหนึ่งชุดที่อยู่ในห้องหนึ่งห้อง โครงงานนี้มีห้าองก์ เรียงตามลำดับที่ผู้เล่นต้องแก้

3. **คอมโพเนนต์ (Component)** — หน่วยของโปรแกรมที่นำไปติดกับวัตถุในฉากได้ ทำหน้าที่อย่างเดียว และทำงานร่วมกับคอมโพเนนต์อื่นที่ติดอยู่บนวัตถุเดียวกันได้ ในเอนจิน Unity คือคลาสที่สืบทอดจาก `MonoBehaviour` โครงงานนี้มีคอมโพเนนต์ชนิดนี้ 51 คลาส

4. **การประกอบแทนการสืบทอด (Composition over Inheritance)** — แนวทางที่สร้างพฤติกรรมของวัตถุจากการนำคอมโพเนนต์หลายตัวมาประกอบกัน แทนการสร้างคลาสลูกที่สืบทอดจากคลาสแม่ โครงงานนี้ไม่ใช้การสืบทอดเลยแม้แต่จุดเดียว

5. **อินเทอร์เฟซ (Interface)** — ข้อตกลงที่ระบุว่าคลาสที่นำไปใช้ต้องมีความสามารถอะไรบ้าง โดยไม่ระบุว่าทำอย่างไร

ทำให้โปรแกรมเรียกใช้ความสามารถนั้นได้โดยไม่ต้องรู้จักคลาสที่ทำงานจริง
โครงงานนี้มีอินเทอร์เฟซ 12 ชนิด

6. **การเกาะเกี่ยว (Coupling)** — ระดับความขึ้นต่อกันระหว่างโมดูล ยิ่งต่ำยิ่งแก้ไขหรือแทนที่โมดูลหนึ่งได้โดยไม่กระทบโมดูลอื่น

7. **การเกาะกลุ่ม (Cohesion)** — ระดับความเกี่ยวเนื่องกันของหน้าที่ภายในโมดูลเดียวกัน ยิ่งสูงยิ่งเข้าใจและทดสอบได้ง่าย

8. **แอสเซมบลีดีฟินิชัน (Assembly Definition)** — ไฟล์ตั้งค่าของ Unity ที่กำหนดให้โฟลเดอร์หนึ่งถูกคอมไพล์เป็นแอสเซมบลีแยก
และระบุได้ว่าแอสเซมบลีนั้นอ้างถึงแอสเซมบลีใดได้บ้าง
ทำให้ขอบเขตระหว่างโมดูลเป็นข้อบังคับระดับการคอมไพล์ โครงงานนี้มีหกไฟล์

9. **พรีแฟบ (Prefab)** — แม่แบบของวัตถุในฉากที่บันทึกไว้เป็นไฟล์ เมื่อนำไปวางในฉากจะได้สำเนาที่ยังเชื่อมกับแม่แบบ การแก้แม่แบบจึงมีผลกับทุกสำเนา โครงงานนี้มี 33 พรีแฟบ

10. **ฉาก (Scene)** — ไฟล์ที่เก็บวัตถุทั้งหมดในพื้นที่เล่นหนึ่งพื้นที่ พร้อมตำแหน่งและค่าที่ตั้งไว้ โครงงานนี้ใช้ฉากเดียวคือ `Assets/Scenes/Main.unity`

11. **การเขียนทับค่าของพรีแฟบ (Prefab Override)** — การแก้ค่าของสำเนาในฉากให้ต่างจากแม่แบบ โดยฉากจะบันทึกไว้เฉพาะค่าที่ต่างเท่านั้น ค่าที่โปรแกรมใช้จริงจึงเป็นผลรวมของค่าในพรีแฟบกับค่าที่ฉากเขียนทับ

12. **สคริปต์เทเบิลอ็อบเจกต์ (ScriptableObject)** — ชนิดข้อมูลของ Unity ที่เก็บค่าไว้เป็นไฟล์แอสเซ็ตแยกจากฉาก ใช้เก็บข้อมูลที่ต้องแก้ได้โดยไม่ต้องแก้โปรแกรม โครงงานนี้มี 12 ไฟล์

13. **การสร้างเนื้อหาด้วยโปรแกรม (Procedural Content Generation)** — การสร้างแอสเซ็ตในเกม เช่น รูปทรง พื้นผิว และการจัดวาง ด้วยโปรแกรมแทนการปั้นด้วยมือในซอฟต์แวร์สามมิติ
โครงงานนี้ใช้วิธีนี้กับแอสเซ็ตเกือบทั้งหมด

14. **ความเป็นอิสระจากอัตราเฟรม (Frame-rate Independence)** — คุณสมบัติที่ทำให้ผลลัพธ์ของการนับเวลาในเกมเท่ากันทุกเครื่อง
แม้เครื่องแต่ละเครื่องจะวาดภาพได้จำนวนเฟรมต่อวินาทีไม่เท่ากัน
ทำได้โดยคำนวณจากเวลาจริงที่ผ่านไปในแต่ละเฟรม ไม่ใช่จากจำนวนเฟรม

15. **ไปป์ไลน์การเรนเดอร์สากล (Universal Render Pipeline, URP)** — ไปป์ไลน์การประมวลผลภาพของ Unity ที่โครงงานนี้ใช้ วัสดุที่สร้างสำหรับไปป์ไลน์เดิมจะใช้กับ URP ไม่ได้โดยตรง

16. **การทดสอบระดับสถาปัตยกรรม (Architecture Fitness Test)** — การทดสอบที่ยืนยัน *คุณสมบัติของโครงสร้างซอฟต์แวร์* เช่น การที่คลาสซึ่งเขียนขึ้นใหม่ต้องทำงานร่วมกับระบบเดิมได้โดยไม่ต้องแก้ระบบเดิม ต่างจากการทดสอบทั่วไปที่ยืนยันค่าที่ฟังก์ชันคืนมา โครงงานนี้มีการทดสอบลักษณะนี้ 15 กรณี

[หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **หมายเหตุสำหรับหัวข้อนี้** แบบฟอร์มของคณะวางช่องไว้สามข้อ แต่ไม่ได้จำกัดจำนวน รายการข้างต้นมี 16 ข้อ ซึ่งยาวกว่าตัวอย่าง ถ้าอาจารย์ต้องการให้สั้นลง **ให้ตัดข้อที่ไม่ได้ถูกอ้างถึงซ้ำในบทที่ 2 ถึง 5 ออกก่อน** คือข้อ 11, 15 และ 16 ตามลำดับ ส่วนข้อ 3 ถึง 8 ตัดไม่ได้ เพราะเป็นศัพท์ที่ทั้งเล่มใช้ตลอด

คำนิยามในหัวข้อนี้เขียนแบบสั้นโดยเจตนา และไม่มีเลขอ้างอิง ส่วนคำอธิบายฉบับเต็มพร้อมเอกสารอ้างอิงอยู่ในบทที่ 2 ตามบทบาทของแต่ละบท คือบทที่ 1 นิยามศัพท์ให้ผู้อ่านตามเรื่องได้ ส่วนบทที่ 2 อธิบายทฤษฎีพร้อมอ้างอิง

บทที่ 2
ทฤษฎีที่เกี่ยวข้อง

ในการปฏิบัติงานสหกิจศึกษาครั้งนี้ เป็นการนำความรู้ทางด้านทฤษฎีวิศวกรรมซอฟต์แวร์และเทคโนโลยีการพัฒนาเกมมาใช้ในการปฏิบัติงานตลอดการดำเนินโครงงาน ซึ่งเป็นทั้งการนำความรู้ที่เคยเรียนมาประยุกต์ใช้และการศึกษาเรียนรู้สิ่งใหม่ ๆ ที่ได้จากการปฏิบัติงานจริง เนื่องจากโครงงานนี้มีลักษณะเป็นระบบซอฟต์แวร์ที่ประกอบด้วยส่วนย่อยจำนวนมากทำงานร่วมกัน ทฤษฎีที่นำมาใช้จึงแบ่งได้เป็นสี่กลุ่ม คือ ทฤษฎีการออกแบบซอฟต์แวร์เชิงโมดูล หลักการออกแบบเชิงวัตถุ ทฤษฎีความถูกต้องของการนับเวลาในเกม และทฤษฎีการทดสอบซอฟต์แวร์ ร่วมกับเทคโนโลยีและเครื่องมือที่ใช้ในการพัฒนา โดยในแต่ละหัวข้อจะระบุตำแหน่งในซอร์สโค้ดของโครงงานที่ทฤษฎีนั้นถูกนำไปใช้จริงกำกับไว้เพื่อให้สามารถตรวจสอบย้อนกลับได้

2.1 ทฤษฎีการออกแบบซอฟต์แวร์เชิงโมดูล

2.1.1 การแยกระบบออกเป็นโมดูลและการซ่อนข้อมูล

การแยกระบบออกเป็นโมดูล (Modular Decomposition) คือการแบ่งซอฟต์แวร์ขนาดใหญ่ออกเป็นหน่วยย่อยที่พัฒนา ทดสอบ และแก้ไขได้โดยอิสระจากกัน Parnas เสนอไว้ว่าเกณฑ์ที่ควรใช้ในการแบ่งโมดูลไม่ใช่การแบ่งตามลำดับขั้นตอนการทำงานของโปรแกรม แต่ควรแบ่งตาม **สิ่งที่แต่ละโมดูลต้องปิดซ่อนไว้ไม่ให้โมดูลอื่นรู้** (Information Hiding) เพราะการเปลี่ยนแปลงในอนาคตมักเกิดขึ้นกับรายละเอียดภายใน ถ้ารายละเอียดนั้นถูกซ่อนไว้ การเปลี่ยนแปลงก็จะไม่ลุกลามออกไปนอกโมดูล [1]

ในโครงงานนี้ ซอร์สโค้ดถูกแบ่งออกเป็นโฟลเดอร์ตาม **โดเมนของระบบ** ไม่ใช่ตามชนิดของอ็อบเจกต์ในเอนจิน รวม 9 กลุ่ม ดังตารางที่ 2.1

ตารางที่ 2.1 โมดูลของโครงงาน จำนวนไฟล์ และขอบเขตระดับการคอมไพล์

#	โมดูล	โฟลเดอร์	Assembly	ไฟล์	บรรทัด
1	Items	Assets/Scripts/Items/	EscapeRoom.Items	2	37
2	Interaction	Assets/Scripts/Interaction/	EscapeRoom.Interaction	11	373

#	โมดูล	โฟลเดอร์	Assembly	ไฟล์	บรรทัด
3	Clock	Assets/Scripts/Clock/	EscapeRoom.Clock	23	1,334
4	Puzzle	Assets/Scripts/Puzzle/	EscapeRoom.Puzzle	8	384
5	World	Assets/Scripts/World/	EscapeRoom.World	3	162
6	Player	Assets/Scripts/Player/	*Assembly-CSharp*	10	830
7	UI	Assets/Scripts/UI/	*Assembly-CSharp*	3	216
8	Game	Assets/Scripts/Game/	*Assembly-CSharp*	3	145
9	Editor tooling	Assets/Scripts/Editor/	*Assembly-CSharp-Editor*	11	5,772
—	ชุดทดสอบ	Assets/Tests/PlayMode/	EscapeRoom.Tests	3	393

สิ่งที่ทำให้การแบ่งนี้เป็นการแบ่งโมดูลจริงตามความหมายของ Parnas ไม่ใช่เพียงการจัดระเบียบโฟลเดอร์ คือ **5 โมดูลแรกมีไฟล์ Assembly Definition (.asmdef) กำกับ** (และชุดทดสอบมีอีกหนึ่งไฟล์ รวมหกไฟล์) ซึ่งทำให้ขอบเขตระหว่างโมดูลกลายเป็นข้อบังคับระดับคอมไพเลอร์ ไม่ใช่เพียงข้อตกลงในการตั้งชื่อ รายละเอียดของกลไกนี้อยู่ในหัวข้อ 2.5.7

ข้อสังเกตที่ควรบันทึก คือโครงงานนี้ **ไม่ใช้ namespace เลยแม้แต่ที่เดียว** ทุกคลาสอยู่ใน global namespace และทุกไฟล์ .asmdef ตั้งค่า "rootNamespace": "" การแบ่งโมดูลที่นี่จึงใช้ .asmdef เป็นเครื่องมือแทน namespace ซึ่งเข้มงวดกว่า เพราะ namespace เพียงจัดกลุ่มชื่อและข้ามได้ด้วยคำสั่ง using ขณะที่ .asmdef ปิดกั้นการอ้างถึงในระดับการคอมไพล์ ต้นทุนที่ตามมาคือหากนำแอสเซ็ตจากภายนอกที่มีคลาสชื่อซ้ำเข้ามา โปรเจกต์จะคอมไพล์ไม่ผ่านทันที

2.1.2 การเกาะเกี่ยว (Coupling) และการเกาะกลุ่ม (Cohesion)

Coupling และ Cohesion เป็นตัวชี้วัดคุณภาพของการแบ่งโมดูลที่ Stevens, Myers และ Constantine เสนอไว้ตั้งแต่ปี ค.ศ. 1974 [2] โดย

- **Coupling** คือระดับความขึ้นต่อกันระหว่างโมดูล ยิ่งต่ำยิ่งดี เพราะโมดูลที่ขึ้นต่อกันน้อยสามารถแก้ไขหรือแทนที่ได้โดยไม่กระทบโมดูลอื่น
- **Cohesion** คือระดับความเกี่ยวเนื่องกันของหน้าที่ภายในโมดูลเดียว ยิ่งสูงยิ่งดี เพราะโมดูลที่ทำหน้าที่เดียวย่อมเข้าใจและทดสอบได้ง่ายกว่าโมดูลที่ทำหลายหน้าที่ปนกัน

เป้าหมายของการออกแบบจึงมักสรุปสั้น ๆ ว่า **Low Coupling, High Cohesion** และในมาตรฐาน ISO/IEC 25010 ความสามารถเช่นนี้จัดอยู่ในคุณลักษณะคุณภาพด้าน Maintainability โดยมี Modularity เป็นคุณลักษณะย่อย [4] อาการของ Coupling ที่สูงเกินไปที่ Fowler เรียกว่า code smell ได้แก่การที่การแก้ไขหนึ่งจุดบังคับให้ต้องแก้ไขอีกหลายจุดตามไปด้วย (Shotgun Surgery) และการที่คลาสหนึ่งต้องรู้รายละเอียดภายในของอีกคลาสหนึ่งมากเกินจำเป็น [3]

ตำแหน่งที่พบในโครงงาน — วัดโดยลบคอมเมนต์ออกจากทุกบรรทัดก่อน แล้วค้นหาชื่อชนิดข้อมูลที่ประกาศในโมดูลอื่น ผลที่ได้มีทั้งด้านที่ดีและด้านที่ยังเป็นข้อจำกัด

ตารางที่ 2.2 ทิศทางการพึ่งพาระหว่างโมดูลที่วัดได้จากซอร์สโค้ด

จากโมดูล	ไปโมดูล	ตัวอย่างหลักฐาน
Items	*(ไม่พึ่งโมดูลใด)*	—
World	*(ไม่พึ่งโมดูลใด)*	—
Interaction	Items	IActor.cs:25, KeyLock.cs:11, PickupItem.cs:8
Clock	Items, Interaction	ClockPartSocket.cs:13, ChimeRelease.cs:20
Puzzle	Interaction	CodeLock.cs:12,13,29,37
Player	Items, Interaction, UI, **Clock**	PlayerInteractor.cs:12,31, HeldSandglass.cs:12
UI	Items, Interaction, **Player**	InventoryUI.cs:18,19,20
Game	Interaction, Player	GameManager.cs:15,17

ด้านที่เป็นไปตามทฤษฎี คือ **โมดูล Clock และ Puzzle ไม่อ้างถึงชนิดข้อมูลใดในโมดูล Player หรือ UI แม้แต่ครั้งเดียว** (ยืนยันโดยการค้นหาคำว่า PlayerInteractor, PlayerHud, PlayerMode, InspectView, InputReader, InventoryUI ในโฟลเดอร์ Clock/ ได้ผลลัพธ์ศูนย์รายการ) และโมดูล Items กับ World ไม่พึ่งโมดูลใดเลย ทำให้ย้ายไปใช้ในโปรเจกต์อื่นได้ทันที

ด้านที่ยังเป็นข้อจำกัด มีสามเรื่องที่ต้องระบุตามจริง

1. **มีวงจรการพึ่งพาระหว่าง Player และ UI** คือ Player อ้าง UI (PlayerInteractor.cs:31-33) และ UI อ้าง Player กลับ (InventoryUI.cs:18-19) วงจรนี้เป็นสาเหตุทางเทคนิคที่ทำให้สองโมดูลนี้ใส่ .asmdef ไม่ได้ (ดูหัวข้อ 2.5.7)
2. **มีการพึ่งพาข้ามชั้นหนึ่งจุด** คือ Player ⟶ Clock ที่ HeldSandglass.cs:12 ซึ่งถือ [SerializeField] private Sandglass glass; เป็นการอ้างถึงคลาสรูปธรรมในโมดูล Clock โดยตรง
3. **มีการอ้างถึงคลาสรูปธรรมผ่าน [SerializeField] รวม 215 จุดใน 53 ไฟล์** การพึ่งพาเหล่านี้ไม่ปรากฏในซอร์สโค้ดว่าปลายทางคืออ็อบเจกต์ใด ต้องเปิดดูหน้าต่าง Inspector ของ Unity จึงจะทราบ

ด้าน Cohesion โค้ดส่วนรันไทม์ 63 ไฟล์มีความยาวเฉลี่ยประมาณ 55 บรรทัดต่อไฟล์ ซึ่งสะท้อนว่าแต่ละคลาสรับผิดชอบหน้าที่แคบ ตัวอย่างคลาสที่ Cohesion สูงชัดเจน เช่น ClockDial.cs (32 บรรทัด ทำหน้าที่แปลงจำนวนนาทีเป็นมุมของเข็มอย่างเดียว) และ Pendulum.cs (61 บรรทัด) ส่วนคลาสที่ Cohesion ต่ำที่สุดคือ InspectView.cs ซึ่งยาว 317 บรรทัด (นับบรรทัดที่ไม่ว่าง) และรับผิดชอบงานที่นับได้ 8 อย่างในคลาสเดียว รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📸 **คำแนะนำการแคปภาพ — ลบออกก่อนส่ง รูปที่ 2.1**
> แผนภาพทิศทางการพึ่งพาระหว่างโมดูลทั้ง 9 กลุ่ม — วาดใหม่ด้วย draw.io หรือ PowerPoint จากตารางที่ 2.2 ให้ลูกศรชี้ทิศทางเดียว และเน้นวงจร Player ⇄ UI ด้วยสีต่างจากเส้นอื่น คำบรรยาย: รูปที่ 2.1 ทิศทางการพึ่งพาระหว่างโมดูลของโครงงาน

[วางรูปที่ 2.1 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-2-01.png

รูปที่ 2.1 ทิศทางการพึ่งพาระหว่างโมดูลของโครงงาน

2.1.3 สถาปัตยกรรมเชิงองค์ประกอบ (Component-Based Architecture)

สถาปัตยกรรมเชิงองค์ประกอบ หรือที่ในวงการพัฒนาเกมเรียกว่า Component Pattern คือการนิยามว่าอ็อบเจกต์หนึ่งในเกม "เป็นอะไร" ด้วย **ชุดของส่วนประกอบที่ถูกนำมาแปะไว้บนอ็อบเจกต์นั้น** แทนการนิยามด้วยคลาสที่สืบทอดกันเป็นลำดับชั้น Nystrom อธิบายว่าแบบแผนนี้เกิดขึ้นเพื่อแก้ปัญหาที่พบบ่อยในเกม คือเมื่อใช้การสืบทอดอ็อบเจกต์ที่ต้องการคุณสมบัติจากหลายสายพันธุ์พร้อมกันจะไม่มีที่ลงในลำดับชั้นเดียว และสุดท้ายโค้ดจะถูกยกขึ้นไปรวมกันในคลาสฐานจนคลาสฐานบวมผิดรูป [5] Gregory อธิบายโครงสร้างนี้ในบริบทของเอนจินเกมเชิงพาณิชย์ว่าเป็นโมเดลที่เอนจินยุคปัจจุบันส่วนใหญ่เลือกใช้ [6]

Unity ใช้โมเดลนี้เป็นสถาปัตยกรรมพื้นฐานของเอนจิน คือ `GameObject` เป็นเพียงภาชนะที่ไม่มีพฤติกรรมของตัวเอง พฤติกรรมทั้งหมดมาจาก Component ที่นำมาแปะ และสคริปต์ที่ผู้พัฒนาเขียนขึ้นจะกลายเป็น Component ได้ด้วยการสืบทอดจากคลาส `MonoBehaviour` [21]

ตำแหน่งที่พบในโครงงาน ตัวอย่างที่ชัดที่สุดคือระบบนาฬิกา ซึ่งเป็นระบบที่ใหญ่ที่สุดในโครงงาน (23 ไฟล์ 1,334 บรรทัด) นาฬิกาทั้ง 46 เรือนในฉากไม่ได้ถูกเขียนเป็นคลาสละเรือน แต่ประกอบขึ้นจากส่วนประกอบที่ทำหน้าที่ละอย่าง ได้แก่ `ClockMovement` (เก็บเวลา), `ClockDial` (แสดงเวลาด้วยเข็ม), `Mainspring` (แหล่งพลังงานที่ไขลานได้), `Pendulum` (ลูกตุ้มที่ถ้าถูกหยุดก็หยุดนาฬิกา), `ClockWinder` (ก้านไขลาน) และ `WeightChain` (โซ่ตุ้มถ่วง) ชนิดของนาฬิกาจึงเป็น **ผลรวมของส่วนประกอบที่แปะอยู่** ไม่ใช่คลาสลูกที่สืบทอดมา เช่น นาฬิกาที่ไม่มี `Mainspring` คือนาฬิกาที่ไม่มีวันหมดพลัง ซึ่งถูกต้องสำหรับนาฬิกาที่ใช้ตุ้มถ่วงหรือไฟบ้าน

ระบบสิ่งที่เปิดได้ใช้แนวทางเดียวกัน คือ `Openable` ถือสถานะเปิดหรือปิดและไม่ขยับอะไรเลย ส่วนการเคลื่อนไหวแยกเป็น `SwingMotion` (บานพับ) และ `SlideMotion` (เลื่อนตรง) ดังนั้นประตู ลิ้นชัก ฝาตู้โชว์ และผนังลับจึงใช้ `Openable` ตัวเดียวกันทั้งหมด

2.1.4 การประกอบแทนการสืบทอด (Composition over Inheritance)

หลักการ Composition over Inheritance ระบุว่าควรสร้างพฤติกรรมใหม่โดยการ **ประกอบ** อ็อบเจกต์ที่มีอยู่เข้าด้วยกัน แทนการ **สืบทอด** จากคลาสฐาน เหตุผลหลักคือการสืบทอดเป็นการผูกชนิดข้อมูลไว้ตั้งแต่ตอนคอมไพล์และสร้างความขึ้นต่อกันที่แน่น

ที่สุดแบบหนึ่งระหว่างคลาสลูกกับคลาสแม่
ขณะที่การประกอบเปลี่ยนได้ตอนรันไทม์และกระทบเฉพาะจุดที่ประกอบ [7] Gamma และคณะสรุปหลักนี้ไว้เป็นหนึ่งในสองหลักพื้นฐานของหนังสือ Design Patterns ว่า *"favor object composition over class inheritance"* [7]

ตำแหน่งที่พบในโครงงาน

โครงงานนี้ปฏิบัติตามหลักนี้อย่างเคร่งครัดในระดับที่วัดได้เป็นตัวเลข ดังตารางที่ 2.3

ตารางที่ 2.3 ตัวชี้วัดที่ยืนยันการใช้ Composition แทน Inheritance

ตัวชี้วัด	ค่าที่วัดได้	วิธีวัด
คำสั่ง abstract	**0**	ค้นหาคำว่า abstract ในสคริปต์รันไทม์ 63 ไฟล์
คำสั่ง virtual	**0**	ค้นหาคำว่า virtual
คำสั่ง override	**0**	ค้นหาคำว่า override
คลาสที่สืบทอดจาก MonoBehaviour โดยตรง	**51 คลาส**	ค้นหารูปแบบ : MonoBehaviour
Interface ที่ประกาศขึ้นเอง	**12 interface ใน 6 ไฟล์**	ค้นหารูปแบบ public interface
[SerializeField]	**215 ครั้ง ใน 53 ไฟล์**	ค้นหาคำว่า [SerializeField แล้วนับทุกครั้งที่พบ ไม่ใช่นับจำนวนบรรทัด
รูปแบบ Singleton (static … Instance)	**0**	ค้นหารูปแบบ static <ชนิด> Instance
FindObjectOfType / FindAnyObjectByType	**0**	ค้นหาชื่อเมธอด
GameObject.Find	**0**	ค้นหาชื่อเมธอด
SendMessage	**0**	ค้นหาชื่อเมธอด
UnityEvent	**0**	ค้นหาชื่อชนิด
DontDestroyOnLoad	**0**	ค้นหาชื่อเมธอด
StartCoroutine	**0**	ค้นหาชื่อเมธอด

ความหมายเชิงวิศวกรรมของตารางนี้อยู่ที่ค่าที่เป็นศูนย์มากกว่าค่าที่ไม่เป็นศูนย์ คือ **ไม่มีลำดับชั้นการสืบทอดใดในโครงงานเลย** ทั้ง 51 คลาสสืบทอดจาก MonoBehaviour ตรง ๆ ในระดับเดียวกันทั้งหมด และไม่มีการใช้กลไกที่ผูกอ็อบเจกต์เข้าหากันแบบทั่วถึง (global) อย่าง

Singleton, FindObjectOfType หรือ SendMessage ซึ่งเป็นแบบแผนที่ทำให้ Coupling สูงโดยที่คอมไพเลอร์ตรวจไม่พบ

กลไกที่โครงงานใช้แทนในการประกาศความขึ้นต่อกันระหว่างส่วนประกอบ คือแอตทริบิวต์ [RequireComponent] ซึ่งพบใน 15 คลาส และ [DisallowMultipleComponent] ซึ่งพบในเกือบทุก MonoBehaviour แอตทริบิวต์แรกทำให้ Unity เพิ่มส่วนประกอบที่จำเป็นให้อัตโนมัติและห้ามลบออก จึงเป็นการประกาศความขึ้นต่อกันที่เอนจินบังคับให้เป็นจริง และช่วยลดปัญหา "ลืมแปะส่วนประกอบ" ซึ่งเป็นผลข้างเคียงหลักของการใช้ Composition

2.2 หลักการออกแบบเชิงวัตถุที่ใช้ในโครงงาน

2.2.1 หลักการผกผันการพึ่งพา (Dependency Inversion Principle)

Dependency Inversion Principle (DIP) เป็นหนึ่งในหลัก SOLID ที่ Martin เสนอ มีใจความสองข้อ คือ (1) โมดูลระดับสูงต้องไม่ขึ้นต่อโมดูลระดับล่าง ทั้งสองควรขึ้นต่อสิ่งที่เป็นนามธรรม และ (2) สิ่งที่เป็นนามธรรมต้องไม่ขึ้นต่อรายละเอียด รายละเอียดต้องขึ้นต่อสิ่งที่เป็นนามธรรม [8] ผลที่ได้คือทิศทางการพึ่งพาถูก "ผกผัน" จากที่โค้ดไหลไปตามลำดับการเรียกใช้ กลายเป็นว่าทั้งผู้เรียกและผู้ถูกเรียกชี้เข้าหา interface ตัวกลางเดียวกัน Martin เรียกเส้นที่ interface นั้นสร้างขึ้นว่าเป็น **เส้นแบ่งเขตของสถาปัตยกรรม** (architectural boundary) [9]

ตำแหน่งที่พบในโครงงาน โครงงานนี้ใช้ DIP ผ่านรูปแบบที่เรียกได้ว่า *interface-based discovery* คือตัวควบคุมไม่ถือการอ้างอิงไปยังผู้ร่วมงานของตน แต่ค้นหาเอาเองตอนเริ่มทำงานด้วยเมธอดตระกูล GetComponents<T>() โดยระบุเป็น interface ไม่ใช่คลาสรูปธรรม จุดที่ใช้รูปแบบนี้มี 4 จุด เรียกว่า *seam* ของสถาปัตยกรรม ได้แก่ IClockDisplay, IClockPart, ILock และ IPowerStore

ตัวอย่างที่ชัดที่สุดคือ ClockMovement ซึ่งเป็นคลาสที่เก็บเวลาของนาฬิกา

```
// Assets/Scripts/Clock/ClockMovement.cs:112-116
_displays = GetComponentsInChildren<IClockDisplay>(true);
// Inactive ones included: a part that is switched off is one the
player has not
// fitted yet, and it has to be in the list so it can start
saying no.
_parts = GetComponentsInChildren<IClockPart>(true);
```

คลาสนี้ **ไม่มีฟิลด์ชื่อ dial** และไม่รู้จักคลาส ClockDial เลย สิ่งที่มันรู้คือมีอะไรบางอย่างที่ implement IClockDisplay อยู่ภายใต้ตัวมัน และมันจะส่งจำนวนนาทีให้ทุกตัวในรายการนั้น ผลคือ ClockMovement (ระดับสูง) ไม่ขึ้นต่อ ClockDial (ระดับล่าง) แต่ทั้งสองขึ้นต่อ IClockDisplay ตรงตามใจความข้อแรกของ DIP

รูปแบบเดียวกันพบที่ Openable ซึ่งถามกลอนทุกตัวที่แปะอยู่กับตัวมัน

```
// Assets/Scripts/Interaction/Openable.cs:20-35
private ILock[] Locks => _locks ?? (_locks =
GetComponents<ILock>());
private ILock BlockingLock { get { foreach (ILock fitted in
Locks) if (fitted.IsLocked) return fitted; return null; } }
```

และที่ IPowerStore ซึ่งถูกค้นหาโดยคลาสสามคลาสที่ไม่รู้จักกัน ด้วยบรรทัดที่เหมือนกันทั้งสามที่

```
// ClockWinder.cs:59 · WeightChain.cs:36 · ClockStriker.cs:73
_power = movement.GetComponentInChildren<IPowerStore>(true);
```

ข้อสำคัญคือ interface เหล่านี้มีผู้ implement มากกว่าหนึ่งรายจริง ซึ่งเป็นสิ่งที่แยกการใช้ interface อย่างมีประโยชน์ออกจากการใช้ interface เกินความจำเป็น ดังตารางที่ 2.4

ตารางที่ 2.4 Interface ที่ประกาศในโครงงานและจำนวนผู้ implement

Interface	โมดูล	จำนวน	รายชื่อผู้ implement
IInteractable	Interaction	9	Openable, PickupItem, ClockWinder, ClockSetter, WeightChain, ClockPartSocket, TestRun, CodeWheel, Prism
ILock	Interaction	6	KeyLock, SealedLock, TimeLock, RunningLock, CodeLock, StarLock
IClockPart	Clock	3	Mainspring, Pendulum, TestRun

Interface	โมดูล	จำนวน	รายชื่อผู้ implement
IItemReceiver	Interaction	2	KeyLock, ClockPartSocket
IClockDisplay	Clock	2	ClockDial, DigitalReadout
IActor	Interaction	1	PlayerInteractor (และ RecordingActor ในชุดทดสอบ)
IInspectable	Interaction	1	Inspectable
IReadable	Interaction	1	WrittenPage
IPowerStore	Clock	1	Mainspring
ICodeInput	Puzzle	1	CodeWheel
IInHandOnly	Interaction	1	TestRun (marker interface ไม่มีสมาชิก)
ITimedTest	Interaction	1	TestRun
รวม		**29 ความสัมพันธ์**	จาก **22 คลาส**

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อจำกัดที่ต้องระบุตามจริง** — ICodeInput ยังไม่สามารถใช้งานตามเจตนาของมันได้ เพราะฟิลด์ใน CodeLock ที่ควรเป็น ICodeInput[] ถูกประกาศเป็นคลาสรูปธรรม CodeWheel[] (Assets/Scripts/Puzzle/CodeLock.cs:17) สาเหตุคือ Unity ไม่สามารถ serialize interface ได้โดยตรง รายละเอียดและผลกระทบอยู่ในบทที่ 5 หัวข้อ 5.3

> **หลักฐานที่ชัดที่สุดของข้อจำกัดนี้อยู่ในไฟล์เดียวกัน** (ตรวจซ้ำ 8 พฤศจิกายน 2569) บรรทัดที่ 16 ของ CodeLock.cs เป็นคำอธิบายฟิลด์ที่เขียนไว้ว่า "Left to right, as the player reads them. Anything implementing ICodeInput." แต่บรรทัดถัดมาคือ private CodeWheel[] wheels; กล่าวคือ **คำอธิบายฟิลด์สัญญาว่ารับอะไรก็ได้ที่สร้างจาก ICodeInput ขณะที่ชนิดข้อมูลที่ประกาศจริงรับได้เฉพาะ CodeWheel** ช่องว่างระหว่างเจตนาที่เขียนไว้กับชนิดข้อมูลที่ประกาศได้จริงปรากฏอยู่ในสองบรรทัดที่ติดกัน และเป็นตัวอย่างที่ดีของข้อจำกัดเชิงเครื่องมือที่สถาปัตยกรรมต้องยอมรับ ไม่ใช่ความบกพร่องของการออกแบบ

2.2.2 หลักการแยกส่วนต่อประสาน (Interface Segregation Principle)

Interface Segregation Principle (ISP) ระบุว่าไม่ควรบังคับให้ผู้ใช้งานต้องขึ้นต่อเมธอดที่ตนไม่ได้ใช้ interface ที่ใหญ่เกินไปควรถูกแยกออกเป็น interface เล็ก ๆ ตามกลุ่มผู้ใช้งาน [8]

ตำแหน่งที่พบในโครงงาน กรณีที่ชัดเจนที่สุดคือ `IActor` ซึ่งเป็นสัญญาที่กำหนดว่า "ผู้ที่มาโต้ตอบกับวัตถุ" สามารถถูกขอให้ทำอะไรได้บ้าง ทั้ง interface มีเพียงสามเมธอด

```
// Assets/Scripts/Interaction/IActor.cs:15-26
public interface IActor
{
    void Say(string message);
    void OfferBag(IItemReceiver to);
    void Take(ItemData item);
}
```

และ `PlayerInteractor` implement ทั้งสามข้อด้วยโค้ดสามบรรทัด

```
// Assets/Scripts/Player/PlayerInteractor.cs:105-109
public void Say(string message) => hud.ShowMessage(message);
public void OfferBag(IItemReceiver to) => inventoryUI.Open(to);
public void Take(ItemData item) => Inventory.Add(item);
```

ที่สำคัญคือเมธอด `IInteractable.Interact()` รับพารามิเตอร์เป็น `IActor` ไม่ใช่ `PlayerInteractor` (Assets/Scripts/Interaction/Interactions.cs:13) ดังนั้นวัตถุทุกชิ้นในเกม เช่น นาฬิกา ลิ้นชัก วงล้อรหัส จึงมองเห็นเพียงสามเมธอดที่มันต้องใช้ ไม่ได้มองเห็นระยะเอื้อม ความเร็วเดิน หรือระบบการยกของขึ้นดูของผู้เล่น ผลที่วัดได้มีสองอย่าง คือ (1) โมดูล `Clock` และ `Puzzle` สามารถใส่ `.asmdef` ที่ไม่อ้างถึงโมดูล `Player` ได้ และ (2) ทดสอบได้โดยไม่ต้องเปิดฉากและไม่ต้องมีผู้เล่นอยู่จริง ดังหลักฐานในหัวข้อ 2.4.2

2.2.3 หลักการเปิด–ปิด (Open–Closed Principle)

Open–Closed Principle (OCP) ซึ่ง Meyer เสนอไว้ครั้งแรก [10] และ Martin นำมาขยายในบริบทของ SOLID [8] ระบุว่าหน่วยซอฟต์แวร์ควร **เปิดให้ขยายได้ แต่ปิดการแก้ไข** กล่าวคือการเพิ่มความสามารถใหม่ควรทำได้ด้วยการเขียนโค้ดใหม่ ไม่ใช่การแก้โค้ดที่ทำงานอยู่แล้ว เพราะการแก้โค้ดเดิมมีความเสี่ยงที่จะทำให้สิ่งที่เคยทำงานได้เสียหาย

ตำแหน่งที่พบในโครงงาน เมื่อประเมินตามเกณฑ์นี้ โครงงานผ่าน 6 สถานการณ์จาก 9 สถานการณ์ ดังตารางที่ 2.5

ตารางที่ 2.5 ผลการประเมินตามหลักการเปิด–ปิด

สิ่งที่ต้องการเพิ่ม	ต้องเขียนใหม่	ต้องแก้ไฟล์เดิมหรือไม่	ผล
วิธีแสดงเวลาแบบใหม่	1 คลาสที่ implement `IClockDisplay`	ไม่ต้อง	ผ่าน
ส่วนประกอบนาฬิกาแบบใหม่	1 คลาสที่ implement `IClockPart`	ไม่ต้อง	ผ่าน
แหล่งพลังงานแบบใหม่	1 คลาสที่ implement `IPowerStore`	ไม่ต้อง	ผ่าน
กลอนแบบใหม่	1 คลาสที่ implement `ILock`	ไม่ต้อง	ผ่าน
วัตถุที่โต้ตอบได้แบบใหม่	1 คลาสที่ implement `IInteractable`	ไม่ต้อง	ผ่าน
ชนิดนาฬิกาใหม่ทั้งเรือน	ไม่ต้องเขียนโค้ด — สร้างไฟล์ `ClockRecipe.asset`	ไม่ต้อง	ผ่าน
อุปกรณ์ใส่รหัสแบบใหม่	1 คลาสที่ implement `ICodeInput`	**ต้องแก้ `CodeLock.cs`**	ไม่ผ่าน
วิธีมองวัตถุแบบใหม่	—	**ต้องเพิ่มค่าใน enum `InspectStyle` และแก้ `InspectView.cs`**	ไม่ผ่าน
สภาพนาฬิกาแบบที่ 6	—	**ต้องเพิ่มค่าใน enum `ClockCondition` และแก้ `ClockMovement.cs`**	ไม่ผ่าน

สามสถานการณ์ที่ไม่ผ่านเกิดจากการเลือกใช้ `enum` ร่วมกับคำสั่ง `switch` แทนการใช้ polymorphism ซึ่งเป็น **การแลกเปลี่ยนที่ตัดสินใจโดยรู้ตัว** ไม่ใช่ความพลาด เกณฑ์ที่ใช้ตัดสินคือมิติที่คาดว่ามีค่าจำกัดและรู้ครบแล้ว (closed set) ใช้ `enum` ส่วนมิติที่คาดว่าจะเพิ่มขึ้นเรื่อย ๆ (open set) ใช้ interface ซึ่งเป็นเกณฑ์ที่ยอมรับได้ในทางวิศวกรรมซอฟต์แวร์ ยกเว้นกรณี `ICodeInput` ที่เจตนาเดิมต้องการให้เป็น open set แต่ผลลัพธ์ในโค้ดไม่เป็นเช่นนั้น

2.2.4 การแยกความกังวล (Separation of Concerns)

Separation of Concerns เป็นหลักที่ Dijkstra อธิบายไว้ว่าเป็นการพิจารณาเรื่องหนึ่ง ๆ แยกจากเรื่องอื่นในแต่ละครั้ง
เพื่อให้สามารถใช้ความคิดกับเรื่องนั้นได้อย่างลึกซึ้งโดยไม่ถูกรบกวนจากเรื่องที่ไม่เกี่ยวข้อง [11]
ในทางปฏิบัติของการออกแบบซอฟต์แวร์
หลักนี้แปลเป็นการแยกส่วนที่รับผิดชอบเรื่องต่างกันออกเป็นหน่วยต่างกัน

ตำแหน่งที่พบในโครงงาน โครงงานนี้ใช้หลักนี้อย่างสม่ำเสมอในสามคู่ที่สำคัญ

1. **แยกสถานะออกจากการเคลื่อนไหว** — Openable ถือสถานะเปิดหรือปิด ตรวจสอบกลอน และส่งสัญญาณเหตุการณ์ โดย **ไม่เขียนค่า transform เลยแม้แต่บรรทัดเดียว** (Openable.cs:1-78) ส่วน SwingMotion และ SlideMotion อ่านค่า IsOpen แล้วขยับวัตถุ โดยไม่รู้ว่าเหตุใดจึงเปิด (SwingMotion.cs:25-31, SlideMotion.cs:25-30)
2. **แยกเวลาออกจากการแสดงเวลา** — ClockMovement เก็บจำนวนนาทีและส่งค่าให้ผู้แสดงผล (ClockMovement.cs:155-159) ส่วน ClockDial แปลงนาทีเป็นมุมของเข็ม (ClockDial.cs:22-37) และ DigitalReadout แปลงนาทีเดียวกันเป็นตัวเลขสี่หลัก (DigitalReadout.cs:44-57) โดยสองคลาสหลังนี้ **ไม่ใช้โค้ดร่วมกันเลยแม้แต่บรรทัดเดียว**
3. **แยกตรรกะออกจากเสียง** — TestRun ควบคุมช่วงเวลาของการทดสอบนาฬิกา ส่วน RunTicking เล่นเสียงติ๊กโดยอ่านค่า _run.Allows เท่านั้นและ **ไม่มีตัวจับเวลาของตัวเอง** (RunTicking.cs:39) ทำให้เสียงไม่สามารถคลาดจากการทดสอบได้ไม่ว่าจะตั้งความยาวของการทดสอบไว้เท่าใด

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อจำกัดที่ต้องระบุตามจริง** —
> การแยกนี้ทำได้สม่ำเสมอในโมดูล Clock และ Interaction แต่ **ไม่ได้ทำในสองจุด** คือ Horologium.Update() ขยับ transform ของจุดแสงด้วยตัวเอง (Horologium.cs:94-106) และ InspectView เรียกใช้ส่วนแสดงผลโดยตรง (InspectView.cs:166,197,364-365) รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3

2.2.5 รูปแบบผู้สังเกตการณ์ (Observer Pattern)

Observer Pattern เป็นแบบแผนเชิงพฤติกรรมที่ Gamma และคณะนิยามไว้ว่าเป็นการกำหนดความสัมพันธ์แบบหนึ่งต่อหลายระหว่างอ็อบเจกต์

เพื่อให้เมื่ออ็อบเจกต์หนึ่งเปลี่ยนสถานะ
อ็อบเจกต์ที่ขึ้นต่อทั้งหมดได้รับการแจ้งและปรับตัวเองอัตโนมัติ
โดยที่ผู้แจ้งไม่จำเป็นต้องรู้ว่าใครเป็นผู้รับฟัง [7] ในภาษา C# แบบแผนนี้มีกลไกรองรับอยู่ในภาษาโดยตรงด้วยคำสั่ง event [20]

ตำแหน่งที่พบในโครงงาน โครงงานใช้ event Action ของ C# จำนวน 3 คู่ ดังตารางที่ 2.6 และ **ไม่ใช้ UnityEvent เลย**

ตารางที่ 2.6 การใช้ Observer Pattern ในโครงงาน

ผู้ประกาศเหตุการณ์	ผู้รับฟัง	ความหมายในเกม	หลักฐาน
Openable.Opened	GameManager.HandleEscaped	ประตูทางออกเปิด จึงจบเกม	Openable.cs:37,76 · GameManager.cs:25-26,42-47
IPowerStore.Filled	ClockStriker.Strike	มีผู้เติมพลังงานให้นาฬิกา นกจึงออกมาขัน	IPowerStore.cs:19 · Mainspring.cs:25,46 · ClockStriker.cs:76,106,110
ICodeInput.Changed	CodeLock.CheckWheels	มีการหมุนวงล้อ จึงตรวจรหัสทันที	CodeWheel.cs:30,63 · CodeLock.cs:43,49,65-76

ตัวอย่างที่แสดงประโยชน์เชิงสถาปัตยกรรมของแบบแผนนี้ได้ชัดที่สุดคือคู่ที่สอง ClockStriker รับฟังเหตุการณ์ว่า "ถูกเติมพลังงาน" ไม่ได้รับฟังว่า "มีคนหมุนก้านไขลาน" ดังนั้นเมื่อนาฬิกาคุกคูถูกเปลี่ยนจากการไขลานด้วย ClockWinder ไปเป็นการดึงโซ่ตุ้มถ่วงด้วย WeightChain คลาส ClockStriker **ไม่ต้องถูกแก้เลยแม้แต่บรรทัดเดียว** เพราะทั้งสองคลาสเรียกเมธอด IPowerStore.Fill() เหมือนกัน (ClockWinder.cs:100, WeightChain.cs:55)

ข้อที่ควรบันทึกไว้ในแง่ความถูกต้องของการใช้งาน คือเหตุการณ์ทั้งสามคู่มีการยกเลิกการรับฟังใน OnDisable ครบทุกคู่ (GameManager.cs:26, ClockStriker.cs:106, CodeLock.cs:46-50) ซึ่งเป็นสิ่งจำเป็นเพื่อป้องกันการรั่วของหน่วยความจำและการเรียกเมธอดบนอ็อบเจกต์ที่ถูกทำลายไปแล้ว

```
// Assets/Scripts/Game/GameManager.cs:25-26 — subscribe และ
unsubscribe เป็นคู่
```

```
private void OnEnable() => wayOut.Opened += HandleEscaped;
private void OnDisable() => wayOut.Opened -= HandleEscaped;
```

2.2.6 การออกแบบที่ขับเคลื่อนด้วยข้อมูล (Data-Driven Design)

Data-Driven Design คือการย้ายสิ่งที่เคยเป็นค่าคงที่หรือตรรกะในซอร์สโค้ดออกมาเป็นข้อมูลภายนอก เพื่อให้เปลี่ยนแปลงได้โดยไม่ต้องคอมไพล์ใหม่ และเพื่อให้ผู้ที่ไม่ได้เขียนโปรแกรมสามารถปรับแต่งเนื้อหาได้ Gregory ระบุว่าแนวทางนี้เป็นสิ่งที่เอนจินเกมเชิงพาณิชย์ต้องมี เพราะงานออกแบบเกมมีการปรับค่าซ้ำจำนวนมากและรอบการคอมไพล์เป็นคอขวด [6]

ตำแหน่งที่พบในโครงงาน โครงงานใช้ `ScriptableObject` ของ Unity (ดูหัวข้อ 2.5.6) เป็นรูปแบบของข้อมูลภายนอก รวม 3 คลาส 12 ไฟล์แอสเซ็ต ได้แก่

- `ItemData` — นิยามว่าของชิ้นหนึ่งคืออะไร (10 ไฟล์ใน `Assets/Scenes/Items/`)
- `HousePlan` — ผังบ้านทั้งหลังในรูปข้อมูล คือสี่เหลี่ยมห้าห้องและรายการแนวผนังพร้อมจุดกลางของช่องเปิด (`Assets/Recipes/HousePlan.asset`)
- `ClockRecipe` — สูตรประกอบนาฬิกาหนึ่งชนิด (`Assets/Recipes/PlankClock.asset`)

กรณี `ClockRecipe` เป็นกรณีที่น่าสนใจที่สุด เพราะเป็นจุดที่ Data-Driven Design กับ Component-Based Architecture มาบรรจบกัน

```
// Assets/Scripts/Clock/ClockRecipe.cs:19-37 — มีรายการเดียว
ไม่แยกว่าอะไรคือจอแสดงผล อะไรคือส่วนประกอบ
[System.Serializable]
public struct Fitting { public GameObject Prefab; public Vector3
At; public Vector3 Turn; [Min(0.01f)] public float Scale; }


[SerializeField] private GameObject caseModel;    // ตัวเรือนเปล่า
เป็นรูปทรงเท่านั้น
[SerializeField] private Fitting[] fittings;      // ทุกอย่างที่แปะลงบนเรือน
เรียงลำดับใดก็ได้
```

สิ่งสำคัญของโครงสร้างนี้อยู่ที่สิ่งที่มัน **ไม่มี** คือไม่มีรายการแยกสำหรับจอแสดงผลและสำหรับส่วนประกอบ ทั้งสองเป็นเพียง "สิ่งที่นำมาแปะ" เหมือนกัน และ `ClockMovement` เป็นผู้แยกเองตอน `Awake` ด้วยการถามว่าสิ่งนั้น implement

interface ใด ผลคือ **ชนิดของนาฬิกากลายเป็นไฟล์ข้อมูล ไม่ใช่เมธอดในโค้ด** ซึ่งพิสูจน์แล้วด้วยชุดทดสอบ RecipeSeamTests.cs (ดูหัวข้อ 2.4.2)

2.3 ทฤษฎีความถูกต้องของการนับเวลาในเกม

> หัวข้อนี้เป็นทฤษฎีที่รองรับปัญหาหลักที่พบระหว่างพัฒนา ซึ่งจะกล่าวถึงในบทที่ 5 หัวข้อ 5.2

2.3.1 วงรอบของเกมและความเป็นอิสระจากอัตราเฟรม

เกมแบบเรียลไทม์ทำงานด้วยโครงสร้างที่เรียกว่า **วงรอบของเกม (Game Loop)** คือวงวนที่ทำงานซ้ำไปเรื่อย ๆ โดยในแต่ละรอบจะรับข้อมูลนำเข้า ปรับสถานะของโลกในเกมให้ก้าวหน้าไปหนึ่งขั้น แล้ววาดภาพออกจอหนึ่งเฟรม Nystrom ชี้ว่าประเด็นสำคัญของวงรอบนี้คือวงวนทำงานด้วยความเร็วที่ไม่แน่นอน ขึ้นกับความสามารถของเครื่องและปริมาณงานในเฟรมนั้น ดังนั้นการปรับสถานะของโลกจึง **ต้องอ้างตามเวลาจริงที่ผ่านไป ไม่ใช่ตามจำนวนรอบของวงวน** [5]

ถ้าโค้ดปรับสถานะด้วยค่าคงที่ต่อหนึ่งเฟรม เช่น เพิ่มตำแหน่งขึ้นทีละ 0.1 หน่วยทุกเฟรม ผลลัพธ์จะผูกกับอัตราเฟรมของเครื่องที่รัน เครื่องที่วาดได้ 120 เฟรมต่อวินาทีจะเคลื่อนที่เร็วเป็นสองเท่าของเครื่องที่วาดได้ 60 เฟรมต่อวินาที อาการนี้เรียกว่าโค้ดที่ **ขึ้นกับอัตราเฟรม** (frame-rate dependent) และเป็นข้อผิดพลาดที่พบบ่อยที่สุดอย่างหนึ่งในการพัฒนาเกม วิธีแก้มาตรฐานคือคูณอัตราต่อวินาทีด้วยช่วงเวลาจริงของเฟรมนั้น ซึ่งใน Unity คือค่า Time.deltaTime ที่นิยามไว้ว่า *"ช่วงเวลาในหน่วยวินาทีจากเฟรมก่อนหน้ามาถึงเฟรมปัจจุบัน"* [13]

ตำแหน่งที่พบในโครงงาน การนับเวลาทุกจุดในโครงงานใช้ Time.deltaTime เป็นฐาน ตัวอย่างเช่น

```
// Assets/Scripts/Clock/ClockMovement.cs:136
double advance = (double)minutesPerSecond * Rate *
Time.deltaTime;
```

```
// Assets/Scripts/Clock/Sandglass.cs:107 — นาฬิกาทรายนับเวลาที่ผ่านไป
_run = Mathf.Min(_run + Time.deltaTime, seconds);
```

```
// Assets/Scripts/Clock/TestRun.cs:77
_ran += Time.deltaTime;
```

หลักฐานยืนยันเชิงลบที่มีน้ำหนัก คือการค้นหาคำว่า `Time.frameCount` ทั้งโครงงาน **ไม่พบการใช้งานเลยแม้แต่ที่เดียว** กล่าวคือไม่มีจุดใดในโครงงานที่นับเวลาด้วยจำนวนเฟรม

2.3.2 ความแม่นยำของเลขทศนิยมและความคลาดเคลื่อนสะสม

เลขทศนิยมแบบจุดลอยตัว (floating-point) ตามมาตรฐาน IEEE 754 เก็บค่าในรูปของแมนทิสซาคูณด้วยสองยกกำลังเลขชี้กำลัง ผลคือ **ระยะห่างระหว่างเลขที่แทนได้สองค่าที่อยู่ติดกัน ไม่คงที่ แต่ใหญ่ขึ้นตามขนาดของค่า** ระยะห่างนี้เรียกว่า ULP (Unit in the Last Place) [16] Goldberg อธิบายไว้ว่าผลสำคัญของคุณสมบัตินี้คือ เมื่อนำเลขขนาดใหญ่มากบวกกับเลขขนาดเล็กมาก ส่วนของเลขเล็กที่ตกอยู่ใต้ ULP ของผลลัพธ์จะถูกปัดทิ้งไปทั้งหมด ปรากฏการณ์นี้เรียกว่าการสูญเสียนัยสำคัญ (loss of significance) และจะสะสมตัวขึ้นเมื่อเกิดซ้ำหลายพันครั้ง [15]

ชนิดข้อมูล `float` ตามมาตรฐาน IEEE 754 binary32 มีแมนทิสซา 24 บิต ส่วน `double` (binary64) มี 53 บิต [16] เมื่อนำมาคำนวณกับค่าที่เกิดขึ้นจริงในโครงงานนี้ จะได้ผลตามตารางที่ 2.7

ตารางที่ 2.7 ระยะห่างระหว่างเลขที่แทนได้ ณ ค่าสะสม 280 นาที (เวลา 4:40)

ชนิดข้อมูล	แมนทิสซา	ULP ณ ค่า 280	การคำนวณ
`float` (binary32)	24 บิต	$\approx 3.05 \times 10^{-5}$ นาที	$2^{8} \times 2^{-23} = 2^{-15}$
`double` (binary64)	53 บิต	$\approx 5.68 \times 10^{-14}$ นาที	$2^{8} \times 2^{-52} = 2^{-44}$

นาฬิกาในเกมนี้ทำงานโดยนำจำนวนนาทีที่เพิ่มขึ้นในหนึ่งเฟรมไปบวกกับจำนวนนาทีสะสมที่มีอยู่แล้ว ซึ่งเป็นรูปแบบของการอินทิเกรตค่าที่ละนิดเข้าสู่ผลรวมที่ใหญ่ขึ้นเรื่อย ๆ นับพันครั้งต่อการเล่นหนึ่งครั้ง จากตารางที่ 2.7 ULP ของ `float` ณ ค่า 280 อยู่ที่ประมาณ 3×10^{-5} นาที ขณะที่นาฬิกาในห้องโถงถูกตั้งค่า `minutesPerSecond` ไว้ที่ 1/60 ซึ่งที่อัตรา 60 เฟรมต่อวินาทีหมายถึงค่าที่เพิ่มขึ้นต่อเฟรมประมาณ 2.8×10^{-4} นาที กล่าวคือค่าที่นำมาบวกอยู่ห่างจาก ULP ของผลรวมเพียงประมาณหนึ่งลำดับความสำคัญเท่านั้น ซึ่งเป็นช่วงที่การปัดทิ้งเริ่มให้ผลที่วัดได้ และเลวร้ายลงอีกเมื่อนาฬิกาเดินช้าเพราะค่าที่เพิ่มต่อเฟรมยิ่งเล็กลง

หลักทั่วไปที่สรุปได้จากเรื่องนี้ และเป็นหลักที่ใช้ได้กับทุกระบบที่สะสมค่า คือ **ผลรวมที่กำลังสะสมและค่าที่นำมาบวกเข้าไป**

ต้องอยู่ภายในช่วงความแม่นยำของชนิดข้อมูลนั้นเมื่อเทียบกันเอง
ระบบใดที่อินทิเกรตค่าเล็กเข้าสู่ผลรวมใหญ่ต่อเนื่องหลายพันเฟรม ย่อมเผชิญปัญหาเดียวกัน

ตำแหน่งที่พบในโครงงาน ClockMovement เก็บจำนวนนาทีสะสมไว้ใน double ไม่ใช่ float

```
// Assets/Scripts/Clock/ClockMovement.cs:50
private double _minutes;
```

โดยส่วนต่อประสานที่เปิดให้ภายนอกยังคงเป็น float (เมธอด Minutes, Advance, Show) เพราะการแปลงชนิดหนึ่งครั้งที่ปลายทางมีผลต่อความคลาดเคลื่อนเพียงประมาณ 0.01 องศาและเกิดครั้งเดียว ส่วนที่ทำใน float ไม่ได้คือ **การสะสม** ดังนั้นทุกอย่างที่บวกเข้ากับ _minutes จึงต้องอยู่ในรูป double ตลอดทาง รวมถึงการวนรอบ 12 ชั่วโมงด้วย เพราะเมธอด Mathf.Repeat ของ Unity รับเฉพาะ float จึงต้องทำการวนรอบในเมธอด OnDial แทน (ClockMovement.cs:163-166)

2.3.3 ลำดับการทำงานของฟังก์ชันเหตุการณ์

ใน Unity สคริปต์ที่สืบทอดจาก MonoBehaviour จะถูกเอนจินเรียกใช้ผ่านฟังก์ชันเหตุการณ์ (event function) ตามลำดับที่กำหนดไว้ในวงรอบของเฟรม โดยฟังก์ชันที่เกี่ยวข้องกับโครงงานนี้คือ Awake (เรียกครั้งเดียวเมื่อสร้างอ็อบเจกต์), Update (เรียกทุกเฟรม) และ LateUpdate (เรียกทุกเฟรม **หลังจาก** Update ของทุกสคริปต์ทำงานครบแล้ว) [14]

ประเด็นที่สำคัญต่อความถูกต้องของโปรแกรม และเป็นสิ่งที่เอกสารของ Unity ระบุไว้เป็นข้อจำกัดอย่างชัดเจน คือ **ลำดับการเรียกฟังก์ชันเหตุการณ์เดียวกันระหว่างอ็อบเจกต์ต่างกันนั้นไม่รับประกัน** เอกสารใช้ถ้อยคำว่า *"โดยทั่วไป ไม่สามารถเชื่อถือลำดับที่ฟังก์ชันเหตุการณ์เดียวกันถูกเรียกใช้สำหรับ GameObject ต่างกันได้ ยกเว้นในกรณีที่ลำดับนั้นถูกระบุไว้ในเอกสารหรือสามารถตั้งค่าได้"* และระบุต่อว่าลำดับระหว่างสคริปต์ต่างชนิดกันสามารถตั้งค่าได้ด้วย Script Execution Order แต่ลำดับระหว่างอินสแตนซ์ของสคริปต์เดียวกันนั้นตั้งไม่ได้เลย [14]

ผลที่ตามมาคือ หากสองส่วนประกอบที่ต้องทำงานเรียงลำดับกันเขียนโค้ดไว้ใน Update ทั้งคู่ โดยไม่ได้ตั้งค่า Script Execution Order ผลลัพธ์ของโปรแกรมจะไม่แน่นอน และอาจต่างกันในแต่ละเครื่องหรือแต่ละครั้งที่รัน วิธีแก้ที่ไม่ต้องพึ่งการตั้งค่าในโปรเจกต์ คือย้ายส่วนที่ต้องทำงานทีหลังไปอยู่ใน LateUpdate เพราะเอนจินรับประกันว่า LateUpdate ทำงานหลัง Update ของทุกสคริปต์

ตำแหน่งที่พบในโครงงาน คลาส TestRun ซึ่งควบคุมช่วงเวลาของการทดสอบนาฬิกา นับเวลาใน LateUpdate ขณะที่ ClockMovement เดินเข็มใน Update

```
// Assets/Scripts/Clock/TestRun.cs:72-81
private void LateUpdate()
{
    if (!_running) return;
    _ran += Time.deltaTime;
    if (_ran >= Length) _running = false;
}
```

นอกจากนี้ยังมีกลไกที่เกี่ยวข้องอีกสองอย่างในโครงงาน คือ

1. **การคิดเฟรมสุดท้ายเป็นเศษส่วน** — TestRun คืนค่าตัวคูณอัตราเป็นสัดส่วนของเวลาที่เหลือต่อความยาวของเฟรมที่กำลังจะใช้ไป ทำให้เฟรมที่คร่อมเส้นสิ้นสุดถูกนับเพียงเท่าที่เหลือจริง เข็มจึงหยุดตรงขีดเดิมเสมอไม่ว่าเฟรมจะยาวเท่าใด

```
// Assets/Scripts/Clock/TestRun.cs:52
public float RateScale => _running ? Mathf.Clamp01((Length - _ran) / Time.deltaTime) : 1f;
```

2. **การตั้งลำดับการทำงานของ InputReader** — คลาสนี้ใช้แอตทริบิวต์ [DefaultExecutionOrder(-100)] (InputReader.cs:8) เพื่อให้แน่ใจว่าอ็อบเจกต์ PlayerControls ถูกสร้างขึ้นก่อนที่คลาสอื่นจะเข้ามาลงทะเบียนรับฟังเหตุการณ์ใน OnEnable ของตนเอง ซึ่งเป็นการใช้กลไกตั้งลำดับอย่างตรงจุดในที่ที่จำเป็นจริง

2.4 ทฤษฎีการทดสอบซอฟต์แวร์

2.4.1 ระดับและชนิดของการทดสอบ

การทดสอบซอฟต์แวร์แบ่งตามขอบเขตของสิ่งที่ถูกทดสอบได้เป็นหลายระดับ โดยสองระดับที่เกี่ยวข้องกับโครงงานนี้คือ

- **Unit test** — ทดสอบหน่วยย่อยที่สุดของโค้ด เช่น เมธอดหรือคลาสเดียว โดยแยกออกจากสิ่งแวดล้อมภายนอกทั้งหมด Meszaros ระบุคุณสมบัติที่ unit test ที่ดีควรมีไว้ว่าต้องทำงานเร็ว ทำงานได้โดยไม่ต้องพึ่งระบบภายนอก และให้ผลเหมือนกันทุกครั้งที่รัน [17]
- **Integration test** — ทดสอบว่าหลายหน่วยทำงานร่วมกันได้ถูกต้อง ซึ่งจำเป็นต้องมีสิ่งแวดล้อมบางส่วนอยู่จริง

ใน Unity ความแตกต่างนี้สะท้อนออกมาเป็นสองโหมดของ Unity Test Framework คือ **EditMode test** ซึ่งรันในกระบวนการของตัวแก้ไข (Editor) โดยไม่ต้องเริ่มเกม และ **PlayMode test** ซึ่งรันในสภาพแวดล้อมเวลาทำงานจริงของเอนจิน จึงสามารถใช้ฟังก์ชันเหตุการณ์และให้เฟรมเดินไปข้างหน้าได้ [25]

ความแตกต่างนี้สำคัญต่อการเขียนรายงานให้ถูกต้อง เพราะการทดสอบที่ต้องอาศัยการเดินของเฟรมนั้นไม่ใช่ unit test ตามความหมายที่เคร่งครัด

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อควรระวังสำหรับผู้เขียน** — โครงงานนี้มี **PlayMode test เท่านั้น** ไม่มี EditMode test และไม่มี unit test ที่รันนอกเอนจิน ในรายงานจึงต้องเรียกการทดสอบชุดนี้ว่า PlayMode test ทุกครั้ง **ห้ามเขียนว่าโครงงานมี unit test**

2.4.2 การทดสอบความเหมาะสมของสถาปัตยกรรม (Architecture Fitness Function)

Ford, Parsons และ Kua เสนอแนวคิด **Architecture Fitness Function** ว่าเป็นการทดสอบอัตโนมัติชนิดหนึ่งที่ไม่ได้ตรวจว่าฟังก์ชันคืนค่าถูกต้องหรือไม่ แต่ตรวจว่า **คุณลักษณะเชิงสถาปัตยกรรมที่ออกแบบไว้ยังเป็นจริงอยู่หรือไม่** ตัวอย่างเช่นการตรวจว่าโมดูลหนึ่งไม่อ้างถึงอีกโมดูลหนึ่ง หรือการตรวจว่าจุดต่อที่ออกแบบให้ขยายได้นั้นขยายได้จริง ประโยชน์ของการทดสอบชนิดนี้คือทำให้ข้ออ้างทางสถาปัตยกรรมกลายเป็นสิ่งที่ตรวจสอบได้โดยอัตโนมัติ ไม่ใช่เพียงข้อตกลงที่เขียนไว้ในเอกสารแล้วค่อย ๆ ถูกละเมิดไปตามเวลา [18]

ตำแหน่งที่พบในโครงงาน โครงงานมี PlayMode test รวม **15 กรณีใน 3 ไฟล์** และลักษณะของการทดสอบเหล่านี้ตรงกับนิยามของ Architecture Fitness Function มากกว่านิยามของ unit test ทั่วไป ดังตารางที่ 2.8

ตารางที่ 2.8 ไฟล์ทดสอบและคุณลักษณะเชิงสถาปัตยกรรมที่แต่ละไฟล์พิสูจน์

ไฟล์	บรรทัด	กรณี	พิสูจน์ว่า
`ActorSeamTests.cs`	198	6	`IActor` ตัดการพึ่งพาคลาส `PlayerInteractor` ได้จริง
`ClockSeamTests.cs`	124	6	`IClockDisplay` และ `IClockPart` เป็นจุดต่อที่ขยายได้จริง
`RecipeSeamTests.cs`	71	3	นาฬิกาที่ประกอบจากไฟล์ข้อมูลทำงานเหมือนที่เขียนด้วยโค้ด

กรณีทดสอบที่แสดงลักษณะนี้ได้ชัดที่สุดสองกรณี คือ

กรณีที่หนึ่ง คลาสจอแสดงผลถูกประกาศขึ้นใหม่ **ภายในไฟล์ทดสอบ** ซึ่งหมายความว่าคลาสนี้ยังไม่มีอยู่ในขณะที่ ClockMovement ถูกคอมไพล์ หากการทดสอบผ่านย่อมพิสูจน์ได้ว่า ClockMovement ไม่ได้ผูกกับคลาสจอแสดงผลใดเป็นการเฉพาะ ซึ่งคือข้ออ้างของ DIP ในหัวข้อ 2.2.1 นั่นเอง

```
// Assets/Tests/PlayMode/ClockSeamTests.cs:41-67
private class CountingDisplay : MonoBehaviour, IClockDisplay
{
    public int Shown; public float Last;
    public void Show(float minutes) { Shown++; Last = minutes; }
}


[UnityTest]
public IEnumerator
A_display_written_after_the_movement_still_gets_the_time()
{
    GameObject go = Clock(out ClockMovement movement);
    var display = go.AddComponent<CountingDisplay>();
    Set(movement, "startHour", 4); Set(movement, "startMinute",
40);
    go.SetActive(true);
    yield return null;
    Assert.Greater(display.Shown, 0, "the movement never called
Show");
    Assert.AreEqual(4 * 60 + 40, display.Last, 1f, "it was handed
the wrong time");
}
```

กรณีที่สอง สัญญา IActor ทั้งสัญญาถูก implement ด้วยคลาส C# ธรรมดาความยาวประมาณสิบบรรทัด ที่ไม่ใช่ MonoBehaviour และไม่ต้องอยู่ในฉากใดเลย แล้วนำคลาสนั้นไปเปิดตู้ ไขกุญแจ และประกอบชิ้นส่วนนาฬิกาได้สำเร็จ ซึ่งพิสูจน์ข้ออ้างของ ISP ในหัวข้อ 2.2.2

```
// Assets/Tests/PlayMode/ActorSeamTests.cs:18-27
private class RecordingActor : IActor
{
    public readonly List<string> Said = new List<string>();
    public readonly List<IItemReceiver> BagOfferedTo = new
List<IItemReceiver>();
    public readonly List<ItemData> Taken = new List<ItemData>();
    public void Say(string message) => Said.Add(message);
    public void OfferBag(IItemReceiver to) =>
BagOfferedTo.Add(to);
    public void Take(ItemData item) => Taken.Add(item);
}
```

> [หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ข้อจำกัดที่ต้องระบุตามจริง** — ชุดทดสอบนี้ครอบคลุมเฉพาะโมดูล Items, Interaction และ Clock **ไม่มีกรณีทดสอบใดครอบคลุมโมดูล Puzzle, World, Player, UI หรือ Game เลย** และในกรณีของสามโมดูลหลัง เหตุผลเป็นข้อจำกัดทางเทคนิคของ Unity ที่ชุดทดสอบซึ่งมี .asmdef อ้างถึงโมดูลใน predefined assembly ไม่ได้ (ดูหัวข้อ 2.5.7) รายละเอียดอยู่ในบทที่ 4 และข้อเสนอแนะอยู่ในบทที่ 5 หัวข้อ 5.3

2.5 เทคโนโลยีที่ใช้ในการปฏิบัติงาน

2.5.1 Unity Engine

Unity เป็นเอนจินเกมสำหรับสร้างเนื้อหาสองมิติและสามมิติที่รองรับการส่งออกไปหลายแพลตฟอร์ม โครงสร้างพื้นฐานของเอนจินใช้โมเดล GameObject–Component ตามที่อธิบายไว้ในหัวข้อ 2.1.3 โดยหน่วยที่ใช้จัดองค์ประกอบของโลกในเกมเรียกว่า Scene และหน่วยที่ใช้เก็บแม่แบบของอ็อบเจกต์ที่นำกลับมาใช้ซ้ำเรียกว่า Prefab

โครงงานนี้ใช้ **Unity เวอร์ชัน 6000.3.5f2** โดยทั้งเกมอยู่ใน Scene เพียงหนึ่งฉาก คือ Assets/Scenes/Main.unity ขนาดประมาณ 2.35 เมกะไบต์ และเป็นฉากเดียวที่อยู่ในรายการ Build Settings ยืนยันจากไฟล์ EditorBuildSettings.asset ซึ่งมีรายการ m_Scenes เพียงรายการเดียว และจากเมธอด GameManager.Restart() (Assets/Scripts/Game/GameManager.cs:30) ที่เรียก SceneManager.LoadScene ด้วยหมายเลขของฉากที่กำลังทำงานอยู่ กล่าวคือโหลดฉากเดิมซ้ำ เพราะไม่มีฉากอื่นให้ไป

ผลของการใช้ฉากเดียวคือไม่ต้องมีระบบคงสถานะข้ามฉาก (ยืนยันจากการค้นหา DontDestroyOnLoad ได้ผลลัพธ์ศูนย์รายการ) แต่ก็หมายความว่าไม่มีระบบบันทึกและโหลดเกม และเมนูหลักเป็นแผงส่วนติดต่อผู้ใช้ภายในฉากเดียวกัน ไม่ใช่ฉากแยก

> [หมายเหตุของร่าง — ลบก่อนส่ง] 📷 **คำแนะนำการแคปภาพ — ลบออกก่อนส่ง รูปที่ 2.2** หน้าต่าง Unity Editor ของโครงงาน — ให้เห็นหน้าต่าง Hierarchy, Scene View, Inspector และ Project พร้อมกันในภาพเดียว โดยเลือกอ็อบเจกต์ที่มีหลาย Component แปะอยู่ เช่น นาฬิกาเรือนหนึ่ง เพื่อให้ภาพนี้สื่อถึงโมเดล GameObject–Component ไปพร้อมกัน คำบรรยาย: รูปที่ 2.2 ส่วนติดต่อผู้ใช้ของ Unity Editor ในโครงงาน



รูปที่ 2.2 ส่วนติดต่อผู้ใช้ของ Unity Editor ในโครงงาน

2.5.2 Universal Render Pipeline (URP)

Universal Render Pipeline เป็นไปป์ไลน์การเรนเดอร์สำเร็จรูปที่สร้างขึ้นบน Scriptable Render Pipeline ของ Unity เอกสารของ Unity อธิบายไว้ว่า URP *"ให้ลำดับงานที่เป็นมิตรต่อศิลปิน*
ซึ่งช่วยให้สร้างกราฟิกที่ถูกปรับแต่งประสิทธิภาพแล้วได้อย่างรวดเร็วและง่ายดาย
ครอบคลุมแพลตฟอร์มตั้งแต่อุปกรณ์พาหะไปจนถึงเครื่องเล่นเกมและเครื่องคอมพิวเตอร์ระดับสูง" [19] ความแตกต่างที่สำคัญในทางปฏิบัติคือ URP ใช้เชดเดอร์และคุณสมบัติของวัสดุชุดของตนเอง ซึ่งไม่เข้ากันกับเชดเดอร์มาตรฐานของไปป์ไลน์แบบเดิม

โครงงานนี้ใช้ **URP เวอร์ชัน 17.3.0** ยืนยันจากไฟล์ `Packages/manifest.json` และจากไฟล์ตั้งค่า `Assets/Settings/UniversalRenderPipelineGlobalSettings.asset` และยืนยันในระดับการใช้งานจริงจากสองจุด คือวัสดุในโครงงานใช้คุณสมบัติชื่อ `_BaseMap` และ `_EmissionMap` ซึ่งเป็นชื่อคุณสมบัติของ URP และ `Assets/Scripts/Game/RoomLighting.cs:62` เรียกใช้ `RenderSettings.reflectionIntensity`

2.5.3 ภาษา C#

C# เป็นภาษาโปรแกรมเชิงวัตถุที่มีการตรวจชนิดข้อมูลตอนคอมไพล์ (statically typed) และทำงานบนสภาพแวดล้อมที่มีการจัดการหน่วยความจำอัตโนมัติ ข้อกำหนดของภาษาถูกรับรองเป็นมาตรฐาน ECMA-334 [20] Unity ใช้ C# เป็นภาษาเดียวสำหรับการเขียนสคริปต์

คุณสมบัติของภาษาที่โครงงานนี้ใช้อย่างมีนัยสำคัญต่อสถาปัตยกรรมมีสามข้อ

1. **`interface`** — ใช้เป็นสัญญากลางที่ทำให้เกิด DIP และ ISP ตามหัวข้อ 2.2.1 และ 2.2.2 รวม 12 interface

2. **event** — ใช้เป็นกลไกของ Observer Pattern ตามหัวข้อ 2.2.5 โครงงานใช้ event Action ของเนมสเปซ System และ **ไม่ประกาศ delegate ของตัวเองเลย** (ยืนยันจากการค้นหาคำว่า delegate ได้ผลลัพธ์ศูนย์รายการ)

3. **double กับ float** — ใช้แยกบทบาทกันตามหัวข้อ 2.3.2 คือใช้ double สำหรับการสะสมค่า และ float สำหรับส่วนต่อประสานที่เชื่อมกับเอนจิน

2.5.4 MonoBehaviour และวงจรชีวิตของส่วนประกอบ

MonoBehaviour เป็นคลาสฐานที่สคริปต์ของ Unity จำนวนมากสืบทอดมา เอกสารอ้างอิงของ Unity ระบุว่าคลาสนี้ *"มีฟังก์ชันวงจรชีวิตที่ทำให้การพัฒนาด้วย Unity ง่ายขึ้น"* และอ็อบเจกต์ของคลาสนี้ดำรงอยู่ในฐานะ Component ของ GameObject เสมอ [21] ฟังก์ชันวงจรชีวิตที่สำคัญและลำดับการเรียกใช้อธิบายไว้ในหัวข้อ 2.3.3

โครงงานนี้มีคลาสที่สืบทอดจาก MonoBehaviour ทั้งหมด **51 คลาสใน 51 ไฟล์** โดยทุกคลาสสืบทอดโดยตรงในระดับเดียวกัน ไม่มีคลาสใดสืบทอดจากคลาสอื่นในโครงงาน และลำดับของสมาชิกในแต่ละคลาสถูกจัดเรียงตามแบบแผนเดียวกันทุกไฟล์ คือค่าคงที่ ตามด้วยฟิลด์ที่ serialize ได้ ตามด้วยฟิลด์สถานะภายใน ตามด้วยเหตุการณ์และคุณสมบัติสาธารณะ ตามด้วยฟังก์ชันวงจรชีวิตเรียงตามลำดับการเรียกใช้ ตามด้วยเมธอดสาธารณะและเมธอดภายใน

2.5.5 Prefab

Prefab คือระบบของ Unity สำหรับเก็บ GameObject พร้อมส่วนประกอบ ค่าคุณสมบัติ และอ็อบเจกต์ลูกทั้งหมด ไว้ในรูปของแอสเซ็ตที่นำกลับมาใช้ซ้ำได้ เอกสารของ Unity อธิบายว่าแอสเซ็ต Prefab ทำหน้าที่เป็นแม่แบบที่ใช้สร้างอินสแตนซ์ของ Prefab ขึ้นในฉาก [22] และการแก้ไขที่แม่แบบจะส่งผลไปยังทุกอินสแตนซ์ที่ยังเชื่อมโยงอยู่

โครงงานนี้ใช้ Prefab เป็นหน่วยของการนำกลับมาใช้ซ้ำอย่างกว้างขวาง ตัวอย่างที่ชัดเจนที่สุดคือ Assets/Prefabs/Clocks/ClockDial.prefab ซึ่งเป็นหน้าปัดนาฬิกาที่ใช้ร่วมกันโดยนาฬิกาทุกเรือน พบอินสแตนซ์ของคลาส ClockDial ในฉากรวม **46 อินสแตนซ์** และ Assets/Prefabs/Clocks/Parts/Mainspring.prefab พบ 45 อินสแตนซ์

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อควรระวังในการอ้างตัวเลข** — เอกสารภายในโครงงาน (CLAUDE.md) ระบุว่ามีนาฬิกา 47 เรือน แต่การนับอินสแตนซ์ของส่วนประกอบ ClockMovement จากฉากจริงได้ **46** ในรายงานให้ใช้เลข 46 และระบุวิธีนับกำกับไว้ว่านับจากจำนวนอินสแตนซ์ของ ClockMovement

กลไกของ Prefab ยังเป็นที่มาของข้อผิดพลาดประเภทหนึ่งที่พบในโครงงานนี้ คือ **ค่าเริ่มต้นที่เขียนไว้ในซอร์สโค้ดของฟิลด์ที่มี** `[SerializeField]` **จะถูกเขียนทับด้วยค่าที่ถูก serialize ไว้ในแอสเซ็ตเสมอตอนที่ Unity โหลดอ็อบเจกต์** ดังนั้นการแก้ค่าเริ่มต้นในโค้ดจึงไม่มีผลต่อ Prefab ที่ถูกสร้างไปก่อนหน้านั้นแล้ว กรณีที่พบจริงในโครงงานและผลกระทบอยู่ในบทที่ 5 หัวข้อ 5.2

2.5.6 ScriptableObject

`ScriptableObject` เป็นชนิดข้อมูลของ Unity ที่สืบทอดจาก `UnityEngine.Object` และ serialize ได้ ต่างจาก `MonoBehaviour` ที่ว่า **อ็อบเจกต์ของมันไม่ได้แปะอยู่กับ GameObject ในฐานะ Component แต่ดำรงอยู่ในโปรเจกต์ในฐานะแอสเซ็ต เป็นอิสระจาก GameObject** เอกสารของ Unity ระบุว่าคุณค่าหลักของ `ScriptableObject` คือการเป็นที่เก็บข้อมูล และการใช้งานที่พบบ่อยคือการเป็นภาชนะสำหรับข้อมูลที่อ็อบเจกต์หลายตัวใช้ร่วมกันตอนทำงาน ซึ่งช่วยลดการใช้หน่วยความจำโดยหลีกเลี่ยงการทำสำเนาค่าเดียวกันหลายชุด [12]

โครงงานนี้ใช้ `ScriptableObject` เป็นกลไกของ Data-Driven Design ตามหัวข้อ 2.2.6 รวม 3 คลาส และ 12 ไฟล์แอสเซ็ต ได้แก่ `ItemData`, `HousePlan` และ `ClockRecipe`

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📷 **คำแนะนำการแคปภาพ — ลบออกก่อนส่ง รูปที่ 2.3** หน้าต่าง Inspector ของไฟล์ `Assets/Recipes/HousePlan.asset` โดยกางรายการห้องและรายการแนวผนังให้เห็น เพื่อแสดงว่าผังบ้านทั้งหลังเป็นข้อมูลไม่ใช่โค้ด คำบรรยาย: `รูปที่ 2.3 ข้อมูลผังบ้านที่เก็บในรูปแอสเซ็ต ScriptableObject`

[วางรูปที่ 2.3 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ `fig-2-03.png`

รูปที่ 2.3 ข้อมูลผังบ้านที่เก็บในรูปแอสเซ็ต ScriptableObject

2.5.7 Assembly Definition

Assembly Definition (ไฟล์นามสกุล .asmdef) เป็นกลไกของ Unity ที่ใช้แบ่งสคริปต์ในโปรเจกต์ออกเป็นแอสเซมบลีหลายชุด แทนที่จะคอมไพล์รวมอยู่ในแอสเซมบลีที่เอนจินสร้างให้โดยปริยาย (predefined assembly) ซึ่งมีชื่อว่า Assembly-CSharp ประโยชน์คือลดเวลาคอมไพล์เมื่อแก้โค้ด และ**ทำให้ขอบเขตการพึ่งพาระหว่างกลุ่มโค้ดกลายเป็นข้อบังคับระดับคอมไพเลอร์**

กฎการอ้างอิงที่สำคัญสามข้อซึ่งเอกสารของ Unity ระบุไว้อย่างชัดเจน คือ [23]

1. โดยปริยาย **predefined assembly อ้างอิงแอสเซมบลีอื่นทั้งหมด** รวมถึงแอสเซมบลีที่สร้างด้วย Assembly Definition เอกสารใช้ถ้อยคำว่า *"คลาสใน predefined assembly สามารถใช้ชนิดข้อมูลทั้งหมดที่นิยามไว้ในแอสเซมบลีอื่นใดในโปรเจกต์ได้"*
2. Unity **ไม่อนุญาต** ให้แอสเซมบลีที่สร้างด้วย Assembly Definition อ้างอิงไปยัง predefined assembly
3. Unity **ไม่อนุญาต** การอ้างอิงแบบวงจร (cyclical reference) คือกรณีที่สองแอสเซมบลีอ้างถึงกันไปกลับ เอกสารระบุว่าหากพบข้อผิดพลาดนี้ ต้องปรับโครงสร้างโค้ดเพื่อตัดวงจรออก หรือย้ายคลาสที่อ้างถึงกันไปอยู่ในแอสเซมบลีเดียวกัน

กฎข้อ 1 และ 2 ร่วมกันให้ผลที่มีประโยชน์เชิงสถาปัตยกรรมอย่างมาก คือ **การวางโค้ดชั้นล่างไว้ในไฟล์ .asmdef และวางโค้ดชั้นบนไว้ใน Assembly-CSharp เป็นการบังคับทิศทางการพึ่งพาให้เป็นทางเดียวโดยอัตโนมัติ** โดยไม่ต้องอาศัยวินัยของผู้เขียนโค้ด

ตำแหน่งที่พบในโครงงาน โครงงานใช้กลไกนี้ตรงตามรูปแบบที่กล่าวมา คือโมดูลชั้นล่างห้าโมดูลมี .asmdef ของตนเอง (EscapeRoom.Items, EscapeRoom.Interaction, EscapeRoom.Clock, EscapeRoom.Puzzle, EscapeRoom.World) ส่วนโมดูลชั้นบนสามโมดูล (Player, UI, Game) ถูกปล่อยไว้ใน Assembly-CSharp ผลคือการเขียนคำสั่งที่อ้างถึงคลาสของผู้เล่นจากภายในโมดูลนาฬิกา เช่น GetComponent<PlayerMovement>() ภายใน ClockDial **จะคอมไพล์ไม่ผ่าน** ตารางการพึ่งพาในหัวข้อ 2.1.2 จึงไม่ใช่เพียงข้อตกลง แต่เป็นข้อผิดพลาดของคอมไพเลอร์

เหตุผลที่ไม่แบ่ง .asmdef ให้ครบทั้งเก้าโมดูล มีข้อจำกัดทางเทคนิคบังคับอยู่สองข้อ

1. **มีวงจร Player ⇄ UI อยู่จริง** ซึ่งขัดกับกฎข้อ 3 โดยตรง สองโมดูลนี้จึงใส่ .asmdef แยกกันไม่ได้ ต้องตัดวงจรก่อน หรือรวมเป็นแอสเซมบลีเดียว

2. **ไฟล์ PlayerControls.cs อยู่ที่ Assets/ ไม่ได้อยู่ใต้ Assets/Scripts/** หากสร้าง EscapeRoom.Player.asmdef ขึ้น คลาสในนั้นจะมองไม่เห็นคลาส PlayerControls เพราะอยู่ต่างแอสเซมบลีกัน และตามกฎข้อ 2 แอสเซมบลีใหม่นั้นจะอ้างถึง predefined assembly ที่ PlayerControls อยู่ไม่ได้ ต้องย้ายไฟล์ก่อน

[หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ข้อจำกัดที่พบเพิ่มเติม** — ไฟล์ EscapeRoom.Puzzle.asmdef ประกาศการอ้างอิงไปยัง EscapeRoom.Items และ EscapeRoom.Clock แต่โค้ดในโฟลเดอร์ Puzzle/ ไม่ได้ใช้ชนิดข้อมูลจากสองโมดูลนั้นเลย (รายการเดียวที่พบคือคำว่า IClockDisplay ในคอมเมนต์ที่ ICodeInput.cs:5) และ EscapeRoom.Tests.asmdef ประกาศอ้าง EscapeRoom.Puzzle ที่ไม่ได้ใช้เช่นกัน การประกาศเกินนี้ไม่กระทบการทำงาน แต่ทำให้ขอบเขตของโมดูลหลวมกว่าที่โค้ดต้องการจริง รายละเอียดอยู่ในบทที่ 5 หัวข้อ 5.3

[หมายเหตุของร่าง — ลบก่อนส่ง] 📷 **คำแนะนำการแคปภาพ — ลบออกก่อนส่ง รูปที่ 2.4** หน้าต่าง Inspector ของไฟล์ Assets/Scripts/Clock/EscapeRoom.Clock.asmdef โดยให้เห็นช่อง Name และรายการ Assembly Definition References ที่มี EscapeRoom.Items และ EscapeRoom.Interaction อยู่ คำบรรยาย: รูปที่ 2.4 การประกาศขอบเขตการอ้างอิงของโมดูลนาฬิกาด้วยไฟล์ Assembly Definition



รูปที่ 2.4 การประกาศขอบเขตการอ้างอิงของโมดูลนาฬิกาด้วยไฟล์ Assembly Definition

2.5.8 Unity Input System

Unity Input System เป็นแพ็กเกจระบบรับข้อมูลนำเข้ารุ่นใหม่ของ Unity ที่แยกการกระทำในเกม (action) ออกจากอุปกรณ์และปุ่มที่ใช้สั่งการกระทำนั้น (binding) โดยนิยามไว้ในไฟล์ .inputactions และ Unity จะสร้างคลาส C# ให้อัตโนมัติจากไฟล์นั้น นอกจากนี้ยังมีแนวคิดของ **Action Map** คือกลุ่มของการกระทำที่เปิดหรือปิดพร้อมกันเป็นชุด [24]

โครงงานนี้ใช้ **Input System เวอร์ชัน 1.17.0** โดยคลาสที่ถูกสร้างอัตโนมัติคือ Assets/PlayerControls.cs และสถาปัตยกรรมการรับข้อมูลนำเข้าของโครงงานมีสองข้อตกลงที่สำคัญ

ข้อแรก **คลาส InputReader เป็นเจ้าของอ็อบเจกต์ PlayerControls เพียงตัวเดียวของเกมทั้งเกม** สร้างใน Awake และคืนทรัพยากรด้วย Dispose() ใน OnDestroy (InputReader.cs:19-20) เหตุผลคือเกมมีการสลับ Action Map สามชุด หากมีคลาสอื่นสร้างอ็อบเจกต์ PlayerControls ขึ้นอีกตัว Action Map ที่ตัวควบคุมโหมดเข้าใจว่าปิดไปแล้วจะยังเปิดอยู่

ข้อสอง **วิธีอ่านข้อมูลนำเข้าถูกเลือกตามชนิดของค่า** ดังตารางที่ 2.9

ตารางที่ 2.9 ข้อตกลงการอ่านข้อมูลนำเข้าตามชนิดของค่า

ชนิดของค่า	ตัวอย่างในเกม	วิธีอ่าน	ตำแหน่งในโค้ด
ค่าต่อเนื่อง	การเดิน การหันมอง	เรียก ReadValue() ทุกเฟรม (polling)	PlayerMovement.cs:43, MouseLook.cs:35
เหตุการณ์การกดครั้งเดียว	ปุ่ม E, F, Q, Submit, Navigate	ลงทะเบียนรับฟังเหตุการณ์ performed	PlayerInteractor.cs:51-52, InspectView.cs:121-123, InventoryUI.cs:43-45

เหตุผลเชิงวิศวกรรมของข้อตกลงนี้คือ ปุ่มที่อ่านด้วยการ polling ทุกเฟรมจะถูกอ่านซ้ำหรือถูกอ่านพลาด ขึ้นกับว่าเฟรมตกลงตรงจังหวะใดของการกด ขณะที่ค่าต่อเนื่องที่อ่านด้วยการรับฟังเหตุการณ์จะได้ค่าเฉพาะขณะที่ค่าเปลี่ยน ไม่ได้ค่าในขณะที่ผู้เล่นกดปุ่มค้างไว้

ข้อตกลงที่ตามมาอีกข้อหนึ่งคือ **การกระทำที่ถูกปิดจะคืนค่าโดยปริยาย (default) อยู่แล้ว จึงไม่ต้องล้างค่า แต่ค่าที่ถูกเก็บสะสมไว้ในฟิลด์ต้องถูกล้างใน OnDisable** มิฉะนั้นค่านั้นจะข้ามไปอยู่ในการเปิดใช้งานครั้งถัดไป โครงงานปฏิบัติตามข้อนี้จริงที่ PlayerMovement.cs:59-63 (ล้างค่าความเร็วและความเร็วการตก), HeadBob.cs:71-76, Flicker.cs:47-53, CuckooBird.cs:41-44 และ RunTicking.cs:51-56

2.5.9 Unity Test Framework และ NUnit

Unity Test Framework เป็นแพ็กเกจที่ใช้เขียนและรันการทดสอบอัตโนมัติใน Unity โดยสร้างอยู่บนเฟรมเวิร์ก **NUnit** ซึ่งเป็นเฟรมเวิร์กการทดสอบหน่วยสำหรับภาษาในตระกูล .NET

[26] และรองรับการทดสอบสองโหมดตามที่อธิบายไว้ในหัวข้อ 2.4.1 คือ EditMode และ PlayMode [25]

โครงงานนี้ใช้ **Unity Test Framework เวอร์ชัน 1.6.0** ยืนยันจาก Packages/manifest.json และจากไฟล์ Assets/Tests/PlayMode/EscapeRoom.Tests.asmdef ซึ่งประกาศการอ้างอิงไปยัง UnityEngine.TestRunner, UnityEditor.TestRunner และ nunit.framework.dll พร้อมตั้งค่า defineConstraints เป็น ["UNITY_INCLUDE_TESTS"] เพื่อให้แอสเซมบลีนี้ถูกคอมไพล์เฉพาะเมื่อมีการรวมชุดทดสอบเท่านั้น

กรณีทดสอบในโครงงานใช้แอตทริบิวต์ [UnityTest] ร่วมกับเมธอดที่คืนค่าชนิด IEnumerator และคำสั่ง yield return null ซึ่งเป็นรูปแบบที่ทำให้เฟรมเดินไปข้างหน้าหนึ่งเฟรมระหว่างการทดสอบ จำเป็นสำหรับการทดสอบสิ่งที่ทำงานใน Update และ LateUpdate ส่วนการตรวจผลลัพธ์ใช้คลาส Assert ของ NUnit

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📷 **คำแนะนำการแคปภาพ — ลบออกก่อนส่ง รูปที่ 2.5** หน้าต่าง Test Runner ของ Unity (Window > General > Test Runner) แท็บ PlayMode ที่กางรายการกรณีทดสอบทั้ง 15 กรณีให้เห็นชื่อครบ **และแสดงผลว่าผ่านทั้งหมด** ⚠ ภาพนี้ต้องรัน Test Runner เองก่อน เพราะผู้ช่วยวิจัยไม่ได้รับอนุญาตให้รันการทดสอบในโครงงาน ภาพเดียวกันนี้จะถูกใช้อ้างในบทที่ 4 ด้วย คำบรรยาย: รูปที่ 2.5 ผลการรันชุดทดสอบ PlayMode ทั้ง 15 กรณีด้วย Unity Test Framework



รูปที่ 2.5 ผลการรันชุดทดสอบ PlayMode ทั้ง 15 กรณีด้วย Unity Test Framework

2.5.10 การสร้างเนื้อหาด้วยกระบวนวิธีและการเขียนสคริปต์สำหรับตัวแก้ไข

การสร้างเนื้อหาด้วยกระบวนวิธี (Procedural Content Generation หรือ PCG) คือการสร้างเนื้อหาของเกม เช่น ภูมิประเทศ ด่าน พื้นผิว หรือรูปทรง ด้วยอัลกอริทึม

แทนการสร้างด้วยมือทีละชิ้น Hendrikx และคณะจัดหมวดเนื้อหาที่สร้างด้วยวิธีนี้ไว้หลายระดับ ตั้งแต่ระดับพื้นผิวและรูปทรง ไปจนถึงระดับการวางผังด่านและกฎของเกม [27] และ Shaker, Togelius กับ Nelson อธิบายว่าประโยชน์หลักของแนวทางนี้คือการลดต้นทุนการผลิตเนื้อหา และการทำให้เนื้อหาเปลี่ยนแปลงได้จากการแก้พารามิเตอร์ [28]

ในกรณีของ Unity การสร้างเนื้อหาด้วยกระบวนวิธีมักทำผ่านการเขียนสคริปต์สำหรับตัวแก้ไข (Editor Scripting) คือสคริปต์ที่ทำงานในตัวแก้ไขเท่านั้นและไม่ถูกรวมไปในเกมที่ส่งออก โดยวางไว้ในโฟลเดอร์ชื่อ Editor ซึ่งทำให้ Unity คอมไพล์เข้าแอสเซมบลี Assembly-CSharp-Editor

ตำแหน่งที่พบในโครงงาน โครงงานนี้มีเครื่องมือสร้างเนื้อหาในโฟลเดอร์ Assets/Scripts/Editor/ รวม **11 ไฟล์ 5,772 บรรทัด** ซึ่งมากกว่าโค้ดส่วนรันไทม์ทั้งหมด (3,481 บรรทัด) โดยทุกตัวเลขบรรทัดในรายงานนี้นับเฉพาะบรรทัดที่ไม่ว่าง เครื่องมือเหล่านี้สร้างทั้งผังบ้าน รูปทรงของวัตถุ พื้นผิว และ Prefab โดยมีรายการตามตารางที่ 2.10

ตารางที่ 2.10 เครื่องมือสร้างเนื้อหาด้วยกระบวนวิธีในโครงงาน

เครื่องมือ	สิ่งที่สร้าง	ขอบเขตที่แก้ไข
HouseBuilder	พื้น ผนัง ฝ้า ช่องเปิด และคิ้ว จากข้อมูลใน HousePlan.asset	กลุ่มอ็อบเจกต์ House/Shell เท่านั้น
DressingBuilder	เครื่องเรือนและของตกแต่งในแต่ละห้อง	กลุ่ม Dressing ของแต่ละห้องเท่านั้น
ClockBuilder, ClockVariants	Prefab นาฬิกาและหน้าปัด	Assets/Prefabs/Clocks/
ClockAssembler	Prefab นาฬิกาจากไฟล์ ClockRecipe.asset	PlankClock.prefab
PuzzleBuilder, MissingThingsBuilder	วัตถุประกอบปริศนาแต่ละองก์	วัตถุเฉพาะตัว
NewspaperBuilder	หนังสือพิมพ์บนโต๊ะ	Room_I_Showroom/Newspaper
TextureKit	พื้นผิวทั้งหมดในเกม เขียนเป็นไฟล์ลง Assets/Textures	Assets/Textures/
BuildKit, InterfaceBuilder	ฟังก์ชันช่วยที่เครื่องมืออื่นเรียกใช้	—

ประเด็นทางวิศวกรรมที่สำคัญของแนวทางนี้คือ **ความเป็น idempotent ของตัวสร้างเนื้อหา** กล่าวคือเครื่องมือที่ทำงานโดยการลบของเดิมแล้วสร้างใหม่ **เมื่อเทียบกับข้อมูลที่มนุษย์แก้ไขไว้** โครงงานนี้แก้ปัญหานี้ด้วยสองวิธีที่พบหลักฐานในฉากจริง จะกลืนงานที่คนแก้ไขไว้ด้วยมือไปด้วย คือ (ก)

การย้ายวัตถุที่คนแก้ไขออกไปอยู่นอกขอบเขตของเครื่องมือ ยืนยันจากตำแหน่งในฉากที่ Room_II_Workshop/Sandglass และ Room_I_Showroom/Newspaper เป็นอ็อบเจกต์ลูกโดยตรงของห้อง ไม่ได้อยู่ใต้กลุ่ม Dressing ขณะที่ของตกแต่งอื่นอยู่ใต้ Dressing ทั้งหมด และ (ข) การอ่านค่าที่คนเขียนไว้ออกมาเก็บก่อน แล้วใส่กลับหลังสร้างใหม่

2.5.11 เครื่องมือพัฒนาอื่นและ Model Context Protocol

เครื่องมือพัฒนาอื่นที่พบหลักฐานการใช้งานในโครงงาน ได้แก่

1. **แพ็กเกจ uGUI เวอร์ชัน 2.0.0** ใช้สร้างส่วนติดต่อผู้ใช้ ยืนยันจากคำสั่ง using UnityEngine.UI และการใช้ชนิดข้อมูล Text แบบเดิม (legacy), Graphic, Canvas และ RectTransform

2. **ส่วนขยายสำหรับสภาพแวดล้อมการพัฒนา** คือ com.unity.ide.rider เวอร์ชัน 3.0.38 และ com.unity.ide.visualstudio เวอร์ชัน 2.0.26 และพบโฟลเดอร์ .vscode/ ในโปรเจกต์

3. **Model Context Protocol (MCP)** เป็นมาตรฐานเปิดสำหรับเชื่อมต่อแอปพลิเคชันปัญญาประดิษฐ์เข้ากับระบบภายนอก เช่น แหล่งข้อมูล เครื่องมือ และลำดับงาน [29] โครงงานนี้ใช้แพ็กเกจ com.coplaydev.unity-mcp เพื่อเปิดช่องให้ควบคุม Unity Editor จากภายนอกผ่านโปรโตคอลนี้ ยืนยันจากไฟล์ .mcp.json ที่ตั้งค่าปลายทางไว้ที่ http://127.0.0.1:8080/mcp

[หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ข้อควรระวังสำหรับผู้เขียน** — ไฟล์ Packages/manifest.json มีรายการ dependency รวม 46 รายการ แต่ส่วนใหญ่เป็นโมดูลที่ Unity ใส่มาตามแม่แบบของโปรเจกต์ ไม่ใช่การตัดสินใจทางเทคนิค โดยเฉพาะ com.unity.ai.navigation, com.unity.timeline, com.unity.visualscripting และ com.unity.multiplayer.center

ไม่พบการใช้งานในซอร์สโค้ดหรือในฉากเลย (ยืนยันจากการค้นหา using Unity.AI.Navigation, using UnityEngine.Timeline, NavMeshAgent และ PlayableDirector ได้ผลลัพธ์ศูนย์รายการ)

ในรายงานห้ามเขียนว่าโครงงานใช้ NavMesh หรือ Timeline และเทคโนโลยีที่เขียนลงบทที่ 2 ควรมีเฉพาะที่พบหลักฐานการใช้งานจริงตามหัวข้อ 2.5.1 ถึง 2.5.11 เท่านั้น

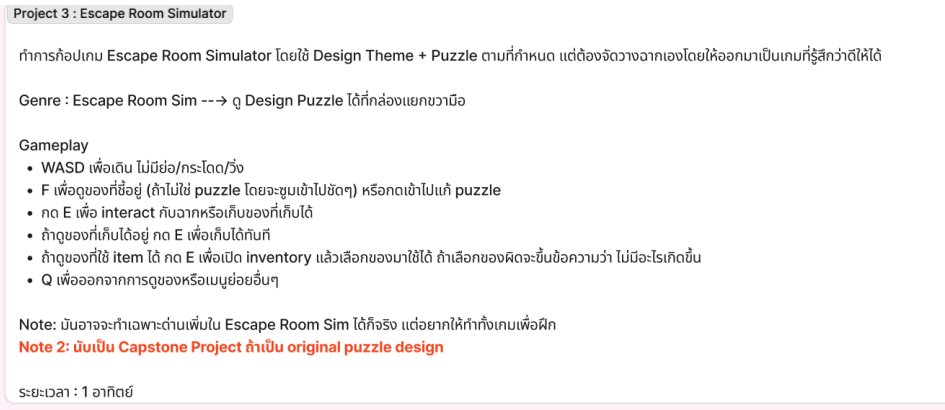
บทที่ 3
ขั้นตอนการดำเนินงาน

การปฏิบัติสหกิจศึกษา ณ บริษัท Lumas corp ในตำแหน่ง Game Developer ประกอบด้วยงานที่ได้รับมอบหมายจากพนักงานที่ปรึกษา และการทำโครงงานสหกิจศึกษาคือเกม Escape Room Simulator ซึ่งเป็นเกมแนวห้องปริศนามุมมองบุคคลที่หนึ่ง พัฒนาด้วยโปรแกรม Unity โดยพนักงานที่ปรึกษาเป็นผู้กำหนดรูปแบบการเล่นและกรอบของงานมาให้ ส่วนการออกแบบปริศนา การวางฉาก และการออกแบบโครงสร้างของซอฟต์แวร์ทั้งหมดเป็นงานที่ผู้จัดทำดำเนินการเอง รายละเอียดของแผนงาน ขอบเขต เครื่องมือ และลำดับขั้นการดำเนินงานมีดังนี้

3.1 แผนงานการทำโครงงาน

โจทย์ที่ได้รับจากพนักงานที่ปรึกษาเป็นเอกสารหนึ่งหน้า กำหนดชื่องานว่า Project 3 : Escape Room Simulator ระบุแนวเกม รูปแบบการควบคุม และเงื่อนไขของงานไว้ครบถ้วน ดังรูปที่ 3.1

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** **รูปที่ 3.1 — มีไฟล์อยู่แล้ว ไม่ต้องถ่ายใหม่:** ใช้ `EscapeReport/picture/Design.png` ได้ทันที



Project 3 : Escape Room Simulator

ทำการก้อปเกม Escape Room Simulator โดยใช้ Design Theme + Puzzle ตามที่กำหนด แต่ต้องจัดวางฉากเองโดยให้ออกมาเป็นเกมที่รู้สึกว่าดีให้ได้

Genre : Escape Room Sim ---> ดู Design Puzzle ได้ที่กล่องแยกขวามือ

Gameplay

- WASD เพื่อเดิน ไม่มีย่อ/กระโดด/วิ่ง
- F เพื่อดูของที่ซื้อยู่ (ถ้าไม่ใช่ puzzle โดยจะซูมเข้าไปชัดๆ) หรือกดเข้าไปแก้ puzzle
- กด E เพื่อ interact กับฉากหรือเก็บของที่เก็บได้
- ถ้าดูของที่เก็บได้อยู่ กด E เพื่อเก็บได้ทันที
- ถ้าดูของที่ใช้ item ได้ กด E เพื่อเปิด inventory แล้วเลือกของมาใช้ได้ ถ้าเลือกของผิดจะขึ้นข้อความว่า ไม่มีอะไรเกิดขึ้น
- Q เพื่อออกจากการดูของหรือเมนูย่อยอื่นๆ

Note: มันอาจจะทำเฉพาะด่านเพิ่มใน Escape Room Sim ได้ก็จริง แต่อยากให้ทำทั้งเกมเพื่อฝึก

Note 2: นับเป็น Capstone Project ถ้าเป็น original puzzle design

ระยะเวลา : 1 อาทิตย์

รูปที่ 3.1 โจทย์ของโครงงานที่ได้รับจากพนักงานที่ปรึกษา

เอกสารโจทย์กำหนดระยะเวลาไว้ **1 สัปดาห์** และระบุเพิ่มเติมว่าโครงงานนี้จะนับเป็น Capstone Project ได้หากการออกแบบปริศนาเป็นผลงานต้นฉบับของผู้จัดทำเอง ซึ่งเป็นเงื่อนไขที่ส่งผลโดยตรงต่อการวางแผน กล่าวคือ

ส่วนของกลไกการเล่นที่โจทย์กำหนดมาแล้วสามารถลงมือพัฒนาได้ทันที
แต่ส่วนของปริศนาต้องใช้เวลาออกแบบก่อนจึงจะเขียนโปรแกรมได้

เนื่องจากโครงงานนี้ไม่มีการใช้ระบบควบคุมเวอร์ชัน (version control) จึงไม่มีประวัติการปรับปรุงให้อ้างอิงได้ แผนงานในหัวข้อนี้จึงสร้างขึ้นจาก **เวลาที่ไฟล์ในโปรเจกต์ถูกเขียนครั้งล่าสุด (last write time)** ซึ่งเก็บไว้เป็นรายการไฟล์ทั้งโปรเจกต์จำนวน 797 ไฟล์ ช่วงเวลาที่ปรากฏคือตั้งแต่ **วันที่ 25 สิงหาคม 2569 เวลา 06:24 น.** ซึ่งเป็นเวลาที่ไฟล์ `ProjectSettings/ProjectVersion.txt` ถูกสร้างพร้อมกับการสร้างโปรเจกต์ ไปจนถึง **วันที่ 8 กันยายน 2569 เวลา 23:19 น.** ซึ่งเป็นเวลาที่ไฟล์ `ProjectSettings/ProjectSettings.asset` ถูกเขียนเป็นครั้งสุดท้าย รวมเป็นช่วงเวลา **15 วันตามปฏิทิน** และมี **10 วันที่มีการทำงานจริง**

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อจำกัดของหลักฐานชุดนี้ ต้องเขียนกำกับไว้ในรายงาน:**
> เวลาแก้ไขไฟล์เป็นหลักฐานที่อ่อนกว่าประวัติจากระบบควบคุมเวอร์ชัน
> เพราะไฟล์ที่ถูกคัดลอกหรือย้ายอาจได้เวลาใหม่ และไฟล์ที่ถูกแก้หลายครั้งจะเหลือให้เห็นเพียงครั้งสุดท้าย
> ดังนั้นตารางในหัวข้อนี้บอกได้ว่า "งานชิ้นนั้นเสร็จสิ้นเมื่อใด" แต่บอกไม่ได้ว่า "เริ่มทำเมื่อใด"

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ? **ต้องให้ผู้เขียนยืนยัน:** โจทย์ระบุ 1 สัปดาห์
> แต่หลักฐานจากไฟล์คือ 15 วัน ต้องระบุในรายงานว่านับช่วงใดเป็นหนึ่งสัปดาห์
> หรือเขียนตามข้อเท็จจริงว่าใช้เวลาเกินกว่าที่โจทย์กำหนด และช่วงฝึกงานจริงคือเดือนกรกฎาคมถึงตุลาคม
> ซึ่งยังไม่มีวันที่เริ่มและสิ้นสุดที่แน่นอน

ตารางที่ 3.1 แผนการปฏิบัติงานและการทำโครงงาน

ลำดับ	กิจกรรม	สัปดาห์ที่ 1 25–29 ส.ค.	สัปดาห์ที่ 2 31 ส.ค. – 4 ก.ย.	สัปดาห์ที่ 3 7–8 ก.ย.	หลังจบการพัฒนา
1	รับโจทย์และศึกษาข้อกำหนดของงาน	■			
2	สร้างโปรเจกต์และติดตั้งเครื่องมือพัฒนา	■			
3	พัฒนาระบบควบคุมผู้เล่นและการรับคำสั่ง	■			
4	พัฒนาระบบโต้ตอบกับวัตถุและระบบนาฬิกา		■		
5	ปรับโครงสร้างแยกส่วนต่อประสานและตัดการพึ่ง		■	■	

ลำดับ	กิจกรรม	สัปดาห์ที่ 1 25–29 ส.ค.	สัปดาห์ที่ 2 31 ส.ค. – 4 ก.ย.	สัปดาห์ที่ 3 7–8 ก.ย.	หลังจบการพัฒนา
	พา				
6	บังคับขอบเขตโมดูลและเขียนชุดทดสอบ		▰	▰	
7	สร้างฉากและทรัพยากรด้วยสคริปต์สำหรับตัวแก้ไข		▰	▰	
8	ออกแบบและพัฒนาปริศนาทั้งห้าองก์		▰	▰	
9	แก้ไขความถูกต้องของการนับเวลา			▰	
10	ทดสอบกับผู้เล่นและจัดทำรูปเล่มรายงาน				▰

[หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **แถวที่ 1 และแถวที่ 10 ไม่มีหลักฐานจากไฟล์** การรับโจทย์เกิดก่อนไฟล์แรกของโปรเจกต์ และการทดสอบกับผู้เล่น 5 คนเป็นข้อมูลที่ผู้เขียนให้มาโดยไม่ระบุวันที่ หากต้องการระบุช่วงเวลาของสองแถวนี้ในรูปเล่ม ผู้เขียนต้องเป็นผู้กรอกเอง

เมื่อนับจำนวนไฟล์ในโปรเจกต์ตามวันที่ถูกเขียนครั้งล่าสุด จะได้ภาพปริมาณงานรายวันดังตารางที่ 3.2 ซึ่งแสดงให้เห็นว่างานไม่ได้กระจายสม่ำเสมอ แต่กระจุกตัวอยู่ในสามวัน คือวันที่ 2, 7 และ 8 กันยายน

ตารางที่ 3.2 จำนวนไฟล์ที่ถูกเขียนครั้งล่าสุดในแต่ละวันของการพัฒนา

วันที่	วัน	ช่วงเวลาที่ทำงาน	ไฟล์ทั้งหมด	ไฟล์ .cs	งานหลักของวันนั้น
25 ส.ค. 2569	อังคาร	06:24–20:40	69	0	สร้างโปรเจกต์ ติดตั้งเครื่องมือ นำเข้าโมเดลตัวละคร สร้างวัสดุชุดแรก
26 ส.ค. 2569	พุธ	02:44–15:10	18	4	ระบบรับคำสั่ง การเดิน การหันกล้อง แสงของห้อง
27 ส.ค. 2569	พฤหัสบดี	13:26–17:11	20	1	สภาพของนาฬิกาในรูปชนิดข้อมูลแจงนับ
28–30 ส.ค.	*ศุกร์–อาทิตย์*	—	0	0	*ไม่มีไฟล์ใดถูกเขียน*

วันที่	วัน	ช่วงเวลาที่ทำงาน	ไฟล์ทั้งหมด	ไฟล์ .cs	งานหลักของวันนั้น
2569					
31 ส.ค. 2569	จันทร์	11:10–19:51	93	5	กลไกเปิด–ปิด ระบบของในกระเป๋า พื้นผิวชุดแรก 44 ภาพ
1 ก.ย. 2569	อังคาร	05:40–11:11	36	3	รูปแบบการดูวัตถุ ส่วนต่อประสานของชิ้นส่วนนาฬิกา
2 ก.ย. 2569	**พุธ**	**00:13–18:43**	**155**	**28**	**ปรับโครงสร้างครั้งที่หนึ่ง ใส่ Assembly Definition เขียนชุดทดสอบชุดแรก**
3 ก.ย. 2569	พฤหัสบดี	12:41–23:40	33	7	ระบบอ่านเอกสาร เครื่องมือวาดพื้นผิว ระบบเล็งและหยิบ
4 ก.ย. 2569	ศุกร์	00:03–21:15	74	6	นาฬิกาทราย ผังบ้านในรูปข้อมูล ปริศนาองก์ที่ 5
5–6 ก.ย. 2569	*เสาร์–อาทิตย์*	—	0	0	*ไม่มีไฟล์ใดถูกเขียน*
7 ก.ย. 2569	**จันทร์**	**10:46–19:29**	**180**	**21**	**ปริศนาองก์ที่ 1, 3, 4 ปรับโครงสร้างครั้งที่สอง ชุดทดสอบชุดที่สาม**
8 ก.ย. 2569	อังคาร	06:19–23:19	16	3	แก้ไขความถูกต้องของการนับเวลา สร้างทรัพยากรรอบสุดท้าย บันทึกฉาก
		รวม	**694**	**78**	

> **หมายเหตุ** จำนวนไฟล์รวมในตารางที่ 3.2 คือ 694 ไฟล์ ไม่ใช่ 797 ไฟล์ เพราะส่วนที่เหลืออีก 103 ไฟล์เป็นทรัพยากรที่นำเข้าจากภายนอกและไฟล์ที่มาพร้อมแม่แบบของ Unity ซึ่งมีวันที่ของตนเองอยู่ก่อนแล้ว คือ พ.ศ. 2562 จำนวน 73 ไฟล์ พ.ศ. 2567 จำนวน 1 ไฟล์ และ พ.ศ. 2568 จำนวน 29 ไฟล์ รายละเอียดอยู่ในหัวข้อ 3.2.3

ข้อสังเกตที่ได้จากตารางที่ 3.2 คือ **ช่วงที่ไม่มีไฟล์ใดถูกเขียนเลยทั้งสองช่วงตรงกับวันหยุดสุดสัปดาห์พอดี** ได้แก่วันเสาร์ที่ 29 สิงหาคมถึงวันอาทิตย์ที่ 30 สิงหาคม และวันเสาร์ที่ 5 กันยายนถึงวันอาทิตย์ที่ 6 กันยายน

ซึ่งสอดคล้องกับลักษณะของงานที่ทำในวันทำการของสถานประกอบการ ส่วนวันศุกร์ที่ 28 สิงหาคมไม่มีไฟล์ใดถูกเขียนเช่นกัน ซึ่งไม่สามารถระบุสาเหตุได้จากหลักฐานที่มี

3.2 รายละเอียดขั้นตอนการทำโครงงาน

โครงงานนี้เป็นการพัฒนาเกมห้องปริศนามุมมองบุคคลที่หนึ่งขึ้นใหม่ทั้งเกม โดยมีเป้าหมายทางวิศวกรรมคือการออกแบบให้ส่วนประกอบของเกมสามารถนำกลับมาใช้ซ้ำและขยายได้โดยไม่ต้องแก้ไขโค้ดเดิม มีรายละเอียดดังนี้

3.2.1 ขอบเขตของโครงงาน

ขอบเขตของโครงงานแบ่งได้เป็นสองส่วน คือส่วนที่โจทย์กำหนดมากับส่วนที่ผู้จัดทำกำหนดขึ้นเอง

ส่วนที่โจทย์กำหนด คือรูปแบบการควบคุมและการโต้ตอบ ซึ่งระบุไว้ในเอกสารโจทย์ตามรูปที่ 3.1 ตารางที่ 3.3 เปรียบเทียบข้อกำหนดแต่ละข้อกับส่วนของโปรแกรมที่รองรับข้อกำหนดนั้น

ตารางที่ 3.3 ข้อกำหนดจากโจทย์และส่วนของโปรแกรมที่รองรับ

ข้อกำหนดจากโจทย์	ส่วนของโปรแกรมที่รองรับ	สถานะ
ใช้ปุ่ม WASD เพื่อเดิน ไม่มีการย่อ กระโดด หรือวิ่ง	PlayerMovement.cs มีเฉพาะการเดินและแรงโน้มถ่วง ไม่มีฟังก์ชันกระโดดหรือย่อ	ทำครบ
ปุ่ม F เพื่อดูวัตถุใกล้ ๆ โดยถ้าเป็นปริศนาจะซูมเข้าไป	InspectStyle กำหนดรูปแบบการดูไว้สามแบบ คือ Hold Turn และ Zoom	ทำครบและขยายเพิ่ม
ปุ่ม E เพื่อโต้ตอบกับฉากหรือเก็บของ	PlayerInteractor.cs ส่งคำสั่งให้วัตถุที่อยู่ใต้เป้าเล็งผ่านส่วนต่อประสาน IInteractable	ทำครบ
ถ้าเป็นของที่เก็บได้ กด E เพื่อเก็บทันที	PickupItem.cs ซึ่งเป็นหนึ่งในผู้ทำตาม IInteractable	ทำครบ
ถ้าเป็นสิ่งที่ใช้ไอเทมได้ กด E เพื่อเปิดกระเป๋าแล้วเลือกของ	IItemReceiver ร่วมกับ InventoryUI.cs	ทำครบ
ถ้าเลือกของผิดให้ขึ้นข้อความว่า "ไม่มีอะไรเกิดขึ้น"	InventoryUI.cs:134 แสดงข้อความนี้เมื่อ IItemReceiver.Give คืนค่าเท็จ	ทำครบ
ปุ่ม Q เพื่อออกจากการดูของหรือเมนูย่อย	PlayerModeController.cs สลับชุดคำสั่งกลับสู่โหมดสำรวจ	ทำครบ

ข้อกำหนดจากโจทย์	ส่วนของโปรแกรมที่รองรับ	สถานะ
ทำทั้งเกม ไม่ใช่เพิ่มด่านในเกมที่มีอยู่แล้ว	โปรเจกต์สร้างขึ้นใหม่ทั้งหมด มีฉากเดียวคือ Main.unity	ทำครบ
การออกแบบปริศนาต้องเป็นผลงานต้นฉบับ	ปริศนาทั้งห้าองก์ออกแบบขึ้นเองทั้งหมด	ทำครบ

ส่วนที่ผู้จัดทำกำหนดขึ้นเอง คือเนื้อหาของเกม ได้แก่ ฉากเป็นร้านนาฬิกาเก่าจำนวนห้าห้องเรียงต่อกันเป็นเส้นทางเดียว ผู้เล่นต้องแก้ปริศนาของห้องหนึ่งจึงจะเปิดประตูไปอีกห้องหนึ่งได้ และปริศนาทั้งห้าองก์ตั้งอยู่บนเรื่องของนาฬิกาทั้งหมด ดังตารางที่ 3.4

ตารางที่ 3.4 ขอบเขตเนื้อหาของเกมที่กำหนดขึ้นเอง

องก์	ห้อง	ขนาดห้อง (เมตร)	แก่นของปริศนา
1	หน้าร้าน	6.0 × 5.0	แยกนาฬิกาที่ยังเดินออกจากนาฬิกาที่เสีย แล้วอ่านตัวเลขตามลำดับที่กำหนด
2	ห้องช่าง	6.0 × 6.8	จับเวลานาฬิกาพกด้วยนาฬิกาทราย เพื่อหาเรือนที่เดินช้า
3	ห้องนั่งเล่น	6.8 × 6.8	นับเสียงร้องของนกคุกคูสามเรือน
4	ห้องนอน	6.8 × 5.0	เรียงลำดับวัตถุสี่ชิ้นตามภาพถ่ายเก่า
5	หลังตู้นาฬิกา	9.2 × 7.5	หมุนปริซึมสามชิ้นให้ลำแสงตกตรงกับกลุ่มดาวบนจอ

ขนาดของโครงงานเมื่อเสร็จสิ้นคือ ไฟล์ซอร์สโค้ดภาษา C# จำนวน 78 ไฟล์ รวม 10,604 บรรทัด (นับบรรทัดที่ไม่ว่าง) ไฟล์ต้นแบบวัตถุ (prefab) 33 ไฟล์ วัสดุ 115 ไฟล์ และ ScriptableObject 12 ไฟล์ โดยในจำนวนซอร์สโค้ดทั้งหมดนี้แบ่งเป็นโค้ดที่ทำงานขณะเล่นเกมจริง 63 ไฟล์ 3,481 บรรทัด โค้ดเครื่องมือสำหรับตัวแก้ไข 11 ไฟล์ 5,772 บรรทัด ชุดทดสอบ 3 ไฟล์ 393 บรรทัด และไฟล์ที่โปรแกรม Unity สร้างให้อัตโนมัติจากไฟล์กำหนดคำสั่งอีก 1 ไฟล์ 958 บรรทัด

สิ่งที่อยู่นอกขอบเขตของโครงงาน และต้องระบุไว้ให้ชัดเจนเพื่อไม่ให้เกิดความเข้าใจผิด ได้แก่

- **ไม่มีระบบบันทึกและโหลดความก้าวหน้าของผู้เล่น** ผู้เล่นต้องเล่นจบในรอบเดียว
- **ไม่มีระบบให้คำใบ้** หากผู้เล่นติดที่องก์ใด เกมไม่มีกลไกช่วยเหลือ

– **ไม่มีระบบเสียงแบบสามมิติ** เสียงทั้งสี่ไฟล์ในเกมตั้งค่า `spatialBlend` เป็น 0 ซึ่งหมายถึงได้ยินเท่ากันทุกตำแหน่ง

– **ไม่มีการสร้างไฟล์ติดตั้งสำหรับผู้เล่นทั่วไป** ฉากที่อยู่ใน Build Settings มีเพียงฉากเดียวคือ `Assets/Scenes/Main.unity` และไม่พบหลักฐานว่ามีการสั่ง Build ออกมาเป็นโปรแกรมที่ทำงานได้ด้วยตนเอง

– **ไม่มีการรองรับหลายภาษา** ข้อความทั้งหมดในเกมเป็นภาษาไทยที่เขียนฝังไว้ในซอร์สโค้ดและในค่าของฟิลด์โดยตรง

3.2.2 เครื่องมือที่ใช้

เครื่องมือที่ใช้ในการพัฒนาแบ่งเป็นสามกลุ่ม คือเอนจินและแพ็กเกจที่ติดตั้งในโปรเจกต์ เครื่องมือเขียนโปรแกรม และเครื่องมือเสริมที่ใช้ขับการทำงานของตัวแก้ไข ดังตารางที่ 3.5

ตารางที่ 3.5 เครื่องมือที่ใช้ในการพัฒนาโครงงาน

กลุ่ม	เครื่องมือ	รุ่น	ใช้ทำอะไร	อ้างอิง
เอนจิน	Unity Editor	6000.3.5f2	เอนจินหลักของเกม	[21]
ไปป์ไลน์เรนเดอร์	Universal Render Pipeline	17.3.0	ไปป์ไลน์การเรนเดอร์ของโปรเจกต์	[19]
ภาษาโปรแกรม	C#	ตามรุ่นที่ Unity 6 รองรับ	ภาษาที่ใช้เขียนทั้งโครงงาน	[20]
ระบบรับคำสั่ง	Unity Input System	1.17.0	กำหนดชุดคำสั่งสามชุดและสร้างคลาส `PlayerControls` ให้อัตโนมัติ	[24]
กรอบการทดสอบ	Unity Test Framework	1.6.0	รันชุดทดสอบแบบ PlayMode	[25]
ไลบรารีข้อยืนยัน	NUnit	มาพร้อม Test Framework	เขียนข้อยืนยันในชุดทดสอบ	[26]
ตัวแก้ไขโค้ด	Visual Studio Code และ JetBrains Rider	`com.unity.ide.visualstudio` 2.0.26 และ `com.unity.ide.rider` 3.0.38	เขียนและแก้ไขซอร์สโค้ด	—

กลุ่ม	เครื่องมือ	รุ่น	ใช้ทำอะไร	อ้างอิง
สะพานเชื่อมภายนอก	Unity MCP	com.coplaydev.unity-mcp	เชื่อม Unity Editor เข้ากับเครื่องมือภายนอกผ่าน Model Context Protocol ที่ http://127.0.0.1:8080/mcp	[29]

สิ่งที่ต้องระบุเพิ่มเติมเกี่ยวกับเครื่องมือมีสามข้อ

ข้อแรก ไม่มีการใช้โปรแกรมสร้างแบบจำลองสามมิติเลย การตรวจสอบไฟล์ทั้งโปรเจกต์ไม่พบไฟล์นามสกุล .blend .obj หรือ .dae แม้แต่ไฟล์เดียว และไฟล์ .fbx ที่พบทั้งหกไฟล์อยู่ในโฟลเดอร์ Assets/Character Models/ ซึ่งเป็นชุดทรัพยากรที่นำเข้าจากภายนอก เรขาคณิตของฉาก เฟอร์นิเจอร์ นาฬิกา และวัตถุปริศนาทั้งหมดถูกสร้างขึ้นจากซอร์สโค้ดในโฟลเดอร์ Assets/Scripts/Editor/ ซึ่งเป็นเหตุผลที่โค้ดกลุ่มนี้มีขนาด 5,772 บรรทัด คิดเป็นร้อยละ 54 ของซอร์สโค้ดทั้งโครงงาน รายละเอียดอยู่ในหัวข้อ 3.3.5

ข้อที่สอง เครื่องมือถูกติดตั้งครบก่อนเริ่มเขียนโค้ดเกม ไฟล์ .mcp.json ซึ่งเป็นไฟล์กำหนดค่าของสะพาน Unity MCP ถูกเขียนเมื่อวันที่ 25 สิงหาคม 2569 เวลา 07:20 น. ซึ่งห่างจากเวลาที่สร้างโปรเจกต์เพียง 56 นาที ขณะที่ไฟล์ซอร์สโค้ดไฟล์แรกของเกมคือ InputReader.cs ถูกเขียนในวันถัดมาเวลา 10:30 น.

ข้อที่สาม โครงงานนี้ไม่มีระบบควบคุมเวอร์ชัน การตรวจสอบไม่พบโฟลเดอร์ .git .svn หรือ .hg และไม่พบไฟล์ .gitignore แม้ว่าในไฟล์ Packages/manifest.json จะมีแพ็กเกจ com.unity.collab-proxy รุ่น 2.11.2 ซึ่งเป็นแพ็กเกจของ Unity Version Control ติดตั้งอยู่ แต่ไม่พบร่องรอยการใช้งานใด ๆ คือไม่มีโฟลเดอร์ .plastic และไม่มีไฟล์กำหนดค่า ข้อจำกัดนี้ส่งผลต่อการเขียนรายงานโดยตรงตามที่อธิบายไว้ในหัวข้อ 3.1 และถูกนำไปเขียนเป็นข้อเสนอแนะในหัวข้อ 5.3

3.2.3 ทรัพยากรภายนอกที่นำมาใช้

ทรัพยากรที่ไม่ได้สร้างขึ้นในโครงงานมีสามกลุ่ม ดังตารางที่ 3.6

ตารางที่ 3.6 ทรัพยากรที่นำเข้าจากภายนอกโครงงาน

ทรัพยากร	จำนวน	ตำแหน่งในโปรเจกต์	วันที่ของไฟล์

ทรัพยากร	จำนวน	ตำแหน่งในโปรเจกต์	วันที่ของไฟล์
โมเดลตัวละครพร้อมภาพพื้นผิวและแอนิเมชัน	73 ไฟล์ (ในจำนวนนี้เป็น .fbx 6 ไฟล์)	Assets/Character Models/	20 มีนาคม 2562
ไฟล์เสียง	4 ไฟล์	Assets/Sound/ คือ clock-bg.mp3, clock-bell.mp3, clock-cuckoo.mp3 และ clock-ticking-down.mp3	ตามวันที่นำเข้า
ไฟล์ที่มาพร้อมแม่แบบ Universal 3D ของ Unity	30 ไฟล์	Assets/Settings/ และ Assets/InputSystem_Actions.inputactions	30 ตุลาคม 2567 และ 31 มกราคม 2568

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** **ต้องให้ผู้เขียนยืนยันก่อนส่งรูปเล่ม:** ที่มาและเงื่อนไขการใช้งาน (สัญญาอนุญาต) ของโมเดลตัวละครและไฟล์เสียงทั้งสี่ไฟล์ ข้อมูลนี้ไม่มีอยู่ในโปรเจกต์และต้องระบุในรายงาน เพราะเป็นทรัพยากรที่ไม่ได้สร้างเอง วันที่ของไฟล์โมเดลตัวละครคือ พ.ศ. 2562 ซึ่งเก่ากว่าโครงงานเจ็ดปี บ่งชี้ว่าเป็นชุดทรัพยากรสำเร็จรูปที่นำมาใช้ แต่ยืนยันแหล่งที่มาจากไฟล์ไม่ได้

ทรัพยากรอื่นทั้งหมดในเกม ได้แก่ ภาพพื้นผิวทุกภาพ วัสดุ 115 ไฟล์ เรขาคณิตของฉากและวัตถุทุกชิ้น รวมถึงภาพในกรอบรูป ภาพถ่ายเก่าที่เป็นคำใบ้ขององค์ที่ 4 และเป้าเล็งบนหน้าจอ ถูกสร้างขึ้นด้วยซอร์สโค้ดในโครงงานเอง

3.3 ขั้นตอนการดำเนินงานที่นักศึกษาปฏิบัติงานหรือโครงงาน

เมื่อเรียงไฟล์ทั้งโปรเจกต์ตามเวลาที่ถูกเขียนครั้งล่าสุด พบว่าการดำเนินงานแบ่งออกได้เป็นแปดขั้นที่มีขอบเขตชัดเจน ดังตารางที่ 3.7 และรูปที่ 3.2

ตารางที่ 3.7 ลำดับขั้นการดำเนินงานทั้งแปดขั้น

ขั้นที่	ชื่อขั้น	ช่วงเวลา	ผลลัพธ์หลักของขั้น
1	เตรียมโปรเจกต์ เครื่องมือ และระบบควบคุมผู้เล่น	25–27 ส.ค. 2569	ผู้เล่นเดินและหันกล้องได้

ขั้นที่	ชื่อขั้น	ช่วงเวลา	ผลลัพธ์หลักของขั้น
2	ระบบโต้ตอบกับวัตถุและโครงของระบบนาฬิกา	31 ส.ค. – 1 ก.ย. 2569	เปิด–ปิดของได้ เก็บของเข้ากระเป๋าได้ ยกของขึ้นดูได้
3	ปรับโครงสร้างครั้งที่หนึ่ง แยกส่วนต่อประสานและตัดการพึ่งพา	2 ก.ย. 2569 เวลา 04:43–14:05 น.	IClockDisplay, IPowerStore, ICodeInput และ IActor
4	บังคับขอบเขตโมดูลและเขียนชุดทดสอบชุดแรก	2 ก.ย. 2569 เวลา 17:41–18:43 น.	Assembly Definition 6 ไฟล์ และชุดทดสอบ 9 กรณี
5	สร้างโลกของเกมด้วยสคริปต์สำหรับตัวแก้ไข	3–4 ก.ย. 2569	ผังบ้านเป็นข้อมูล เครื่องมือวาดพื้นผิว และห้องทั้งห้าห้อง
6	ออกแบบและพัฒนาปริศนาทั้งห้าองก์	4 และ 7 ก.ย. 2569	ปริศนาครบห้าองก์พร้อมกุญแจที่เชื่อมแต่ละองก์เข้าด้วยกัน
7	ปรับโครงสร้างครั้งที่สองและเพิ่มชุดทดสอบ	7 ก.ย. 2569 เวลา 16:04–19:29 น.	รวมส่วนต่อประสานไว้ไฟล์เดียว และชุดทดสอบเพิ่ม 6 กรณี
8	แก้ไขความถูกต้องของการนับเวลาและปิดงาน	8 ก.ย. 2569	นาฬิกาและนาฬิกาทรายเดินเท่ากันทุกเครื่อง

[หมายเหตุของร่าง — ลบก่อนส่ง] **รูปที่ 3.2 — ต้องวาดเอง:**
เป็นแผนภาพแท่งแนวนอนแปดแท่งเรียงตามเวลา โดยให้ขั้นที่ 3 กับขั้นที่ 7 เป็นสีเดียวกัน (การปรับโครงสร้าง) และขั้นที่ 4 กับส่วนท้ายของขั้นที่ 7 เป็นอีกสีหนึ่ง (การเขียนชุดทดสอบ) เพื่อให้เห็นว่าทุกครั้งที่ปรับโครงสร้างจะตามด้วยการเขียนชุดทดสอบเสมอ

[วางรูปที่ 3.2 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-3-02.png

รูปที่ 3.2 ลำดับขั้นการดำเนินงานทั้งแปดขั้นเรียงตามเวลา

3.3.1 ขั้นที่ 1 เตรียมโปรเจกต์ เครื่องมือ และระบบควบคุมผู้เล่น

โปรเจกต์ถูกสร้างขึ้นเมื่อวันที่ 25 สิงหาคม 2569 เวลา 06:24 น. ด้วยแม่แบบ Universal 3D ของ Unity รุ่น 6000.3.5f2 จากนั้นภายในวันเดียวกันได้ติดตั้งสะพาน Unity MCP นำเข้าชุดโมเดลตัวละคร สร้างวัสดุชุดแรก 22 ไฟล์ และสร้างไฟล์กำหนดคำสั่ง `PlayerControls.inputactions` ของตนเองขึ้นมาใช้แทนไฟล์ `InputSystem_Actions.inputactions` ที่มากับแม่แบบ

วันถัดมาจึงเริ่มเขียนซอร์สโค้ดของเกม โดยสี่ไฟล์แรกของโครงงานคือ

เวลา	ไฟล์	หน้าที่
26 ส.ค. 10:30 น.	`Scripts/Player/InputReader.cs`	ถือวัตถุ `PlayerControls` เพียงตัวเดียวของทั้งเกม
26 ส.ค. 10:30 น.	`Scripts/Player/MouseLook.cs`	หันกล้องตามการขยับเมาส์
26 ส.ค. 13:10 น.	`Scripts/Player/PlayerMovement.cs`	เดินด้วย `CharacterController`
26 ส.ค. 14:45 น.	`Scripts/Game/RoomLighting.cs`	ความสว่างของห้อง

ในขั้นนี้มีการตัดสินใจทางสถาปัตยกรรมหนึ่งข้อที่ส่งผลไปตลอดโครงงาน คือ **กำหนดให้ `InputReader` เป็นเจ้าของวัตถุ `PlayerControls` เพียงตัวเดียว** และกำหนดกฎการอ่านค่าคำสั่งไว้สองแบบตามชนิดของค่า คือค่าที่ต่อเนื่อง เช่น

การเดินและการหันกล้อง ให้อ่านด้วยการเรียก ReadValue() ทุกเฟรม ส่วนปุ่มที่กดเป็นครั้ง ๆ เช่น E, F และ Q ให้สมัครรับเหตุการณ์ performed กฎข้อนี้ถูกปฏิบัติตามในทุกไฟล์ที่เขียนขึ้นภายหลัง และเป็นสิ่งที่ป้องกันปัญหาปุ่มที่กดครั้งเดียวแล้วถูกอ่านซ้ำหรือถูกอ่านข้าม

ไฟล์เดียวที่เขียนในวันที่ 27 สิงหาคมคือ Scripts/Clock/ClockCondition.cs ซึ่งเป็นชนิดข้อมูลแจงนับที่ระบุสภาพของนาฬิกาห้าแบบ คือ Running, Slow, Reversed, Stopped และ Broken การที่ไฟล์นี้ถูกเขียนตั้งแต่วันที่สามของโครงงานแสดงว่าแนวคิดเรื่องนาฬิกาที่มีสภาพแตกต่างกันถูกกำหนดไว้ตั้งแต่ก่อนที่ระบบนาฬิกาจริงจะถูกเขียน

3.3.2 ขั้นที่ 2 ระบบโต้ตอบกับวัตถุและโครงของระบบนาฬิกา

ขั้นนี้สร้างกลไกพื้นฐานที่ปริศนาทุกองก์จะใช้ร่วมกัน โดยแยกความรับผิดชอบออกเป็นชิ้นเล็ก ๆ ชิ้นละหนึ่งหน้าที่ แทนที่จะเขียนคลาสเดียวที่ทำหลายอย่าง ตัวอย่างที่ชัดที่สุดคือการแยกสิ่งที่เปิดได้ออกจากสิ่งที่ทำให้เคลื่อนที่

- Scripts/Interaction/SwingMotion.cs และ SlideMotion.cs (31 ส.ค. 11:10 น.) รับผิดชอบเฉพาะการเคลื่อนที่ คือแบบบานพับกับแบบเลื่อน โดยอ่านสถานะเปิด–ปิดจากที่อื่นและไม่รู้ว่าทำไมจึงเปิด
- Scripts/Items/Inventory.cs (31 ส.ค. 11:11 น.) เก็บรายการของที่ผู้เล่นถืออยู่
- Scripts/Player/BorrowedProp.cs (31 ส.ค. 11:10 น.) จดจำสภาพเดิมของวัตถุที่ถูกยกขึ้นมาดู เพื่อคืนสภาพให้ครบเมื่อวางกลับ
- Scripts/Interaction/InspectStyle.cs และ Inspectable.cs (1 ก.ย. 08:16 น. และ 10:55 น.) กำหนดรูปแบบการดูวัตถุสามแบบ
- Scripts/Clock/IClockPart.cs (1 ก.ย. 09:18 น.) เป็นส่วนต่อประสานตัวแรกของระบบนาฬิกา ซึ่งนิยามว่าชิ้นส่วนที่ทำให้นาฬิกาเดินหรือไม่เดินคืออะไร

วันที่ 31 สิงหาคมยังเป็นวันที่ภาพพื้นผิวชุดแรกจำนวน 44 ภาพและต้นแบบวัตถุสองชิ้นแรกคือ BrassKeyProp.prefab และ DisplayCase.prefab ถูกสร้างขึ้น ซึ่งหมายความว่าตู้โชว์ของปริศนาองก์ที่ 1 เป็นวัตถุปริศนาชิ้นแรกของเกม

> [หมายเหตุของร่าง — ลบก่อนส่ง] ⚠ **ข้อเท็จจริงที่ต้องบันทึกไว้ และจะถูกนำไปอธิบายในหัวข้อ 5.2:** ไฟล์ DisplayCase.prefab ถูกเขียนครั้งสุดท้ายในวันนี้ คือวันที่ 31 สิงหาคม 2569 เวลา 11:31 น. และไม่เคยถูกเขียนอีกเลยตลอดโครงงาน ขณะที่ไฟล์ Scripts/Puzzle/CodeLock.cs

> ซึ่งเป็นคลาสที่ควบคุมรหัสของตู้นี้ถูกเขียนในวันที่ 7 กันยายน คือหลังจากนั้นเจ็ดวัน ความต่างของวันที่นี้เป็นต้นเหตุของข้อขัดแย้งเรื่องรหัสขององก์ที่ 1 ที่พบในเฟสการสำรวจ ดูรายละเอียดในหัวข้อ 3.3.6

3.3.3 ขั้นที่ 3 ปรับโครงสร้างครั้งที่หนึ่ง แยกส่วนต่อประสานและตัดการพึ่งพา

วันที่ 2 กันยายน 2569 เป็นวันที่มีไฟล์ .cs ถึงสถานะสุดท้ายมากที่สุดของโครงงาน คือ 28 ไฟล์ และเป็นวันที่โครงสร้างของซอฟต์แวร์เปลี่ยนไปอย่างมีนัยสำคัญ โดยเกิดขึ้นเป็นสองจังหวะที่แยกกันชัดเจน

จังหวะแรก เวลา 04:43–04:52 น. แยกส่วนต่อประสานของระบบนาฬิกาออกมา ไฟล์หกไฟล์ถูกเขียนภายในเก้านาที คือ

เวลา	ไฟล์	สิ่งที่นิยาม
04:43:45 น.	Scripts/Clock/IClockDisplay.cs	สิ่งที่แสดงเวลาได้คืออะไร
04:43:55 น.	Scripts/Clock/IPowerStore.cs	ที่เก็บกำลังของนาฬิกาคืออะไร
04:44:17 น.	Scripts/Clock/ClockDial.cs และ Mainspring.cs	ผู้ทำตามส่วนต่อประสานทั้งสอง
04:52:22 น.	Scripts/Clock/DigitalReadout.cs	**ผู้ทำตาม IClockDisplay รายที่สอง**
04:52:50 น.	Scripts/Puzzle/ICodeInput.cs	สิ่งที่ผู้เล่นหมุนเพื่อป้อนรหัสคืออะไร

ลำดับของไฟล์ในจังหวะนี้มีความหมาย คือ IClockDisplay ถูกเขียนก่อน แล้วผู้ทำตามรายที่สองคือ DigitalReadout ตามมาในอีกแปดนาทีถัดมา ซึ่งเป็นการพิสูจน์ด้วยการใช้งานจริงว่าส่วนต่อประสานที่แยกออกมานั้นรองรับการแสดงเวลาที่ต่างรูปแบบกันโดยสิ้นเชิงได้จริง คือหน้าปัดแบบเข็มกับตัวเลขที่เลื่อนแถบ โดยที่ ClockMovement ซึ่งเป็นผู้ขับเคลื่อนไม่ต้องรู้ว่าปลายทางเป็นแบบใด

จังหวะที่สอง เวลา 14:04–14:05 น. ตัดการพึ่งพาที่คลาสของปริศนามีต่อคลาสของผู้เล่น จังหวะนี้กินเวลาเพียงหนึ่งนาที และเป็นการลดการเกาะเกี่ยว (coupling) ที่ชัดเจนที่สุดของโครงงาน

- เวลา **14:04:12 น.** ไฟล์ Scripts/Interaction/IActor.cs ถูกสร้างขึ้น เป็นส่วนต่อประสานที่มีเพียงสามเมธอด คือ Say สำหรับพูดข้อความ OfferBag สำหรับเปิดกระเป๋าให้เลือกของ และ Take สำหรับรับของเข้ากระเป๋า

– เวลา **14:05:12 น.** คือหนึ่งนาทีถัดมา ไฟล์จำนวน 10 ไฟล์ถูกเขียนทับพร้อมกัน โดยเวลาแก้ไขตรงกันถึงระดับวินาที ซึ่งบ่งชี้ว่าเป็นการแก้แบบชุดเดียวครั้งเดียว ดังตารางที่ 3.8

ตารางที่ 3.8 ไฟล์ที่ถูกแก้ไขพร้อมกันในวันที่ 2 กันยายน 2569 เวลา 14:05:12 น.

โมดูล	ไฟล์
Clock	ClockPartSocket.cs, ClockSetter.cs, ClockWinder.cs, RunningLock.cs, TimeLock.cs, WeightChain.cs
Interaction	KeyLock.cs, Openable.cs, PickupItem.cs
Puzzle	CodeWheel.cs

ไฟล์ทั้งสิบไฟล์นี้คือไฟล์ที่เดิมรับพารามิเตอร์เป็นคลาส PlayerInteractor ซึ่งเป็นคลาสของผู้เล่น หมายความว่านาฬิกา ลิ้นชัก และวงล้อรหัสต่างถูกคอมไพล์โดยอ้างถึงคลาสของผู้เล่นของเกมนี้โดยตรง การเปลี่ยนให้รับ IActor แทนทำให้ไฟล์ทั้งสิบไฟล์ไม่ต้องรู้จักคลาสของผู้เล่นอีกต่อไป และฝั่งผู้เล่นเพียงประกาศว่าตนเป็น IActor ด้วยการเขียนเมธอดสามบรรทัดใน PlayerInteractor.cs:105-109

ผลที่วัดได้จากการเปลี่ยนแปลงนี้คือ โมดูล Clock และ Puzzle ไม่มีการอ้างถึงโมดูล Player เหลืออยู่เลยแม้แต่แห่งเดียว ซึ่งเป็นเงื่อนไขที่จำเป็นต่อขั้นที่ 4

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ❓ **ข้อมูลที่ไม่มีในโปรเจกต์:**
> หลักฐานบอกได้ว่าการตัดการพึ่งพาเกิดขึ้นเมื่อใดและเปลี่ยนอะไร
> แต่บอกไม่ได้ว่าผู้จัดทำตัดสินใจทำเพราะพบปัญหาอะไรขณะพัฒนา
> ผู้เขียนควรเพิ่มคำอธิบายส่วนนี้ด้วยตนเองหนึ่งย่อหน้า

3.3.4 ขั้นที่ 4 บังคับขอบเขตโมดูลด้วย Assembly Definition และเขียนชุดทดสอบชุดแรก

เมื่อการพึ่งพาถูกตัดเรียบร้อยแล้ว ขั้นถัดมาคือการทำให้ขอบเขตของโมดูลเป็นสิ่งที่คอมไพเลอร์บังคับได้ ไม่ใช่เพียงข้อตกลงที่ผู้เขียนต้องจำเอง

ในวันเดียวกันเวลา **17:41:15 น.** ไฟล์ Assembly Definition จำนวนห้าไฟล์ถูกสร้างขึ้นภายในช่วงเวลาเพียง 10 มิลลิวินาที ได้แก่ EscapeRoom.Items,

EscapeRoom.Interaction, EscapeRoom.Clock, EscapeRoom.Puzzle และ EscapeRoom.World จากนั้นอีก 52 วินาทีต่อมาคือเวลา 17:42:07 น. จึงสร้าง EscapeRoom.Tests.asmdef สำหรับชุดทดสอบ การที่ทั้งห้าไฟล์ถูกสร้างพร้อมกันแสดงว่าขอบเขตของโมดูลทั้งหมดถูกกำหนดในคราวเดียว ไม่ได้ทยอยใส่ทีละโมดูล

สิ่งที่ตามมาในช่วงเย็นวันเดียวกันคือ

เวลา	ไฟล์	สิ่งที่เกิดขึ้น
18:26:29 น.	Scripts/Clock/Pendulum.cs และ Scripts/Editor/ClockBuilder.cs	ปรับลูกตุ้มให้ค้นหานาฬิกาของตนเองด้วย GetComponentInParent แทนการลากอ้างอิงในตัวตรวจสอบ
18:26:51 น.	Scripts/Clock/ClockRecipe.cs	นิยามสูตรของนาฬิกาหนึ่งชนิดให้เป็นไฟล์ข้อมูล
18:28:24 น.	Scripts/Editor/ClockAssembler.cs	เครื่องมือแปลงสูตรเป็นต้นแบบวัตถุนาฬิกา
18:29 น.	PlankClock.prefab และต้นแบบชิ้นส่วนอีกสามไฟล์	นาฬิกาเรือนแรกที่เกิดจากไฟล์ข้อมูลล้วน โดยไม่มีเมธอดในภาษา C# รองรับ
18:31:26 น.	Tests/PlayMode/ClockSeamTests.cs	ชุดทดสอบชุดแรก 6 กรณี
18:32:21 น.	Tests/PlayMode/RecipeSeamTests.cs	ชุดทดสอบ 3 กรณี
18:43:37 น.	MODULE-PLAN.md	เอกสารอธิบายการแบ่งโมดูล ขนาด 41,116 ไบต์

ข้อสังเกตที่สำคัญสองข้อจากลำดับนี้

ข้อแรก ชุดทดสอบถูกเขียนขึ้นทันทีหลังการปรับโครงสร้าง ไม่ใช่ก่อน ไฟล์ทดสอบทั้งสองไฟล์ถูกสร้างขึ้นห่างจากไฟล์ที่มันทดสอบเพียงห้านาทีและสี่นาทีตามลำดับ และห่างจากไฟล์ Assembly Definition 50 นาที รูปแบบนี้ไม่ใช่การพัฒนาที่ขับเคลื่อนด้วยการทดสอบ (Test-Driven Development) แต่เป็นการทดสอบความเหมาะสมของสถาปัตยกรรมตามทฤษฎีในหัวข้อ 2.4.2 คือเขียนขึ้นเพื่อยืนยันว่ารอยต่อที่เพิ่งแยกออกมาใช้งานได้จริง และเพื่อป้องกันไม่ให้ถูกทำพังในภายหลัง

ข้อที่สอง เอกสาร MODULE-PLAN.md ถูกเขียนหลังการแบ่งโมดูลเสร็จแล้ว ไม่ใช่ก่อนเริ่มงาน ชื่อไฟล์อาจทำให้เข้าใจว่าเป็นแผนที่วางไว้ล่วงหน้า แต่เวลาของไฟล์คือ 18:43 น. ซึ่งอยู่หลัง Assembly Definition หนึ่งชั่วโมงกับสองนาที และอยู่หลังชุดทดสอบ 11 นาที

ไม่พบร่องรอยว่าการใส่ Assembly Definition ทำให้เกิดข้อผิดพลาดในการคอมไพล์ที่ต้องตามไล่แก้ เพราะหลังเวลา 17:42 น. ไม่มีไฟล์ซอร์สโค้ดใดถูกเขียนอีกเลยจนถึงเวลา 18:26 น. และไฟล์ที่ถูกเขียนในเวลานั้นเป็นงานเรื่องสูตรนาฬิกา ไม่ใช่การแก้การอ้างอิงระหว่างโมดูล

> **การตีความ (ไม่ใช่ข้อเท็จจริงจากไฟล์):** ลำดับที่พบคือตัดการพึ่งพาให้เสร็จก่อนในเวลา 14:05 น. แล้วจึงใส่ Assembly Definition ในเวลา 17:41 น. ซึ่งห่างกันสามชั่วโมงครึ่ง อ่านได้ว่า Assembly Definition ถูกใช้เป็นกลไกล็อกโครงสร้างไว้หลังจากที่จัดการพึ่งพาเรียบร้อยแล้ว ไม่ได้ใช้เป็นเครื่องมือค้นหาว่ามีการพึ่งพาใดเหลืออยู่ ซึ่งเป็นคำอธิบายว่าเหตุใดจึงไม่มีข้อผิดพลาดตามมา

3.3.5 ขั้นที่ 5 สร้างโลกของเกมด้วยสคริปต์สำหรับตัวแก้ไข

ขั้นนี้คือการสร้างฉากและทรัพยากรทั้งหมดของเกม โดยใช้วิธีเขียนโปรแกรมสร้างแทนการปั้นด้วยโปรแกรมสามมิติ เครื่องมือหลักสามตัวที่ถึงสถานะสุดท้ายในขั้นนี้คือ

เวลา	ไฟล์	หน้าที่
3 ก.ย. 16:59 น.	Scripts/Editor/TextureKit.cs	วาดภาพพื้นผิวทุกชนิดลงไฟล์ใน Assets/Textures ได้แก่ ปูน ไม้ปูพื้น พรม ลายไม้ อิฐ หิน ผ้าทอ ท้องฟ้ากลางคืน และเป้าเล็ง
4 ก.ย. 07:27 น.	Scripts/World/HousePlan.cs	นิยามผังบ้านให้เป็น ScriptableObject คือรายการห้องห้าห้องกับรายการแนวผนังและตำแหน่งช่องประตู
7 ก.ย. 11:23 น.	Scripts/Editor/HouseBuilder.cs	อ่านผังบ้านจากไฟล์ข้อมูลแล้วสร้างพื้น ผนัง ฝ้า ช่องหน้าต่าง และคิ้วบัวทั้งหมด

หลักการที่ใช้ในการแยกขอบเขตของเครื่องมือเหล่านี้คือ **เครื่องมือแต่ละตัวต้องแตะเฉพาะส่วนที่เป็นของตนเองเท่านั้น** HouseBuilder สร้างใหม่เฉพาะกลุ่มวัตถุ House/Shell และไม่แตะอะไรที่อยู่ใต้ House/Props ส่วน DressingBuilder สร้างใหม่เฉพาะกลุ่ม Dressing ของแต่ละห้อง ข้อตกลงนี้ทำให้สามารถสั่งสร้างฉากใหม่ได้ทุกเมื่อโดยไม่ทำลายงานที่วางด้วยมือ

ปัญหาที่ตามมาจากวิธีนี้คือ **วัตถุที่คนเป็นผู้แก้ไขเนื้อหาจะถูกลบทิ้งทุกครั้งที่สั่งสร้างใหม่** ซึ่งโครงงานแก้ไว้สองวิธี วิธีแรกคือย้ายวัตถุนั้นออกนอกขอบเขตของเครื่องมือ เช่น

นาฬิกาทรายในห้องช่างและหนังสือพิมพ์ในห้องหน้าร้าน ถูกวางเป็นลูกโดยตรงของห้อง ไม่ได้อยู่ใต้กลุ่ม `Dressing` วิธีที่สองคืออ่านข้อความที่คนเขียนไว้ออกมาเก็บก่อน แล้วใส่กลับหลังสร้างใหม่ ซึ่งเป็นวิธีที่ `NewspaperBuilder` ใช้

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📷 **รูปที่ 3.3 — ต้องแคปจาก Unity:** เปิดเมนู `Tools > Escape Room` ให้กางออกทุกระดับ แล้วแคปทั้งเมนู เพื่อให้เห็นว่าเครื่องมือสร้างเนื้อหาถูกจัดเป็นหมวด House, Clocks และ Puzzle

[วางรูปที่ 3.3 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-3-03.png

รูปที่ 3.3 เมนูเครื่องมือสร้างเนื้อหาที่พัฒนาขึ้นในโครงงาน

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** 📷 **รูปที่ 3.4 — ต้องแคปจาก Unity:** มุมมอง Scene View จากด้านบน (Top) ให้เห็นผังบ้านทั้งห้าห้องพร้อมกัน **ห้ามใช้กล้องของผู้เล่นถ่าย** เพราะในโหมดแก้ไขตำแหน่งกล้องจะถูกบันทึกลงฉาก ทำให้เกมเริ่มด้วยมุมกล้องผิด

[วางรูปที่ 3.4 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-3-04.png

รูปที่ 3.4 ผังบ้านทั้งห้าห้องที่สร้างจากไฟล์ `HousePlan.asset`

3.3.6 ขั้นที่ 6 ออกแบบและพัฒนาปริศนาทั้งห้าองก์

ปริศนาทั้งห้าองก์ถูกพัฒนาในสองวัน คือวันที่ 4 กันยายนและวันที่ 7 กันยายน 2569 โดยแต่ละองก์มีคลาสเฉพาะของตนเองเพียงไม่กี่คลาส
เพราะกลไกพื้นฐานทั้งหมดถูกสร้างไว้แล้วในขั้นที่ 2 ถึง 4

องก์	ไฟล์เฉพาะขององก์	เวลาที่ถึงสถานะสุดท้าย
5	StarLock.cs, Horologium.cs, Prism.cs และต้นแบบวัตถุ Sundial, StarScreen, StarChart, MoonDome และ Prism*Prop	4 ก.ย. 06:11–08:05 น. และ 21:13 น.
2	Sandglass.cs, HeldSandglass.cs และ RunTicking.cs	3 ก.ย. 23:28 น. ถึง 7 ก.ย. 15:26 น.
4	StampedNumber.cs, StampedCombination.cs, CodeLock.cs และ MissingThingsBuilder.cs	7 ก.ย. 10:46–10:58 น.
3	CuckooBird.cs, ClockStriker.cs และต้นแบบ CuckooClock.prefab	7 ก.ย. 16:46–17:23 น.
1	CompassNeedle.cs และส่วนที่อยู่ใน PuzzleBuilder.cs	7 ก.ย. 18:17 น. และ 8 ก.ย. 07:18 น.

> **การตีความ (ต้องระบุข้อจำกัดกำกับ):** จากตารางนี้
> **องก์ที่ปรากฏเป็นองก์สุดท้ายของเกมกลับมีไฟล์ที่ถึงสถานะสุดท้ายก่อนองก์อื่น และองก์แรกของเกมมีไฟล์ที่ถึงสถานะสุดท้ายท้ายที่สุด** อย่างไรก็ตาม
> เวลาแก้ไขครั้งสุดท้ายไม่ใช่เวลาที่เริ่มเขียน และไฟล์ PuzzleBuilder.cs
> เป็นเครื่องมือที่สร้างวัตถุให้หลายองก์รวมกัน จึงเหลือให้เห็นเพียงการแก้ครั้งสุดท้าย
> ข้อสรุปที่หลักฐานรองรับได้จริงคือ **งานขององก์ที่ 5 เสร็จก่อน และงานขององก์ที่ 1 เป็นงานที่ถูกแตะเป็นครั้งสุดท้าย** ไม่ใช่ว่าเขียนเรียงตามลำดับนี้

สิ่งที่ยืนยันได้จากซอร์สโค้ดคือ
ปริศนาทั้งห้าองก์ใช้กลไกชุดเดียวกันโดยไม่ต้องแก้โค้ดของกลไกนั้น ตัวอย่างที่ชัดที่สุดสามกรณีคือ

1. **ClockPartSocket ถูกใช้ 5 จุด** คือสองจุดในองก์ที่ 4 สำหรับใส่ลูกตุ้มและตุ้มน้ำหนักของนาฬิกาตู้ยืน และสามจุดในองก์ที่ 5

สำหรับใส่ปริซึมทั้งสามชิ้น โดยคลาสนี้ไม่ได้ถูกแก้ไขเพื่อรองรับปริซึมเลย เพราะรับค่าเป็น ItemData ซึ่งเป็นไฟล์ข้อมูล ไม่ใช่ชนิดของวัตถุ

2. **CodeLock และ CodeWheel ถูกใช้ใน 3 องก์** คือองก์ที่ 1 ที่ตู้โชว์ องก์ที่ 3 ที่ฐานของกรงนก และองก์ที่ 4 ที่ประตูตู้เสื้อผ้า

3. **กลอนทุกชนิดเป็นผู้ทำตามส่วนต่อประสาน ILock เดียวกัน** ทำให้ Openable ซึ่งเป็นคลาสของสิ่งที่เปิดได้ ถามกลอนทุกตัวที่ติดอยู่กับตนว่าล็อกอยู่หรือไม่ โดยไม่ต้องรู้ว่ากลอนนั้นต้องการกุญแจ รหัส เวลา หรือรูปแบบของดวงดาว

ในองก์ที่ 4 มีการออกแบบที่ทำให้คำตอบของปริศนากลายเป็นข้อมูลแทนที่จะเป็นตัวเลขในโค้ด คือคลาส StampedNumber เก็บเลขประจำวัตถุหนึ่งชิ้นและพิมพ์เลขนั้นลงบนป้ายหลังวัตถุ ส่วน StampedCombination ถือรายการของวัตถุเหล่านั้นตามลำดับที่ปรากฏในภาพถ่าย แล้วส่งรหัสที่ได้ให้กลอนรหัส ผลคือการเปลี่ยนคำตอบของปริศนาองก์นี้ทำได้จากตัวตรวจสอบของ Unity โดยไม่ต้องแก้ซอร์สโค้ดเลย และการเพิ่มความยาวของรหัสจากสี่หลักเป็นห้าหลักทำได้ด้วยการเพิ่มสมาชิกในรายการและเพิ่มวงล้ออีกหนึ่งวง

รูปแบบเดียวกันนี้พบในองก์ที่ 1 ด้วย แต่ใช้กลไกของ Unity คนละชั้น คือรหัส 1479 ของตู้โชว์ไม่ได้เก็บไว้ในซอร์สโค้ดและไม่ได้เก็บไว้ในไฟล์ต้นแบบ แต่เก็บเป็น **ค่าที่ฉากเขียนทับไฟล์ต้นแบบ** (prefab instance override) กล่าวคือไฟล์ Assets/Prefabs/Puzzle/DisplayCase.prefab บันทึกค่าไว้เป็น 4, 7, 1, 9 แล้วฉาก Main.unity เขียนทับสามตำแหน่งแรกเป็น 1, 4, 7 ส่วนตำแหน่งที่สี่คงค่า 9 จากไฟล์ต้นแบบไว้ ผลลัพธ์จึงเป็น 1479 ตรงกับที่ออกแบบและตรงกับค่าเริ่มต้นที่เขียนไว้ใน Scripts/Puzzle/CodeLock.cs:21 รายละเอียดและวิธีตรวจอยู่ในบทที่ 4 หัวข้อ 4.1.2

3.3.7 ขั้นที่ 7 ปรับโครงสร้างครั้งที่สองและเพิ่มชุดทดสอบ

ช่วงบ่ายและเย็นของวันที่ 7 กันยายน 2569 เกิดการปรับโครงสร้างรอบที่สอง โดยมีรูปแบบเดียวกับขั้นที่ 3 และ 4 คือปรับโครงสร้างแล้วตามด้วยการเขียนชุดทดสอบทันที

เวลา	ไฟล์	สิ่งที่เกิดขึ้น
16:04:39 น.	Scripts/Interaction/Interactions.cs	รวมส่วนต่อประสานของการโต้ตอบไว้ในไฟล์เดียว
16:04:57 น.	Scripts/Player/InspectView.cs	ปรับระบบการยกวัตถุขึ้นดูให้ทำงานผ่านส่วนต่อประสานเหล่านั้น

เวลา	ไฟล์	สิ่งที่เกิดขึ้น
16:53 น.	Assets/InitTestScene855cb578-5e59-4f44-aa4a-008404e0b48a.unity	ไฟล์ฉากชั่วคราวที่หลงเหลืออยู่
17:15:15 น.	Tests/PlayMode/ActorSeamTests.cs	ชุดทดสอบชุดที่สาม 6 กรณี
18:16–18:17 น.	ItemData.cs, InventoryUI.cs และ CompassNeedle.cs	ปรับระบบกระเป๋าให้ซ่อนของที่ใช้กับสิ่งอื่นไม่ได้
18:23:16 น.	MODULARITY.md	เอกสารอธิบายความเป็นโมดูล ขนาด 38,063 ไบต์
19:29:02 น.	Scripts/Game/GameManager.cs	เงื่อนไขการจบเกม

ไฟล์ InitTestScene เป็นหลักฐานว่าชุดทดสอบเคยถูกรันจริงอย่างน้อยหนึ่งครั้ง ไฟล์นี้มีรูปแบบชื่อว่า InitTestScene ตามด้วยรหัส GUID ซึ่งเป็นรูปแบบที่ Unity Test Framework สร้างขึ้นเมื่อเริ่มรันชุดทดสอบแบบ PlayMode และตามปกติจะถูกลบทิ้งเมื่อรันเสร็จ การที่ไฟล์นี้ยังค้างอยู่ในโปรเจกต์บ่งชี้ว่าการรันครั้งนั้นจบลงโดยไฟล์ไม่ถูกเก็บกวาด ไฟล์นี้มีขนาด 5,704 ไบต์ เป็นฉากเปล่า และ **ไม่ได้อยู่ใน Build Settings** จึงไม่ใช่ส่วนหนึ่งของเกมที่ผู้เล่นได้เล่น

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** **ระดับความเชื่อมั่น:** การมีอยู่ของไฟล์และเวลาของไฟล์ = ยืนยันแล้ว · การสรุปว่าไฟล์นี้เกิดจากการรันชุดทดสอบ = เป็นการตีความจากรูปแบบการตั้งชื่อของ Unity Test Framework ไม่ใช่สิ่งที่ไฟล์ระบุด้วยตนเอง
>
> ⚠ **ไฟล์นี้ยังอยู่ในโปรเจกต์จนถึงปัจจุบัน** ผู้เขียนควรตัดสินใจว่าจะลบทิ้งก่อนส่งงาน เพราะเป็นไฟล์ค้างจากเครื่องมือและไม่ใช่ส่วนหนึ่งของเกม หรือจะเก็บไว้เป็นหลักฐานประกอบรายงานว่าเคยรันชุดทดสอบ ซึ่งถ้าเก็บไว้ก็ต้องอธิบายสถานะของไฟล์นี้ไว้ด้วย

เช่นเดียวกับเอกสาร MODULE-PLAN.md ในขั้นที่ 4 เอกสาร MODULARITY.md ก็ถูกเขียนหลังจากงานที่มันอธิบายเสร็จแล้ว คือเวลา 18:23 น. ซึ่งอยู่หลังการปรับโครงสร้างสองชั่วโมงกับ 19 นาที

3.3.8 ขั้นที่ 8 แก้ไขความถูกต้องของการนับเวลาและปิดงาน

วันสุดท้ายของการพัฒนาคือวันที่ 8 กันยายน 2569 และงานชิ้นแรกของวันคือการแก้ปัญหาที่ผู้จัดทำระบุว่าเป็นปัญหาใหญ่ที่สุดของโครงงาน คือ **การจับเวลาของเกมผูกอยู่กับความเร็วของเครื่องที่เล่น** กล่าวคือ เครื่องที่ทำงานเร็วจะทำให้นาฬิกาในเกมเดินเร็ว และเครื่องที่ทำงานช้าจะทำให้นาฬิกาเดินช้า ทั้งที่นาฬิกาทรายในองก์ที่ 2 ต้องจับเวลาได้สิบวินาทีเท่ากันทุกเครื่อง เพราะคำตอบของปริศนาองก์นั้นคือการเปรียบเทียบว่านาฬิกาเรือนใดเดินไม่ทันทราย

สองไฟล์ที่แก้ปัญหานี้คือ `Scripts/Clock/ClockMovement.cs` เวลา 06:19:48 น. และ `Scripts/Clock/TestRun.cs` เวลา 06:20:06 น. ซึ่งเป็นสองในสามไฟล์สุดท้ายของทั้งโครงงาน วิธีแก้ประกอบด้วยสี่กลไกที่แยกจากกัน ดังตารางที่ 3.9

ตารางที่ 3.9 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา

ลำดับ	กลไก	ตำแหน่งในซอร์สโค้ด	ปัญหาที่แก้
1	นับเวลาด้วย `Time.deltaTime` ซึ่งเป็นเวลาจริงของเฟรมนั้น ไม่ใช่จำนวนเฟรม	`ClockMovement.cs:136`, `Sandglass.cs:107` และ `TestRun.cs:77`	เครื่องที่มีอัตราเฟรมต่างกันนับเวลาได้เท่ากัน
2	สะสมเวลาในชนิดข้อมูล `double` แทน `float`	`ClockMovement.cs:50`	ความคลาดเคลื่อนสะสมจากการบวกเลขทศนิยมที่มีขนาดต่างกันมาก
3	คิดเฟรมสุดท้ายของการทดสอบเป็นเศษส่วนของเฟรม	`TestRun.cs:52`	เข็มหยุดตรงตำแหน่งเดิมเสมอ ไม่เลยไปตามความยาวของเฟรม
4	ให้ตัวจับเวลานับใน `LateUpdate` ส่วนนาฬิกาเดินใน `Update`	`TestRun.cs:72-81`	ลำดับการทำงานของ `Update` ระหว่างคอมโพเนนต์ที่ไม่ถูกกำหนด

กลไกทั้งสี่ข้อตั้งอยู่บนทฤษฎีที่อธิบายไว้ในหัวข้อ 2.3 คือความเป็นอิสระจากอัตราเฟรม ความแม่นยำของเลขทศนิยม และลำดับการทำงานของฟังก์ชันเหตุการณ์ และการตรวจสอบซอร์สโค้ดทั้งโปรเจกต์ไม่พบการใช้ `Time.frameCount` แม้แต่แห่งเดียว

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อจำกัดของหลักฐานที่ต้องระบุ:**
> เนื่องจากไม่มีระบบควบคุมเวอร์ชัน จึงเห็นได้เฉพาะโค้ดชุดที่แก้แล้ว
> **ไม่สามารถยืนยันรูปแบบของโค้ดชุดเดิมที่มีปัญหาได้**

> ในรายงานจึงต้องเขียนอาการของปัญหาเดิมในฐานะคำบอกเล่าของผู้จัดทำ
> และใช้โค้ดปัจจุบันเป็นหลักฐานของ *วิธีแก้* เท่านั้น ห้ามเขียนว่าโค้ดเดิมเขียนอย่างไรโดยไม่มีหลักฐาน

งานที่เหลือของวันสุดท้ายเรียงตามเวลาคือ สั่งเครื่องมือ `PuzzleBuilder` สร้างเรขาคณิตของกรงนกใหม่เมื่อเวลา 07:18 น. ปรับปรุงเอกสาร `CLAUDE.md` เมื่อเวลา 07:49 น. บันทึกฉาก `Main.unity` เป็นครั้งสุดท้ายเมื่อเวลา 09:41 น. ปรับการตั้งค่าไปป์ไลน์เรนเดอร์เมื่อเวลา 09:46 น. และบันทึกไฟล์ตั้งค่าโปรเจกต์เป็นครั้งสุดท้ายเมื่อเวลา 23:19 น. ซึ่งเป็นเวลาที่ไฟล์ในโปรเจกต์ถูกเขียนเป็นครั้งสุดท้าย

3.3.9 สรุปรูปแบบการดำเนินงานที่พบ

เมื่อพิจารณาทั้งแปดขั้นรวมกัน พบรูปแบบการทำงานที่เกิดซ้ำสามรูปแบบ ซึ่งเป็นสิ่งที่หลักฐานด้านเวลารองรับได้

รูปแบบที่หนึ่ง สร้างกลไกกลางก่อน แล้วจึงสร้างปริศนา กลไกพื้นฐานทั้งหมด ได้แก่ การเปิด–ปิด การล็อก การเก็บของ การยกขึ้นดู และระบบนาฬิกา ถูกสร้างเสร็จในขั้นที่ 2 ถึง 4 คือภายในวันที่ 2 กันยายน ส่วนปริศนาทั้งห้าองก์ถูกสร้างในขั้นที่ 6 คือวันที่ 4 และ 7 กันยายน ผลคือปริศนาแต่ละองก์มีโค้ดเฉพาะของตนเองเพียงไม่กี่ไฟล์ เพราะส่วนที่เหลือใช้ของเดิมได้

รูปแบบที่สอง ปรับโครงสร้างแล้วเขียนชุดทดสอบทันที เกิดขึ้นสองครั้งและมีระยะห่างใกล้เคียงกันทั้งสองครั้ง คือครั้งแรกในวันที่ 2 กันยายน ปรับโครงสร้างเวลา 14:05 น. และ 17:41 น. แล้วเขียนชุดทดสอบเวลา 18:31 น. ครั้งที่สองในวันที่ 7 กันยายน ปรับโครงสร้างเวลา 16:04 น. แล้วเขียนชุดทดสอบเวลา 17:15 น. ชุดทดสอบทั้งหมด 15 กรณีของโครงงานนี้จึงเกิดจากการปรับโครงสร้างสองครั้งนี้เท่านั้น

รูปแบบที่สาม เอกสารถูกเขียนหลังงานเสร็จ เอกสารทั้งสามไฟล์ของโครงงาน คือ `MODULE-PLAN.md` `MODULARITY.md` และ `CLAUDE.md` ล้วนถูกเขียนครั้งสุดท้ายหลังจากงานที่มันอธิบายเสร็จสิ้นแล้ว เอกสารเหล่านี้จึงมีลักษณะเป็นบันทึกการตัดสินใจทางสถาปัตยกรรม (Architecture Decision Record) มากกว่าเป็นข้อกำหนดที่วางไว้ล่วงหน้า

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** ⚠ **ข้อควรระวังในการอ้างเอกสารภายในโครงงาน:**
> การสำรวจในเฟสที่ 1 พบว่าเอกสาร `CLAUDE.md` ไม่ตรงกับซอร์สโค้ดและฉากจริงสี่จุด ได้แก่ จำนวนนาฬิกาในฉาก ตารางการพึ่งพาระหว่างโมดูลสองแถว และจำนวนผู้ทำตามส่วนต่อประสาน ดังนั้นทุกตัวเลขในรายงานต้องอ้างจากซอร์สโค้ดและฉากจริง ไม่ใช่จากเอกสารในโปรเจกต์

บทที่ 4
ผลการดำเนินงาน

บทนี้รายงานผลที่ได้จากการดำเนินงานตามขั้นตอนทั้งแปดขั้นในบทที่ 3 โดยแยกเป็นสี่ส่วน คือผลลัพธ์ที่เป็นตัวเกม ผลการวิเคราะห์โครงสร้างของซอฟต์แวร์ที่เขียนขึ้น ผลการทดสอบ และสรุปผลเทียบกับขอบเขตที่กำหนดไว้

ตัวเลขทุกตัวในบทนี้มาจากการวัดไฟล์ในโปรเจกต์โดยตรง ไม่ได้มาจากเอกสารประกอบที่เขียนไว้ภายในโปรเจกต์ เหตุผลคือระหว่างการตรวจสอบพบว่าเอกสารภายในโปรเจกต์ระบุตัวเลขไม่ตรงกับไฟล์จริงอยู่สี่จุด (รายละเอียดในหัวข้อ 4.4.2) จึงถือไฟล์เป็นหลักฐาน และระบุวิธีวัดกำกับไว้ทุกตาราง เพื่อให้ตรวจซ้ำได้

4.1 ผลการดำเนินงาน

4.1.1 ผลลัพธ์ที่ส่งมอบ

ผลลัพธ์ของโครงงานคือเกม escape room มุมมองบุคคลที่หนึ่งที่เล่นจบได้หนึ่งรอบ ประกอบด้วยห้องห้าห้องเรียงต่อกันเป็นเส้นทางเดียว และปริศนาห้าองก์ที่ต้องแก้เรียงตามลำดับ ปริมาณของผลลัพธ์ที่วัดได้แสดงในตารางที่ 4.1

ตารางที่ 4.1 ปริมาณผลลัพธ์ของโครงงานที่วัดได้จากไฟล์ในโปรเจกต์

รายการ	จำนวน	วิธีวัด
ห้องที่เล่นได้	5	นับวัตถุลูกของ House/Props ในฉาก Main.unity
ปริศนาที่ต้องแก้	5 องก์	ตารางที่ 4.2
ประตูที่มีกลไก	5 บาน	นับวัตถุลูกของ House/Doors
ไฟล์โปรแกรมภาษา C#	78 ไฟล์	นับไฟล์ .cs ทั้งโปรเจกต์
จำนวนบรรทัดของโปรแกรม	10,604 บรรทัด	นับบรรทัดที่ไม่ว่าง · ตารางที่ 4.4
คลาสที่สืบทอดจาก MonoBehaviour	51 คลาส	ค้นข้อความ : MonoBehaviour
ส่วนต่อประสาน (interface) ที่ประกาศขึ้น	12 รายการ	ค้นข้อความ public interface

รายการ	จำนวน	วิธีวัด
Prefab	33 ไฟล์	นับไฟล์ใน Assets/Prefabs/
Material	115 ไฟล์	นับไฟล์ .mat
ScriptableObject	12 ไฟล์	ItemData 10, HousePlan 1, ClockRecipe 1
นาฬิกาในฉาก	46 เรือน	find_gameobjects ด้วย component ClockMovement ยืนยันซ้ำเมื่อ 6 พ.ย. 2569
ชุดทดสอบอัตโนมัติ	15 กรณี	นับ attribute [UnityTest]

รูปที่ 4.1 ถึงรูปที่ 4.5 แสดงห้องทั้งห้าห้องตามลำดับที่ผู้เล่นเดินผ่าน



รูปที่ 4.1 ห้องหน้าร้าน องก์ที่หนึ่ง



รูปที่ 4.2 ห้องช่าง องก์ที่สอง

[วางรูปที่ 4.3 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-03.png

รูปที่ 4.3 ห้องนั่งเล่น องก์ที่สาม

[วางรูปที่ 4.4 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-04.png

รูปที่ 4.4 ห้องนอน องก์ที่สี่

[วางรูปที่ 4.5 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-05.png

รูปที่ 4.5 ห้องหลังตู้นาฬิกา องก์ที่ห้า

> **ข้อควรระวังที่ตรวจพบระหว่างร่างบทนี้ — ภาพเก่าทั้งหมดใช้ไม่ได้** โปรเจกต์มีภาพถ่ายหน้าจอเก็บไว้ 58 ไฟล์ในโฟลเดอร์ Shots/ แต่เมื่อตรวจเวลาแก้ไขไฟล์พบว่า **ทุกไฟล์มีวันที่อยู่ระหว่าง 31 สิงหาคม 11:28 น. ถึง 2 กันยายน 2569 เวลา 00:31 น.** และไม่มีไฟล์ใดใหม่กว่านั้นเลย ซึ่งเป็นช่วงก่อนงานในวันที่ 2, 3, 4, 7 และ 8 กันยายน — นั่นคือก่อนการสร้างโลกของเกมด้วยสคริปต์ทั้งหมด (ขั้นที่ 5) ก่อนปริศนาองก์ที่สี่และห้า

> (ขั้นที่ 6) และก่อนการขยายห้องที่ห้า **สรุปคือไม่มีภาพใดในโปรเจกต์ที่แสดงเกมในสถานะสุดท้าย** ภาพประกอบทุกภาพในบทนี้ต้องถ่ายใหม่จากการกด Play ในสถานะปัจจุบัน *วิธีวัด:* stat เวลาแก้ไขของไฟล์ .png ทั้ง 58 ไฟล์ · ไฟล์ที่ใหม่สุดคือ clean_view.png (2 ก.ย. 00:31)

4.1.2 ผลการพัฒนาปริศนาทั้งห้าองก์

ปริศนาทั้งห้าองก์ทำงานได้ และคำตอบของแต่ละองก์ถูกยืนยันโดยอ่านค่าที่ตั้งไว้จากไฟล์ asset โดยตรง ไม่ใช่จากเอกสารหรือจากการเล่น ผลแสดงในตารางที่ 4.2

ตารางที่ 4.2 คำตอบของปริศนาแต่ละองก์และหลักฐานที่ยืนยัน

องก์	ห้อง	กลไกที่ผู้เล่นต้องทำ	คำตอบ	แหล่งที่อ่านค่าได้
1	หน้าร้าน	อ่านเลขจากนาฬิกาที่ลูกตุ้มยังแกว่ง เรียงจากซ้ายไปขวา	**1479**	Scenes/Main.unity ⟶ ค่าที่ override ไว้บน CodeLock ของ DisplayCase (ดูหมายเหตุ)
2	ห้องช่าง	จับเวลานาฬิกาพกเทียบนาฬิกาทราย หาเรือนที่เดินช้า แล้วตั้งเวลาตามหน้าปัดของเรือนนั้น	**4:40**	Scenes/Main.unity ⟶ TimeLock บน BenchDrawer (hour: 4, minute: 40)
3	ห้องนั่งเล่น	ดึงโซ่ใต้นาฬิกาคุกคูสามเรือนแล้วนับเสียงนก	**502**	Scenes/Main.unity ⟶ CodeLock ของกรงนก
4	ห้องนอน	หาของสี่ชิ้นในภาพถ่าย พลิกอ่านเลขหลังของ แล้วใส่ตามลำดับในภาพ	**4692**	Scenes/Main.unity ⟶ CodeLock ของตู้เสื้อผ้า
5	หลังตู้นาฬิกา	หมุนปริซึมสามชิ้นจนจุดแสงหกจุดตรงกับเครื่องหมายบนจอ	**2 · 3 · 4**	Prefabs/Puzzle/Sundial.prefab ⟶ Horologium.answer

วิธีอ่านค่า: ค่าทั้งห้าองก์อ่านจาก Unity Editor ที่เปิดโปรเจกต์อยู่ ผ่าน Unity MCP เมื่อวันที่ 6 พฤศจิกายน 2569 ซึ่งคืนค่าที่ประกอบเสร็จแล้วหลังรวม prefab กับค่าที่ override ไว้ในฉาก และสอบทานกับค่าในไฟล์ YAML ของฉากและของ prefab ทุกค่า

> **หมายเหตุองก์ที่หนึ่ง — ค่าที่โปรแกรมใช้มาจากฉาก ไม่ใช่จาก prefab** ค่าที่บันทึกอยู่ในไฟล์ Prefabs/Puzzle/DisplayCase.prefab คือ 4, 7, 1, 9 แต่ **นั่นไม่ใช่ค่าที่เกมใช้** เพราะฉาก Main.unity override ค่านั้นไว้สามตำแหน่ง

ตำแหน่ง	ค่าใน prefab	ค่าที่ฉาก override	ค่าที่ใช้จริง
combination[0]	4	1	**1**
combination[1]	7	4	**4**
combination[2]	1	7	**7**
combination[3]	9	*ไม่ override*	**9**

> ผลลัพธ์คือ **1479** ตรงกับค่าตั้งต้นที่เขียนไว้ในซอร์สโค้ด (Assets/Scripts/Puzzle/CodeLock.cs:21 ⟶ private int[] combination = { 1, 4, 7, 9 };)
> ตรงกับผลคำนวณลำดับการกวาดสายตาจากตำแหน่งจริงของนาฬิกาในฉาก และตรงกับค่าที่ Unity Editor รายงาน คือ combination: [1,4,7,9]
> ตำแหน่งที่สี่ไม่มี override เพราะค่าใน prefab เป็น 9 อยู่แล้ว Unity จะบันทึก override เฉพาะสมาชิกที่ค่าต่างจาก prefab เท่านั้น
> **ข้อควรระวังในการตรวจค่าแบบนี้** path ของ override สมาชิกอาเรย์ใน YAML เขียนในเครื่องหมายคำพูดเดี่ยว คือ propertyPath: 'combination.Array.data[0]' การค้นหาด้วยคำว่า propertyPath: combination จะไม่พบอะไรเลยและทำให้สรุปผิดว่าไม่มี override
> **การอ่านค่าที่ประกอบเสร็จจาก Editor จึงเชื่อถือได้มากกว่าการอ่านไฟล์ YAML เอง** และควรใช้เป็นแหล่งหลักเมื่อ Editor เปิดอยู่

คำตอบขององค์ที่สี่ยืนยันได้จากสองทางที่เป็นอิสระจากกัน และทั้งสองทางให้ผลตรงกัน ซึ่งเป็นหลักฐานที่หนักกว่าการอ่านค่าเดียว

ทางที่อ่าน	ค่าที่ได้
ค่าที่บันทึกไว้บน CodeLock ของตู้เสื้อผ้า	4, 6, 9, 2
StampedNumber.digit ของของทั้งสี่ชิ้น เรียงตามลำดับในรายการ StampedCombination.asPictured คือ Teacup 4 · Candlestick 6 · Vase 9 · HandMirror 2	4, 6, 9, 2

ทางที่สองมีความหมายเพิ่มขึ้นอีกชั้น เพราะมันคือ **เลขที่พิมพ์อยู่บนป้ายหลังของที่ผู้เล่นพลิกอ่านจริง** เรียงตามลำดับที่ของวางอยู่ในภาพถ่าย การที่ค่าทั้งสองทางตรงกันหมายความว่าคำใบ้ที่ผู้เล่นเห็นกับรหัสที่กลอนต้องการเป็นสิ่งเดียวกัน

ไม่ได้ตั้งค่าไว้แยกกันสองที่ ซึ่งเป็นผลของการที่ StampedCombination ประกอบรหัสขึ้นจาก StampedNumber แต่ละชิ้นแทนที่จะเก็บรหัสเป็นตัวเลขของตัวเอง

ผลที่สำคัญกว่าตัวคำตอบคือ **ปริศนาสามองก์จากห้าองก์ใช้โปรแกรมชุดเดียวกันทั้งหมด** ต่างกันเพียงค่าที่ตั้งใน Inspector และจำนวนวงล้อ ดังแสดงในตารางที่ 4.3

ตารางที่ 4.3 กลไกที่แต่ละองก์ใช้ร่วมกันและส่วนที่เขียนขึ้นเฉพาะ

กลไก	องก์ที่ใช้	คลาสที่ใช้ร่วมกัน	คลาสที่เขียนขึ้นเฉพาะองก์
รหัสตัวเลข	1, 3, 4	CodeLock + CodeWheel + Openable	องก์ 4 เพิ่ม StampedNumber, StampedCombination
ตั้งเวลาให้ตรง	2	TimeLock + ClockSetter + ClockMovement	TestRun, RunTicking, Sandglass, HeldSandglass
ใช้สิ่งของ (กุญแจ)	ทุกองก์	KeyLock + IItemReceiver + Inventory + PickupItem	—
ประกอบชิ้นส่วนเข้าช่อง	4, 5	ClockPartSocket	—
สังเกตและจัดตำแหน่ง	5	Openable + StarLock	Horologium, Prism
เปิดเองเมื่อเงื่อนไขครบ	4	Openable + SealedLock	ChimeRelease

– *หลักฐาน:* CodeLock 3 instance, CodeWheel 11 instance (4 + 3 + 4) ในฉาก ยืนยันด้วย find_gameobjects เมื่อ 6 พ.ย. 2569 · ไม่พบคลาสที่ตั้งชื่อผูกกับองก์ใดองก์หนึ่ง เช่น Puzzle1 หรือ HallPuzzle

– **ผลเชิงวิศวกรรม:** ปริศนาที่ผู้เล่นรับรู้ว่าต่างกันสามแบบ (อ่านเลขจากนาฬิกา นับเสียงนก เรียงตามภาพถ่าย) ใช้ implementation เดียวกันทั้งหมด ความต่างอยู่ที่ *คำใบ้* ไม่ใช่ที่ *กลอน* ซึ่งเป็นผลโดยตรงของการแยกหน้าที่ระหว่าง Openable กับ ILock

[วางรูปที่ 4.6 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-06.png

รูปที่ 4.6 ผนังนาฬิกาเจ็ดเรือนขององค์ที่หนึ่ง ให้เห็นลูกตุ้มที่แกว่งและที่หยุด

[วางรูปที่ 4.7 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-07.png

รูปที่ 4.7 การยกนาฬิกาขึ้นดูและแป้นเลขที่ปรากฏเมื่อไขลาน

[วางรูปที่ 4.8 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-08.png

รูปที่ 4.8 นาฬิกาพกและนาฬิกาทรายที่ชูขึ้นข้างกันขณะจับเวลาในองก์ที่สอง



รูปที่ 4.9 ภาพถ่ายเก่าและป้ายเลขหลังของในองก์ที่สี่



รูปที่ 4.10 จุดแสงหกจุดบนจอก่อนและหลังหมุนปริซึมถูกต้อง ในองก์ที่ห้า

4.1.3 ผลของการสร้างเนื้อหาด้วยโปรแกรม

สัดส่วนของงานเขียนโปรแกรมที่วัดได้ให้ผลที่ไม่ตรงกับความคาดหมายทั่วไปของโครงงานเกม ดังตารางที่ 4.4

ตารางที่ 4.4 จำนวนบรรทัดของโปรแกรมแยกตามบทบาท

บทบาท	ไฟล์	บรรทัด	สัดส่วน	ทำงานในเกมหรือไม่
โปรแกรมส่วนที่ทำงานในเกม	63	3,481	32.8%	ทำงาน
โปรแกรมสร้างเนื้อหาสำหรับตัวแก้ไข	11	5,772	54.4%	**ไม่ทำงาน** ไม่ติดไปกับเกมที่ build
ชุดทดสอบ	3	393	3.7%	ทำงานเฉพาะเมื่อรันทดสอบ
ไฟล์ที่ Unity สร้างให้จากไฟล์ค่าอินพุต	1	958	9.0%	ทำงาน แต่ไม่ได้เขียนเอง
รวม	**78**	**10,604**	**100%**	

วิธีนับ: นับ **บรรทัดที่ไม่ว่าง** ของไฟล์ .cs ทุกไฟล์ใน Assets/ การนับทุกบรรทัดรวมบรรทัดว่างจะได้ 12,623 บรรทัด แยกเป็น 4,196 / 6,954 / 484 / 989 ตามลำดับเดียวกัน ตัวเลขในตารางนี้และทุกตัวเลขบรรทัดในรายงานใช้วิธีเดียวกันทั้งหมด คือไม่นับบรรทัดว่าง

– **ผลที่พบ:** โปรแกรมเกินครึ่งของโครงงาน (5,772 จาก 10,604 บรรทัด) เป็นเครื่องมือสร้างเนื้อหา ไม่ใช่ตัวเกม ไฟล์ที่ใหญ่ที่สุดในโครงงานคือ Assets/Scripts/Editor/PuzzleBuilder.cs 1,727 บรรทัด และ DressingBuilder.cs 966 บรรทัด ซึ่งทั้งสองไฟล์ไม่มีบรรทัดใดทำงานขณะผู้เล่นเล่น

– **ผลสืบเนื่องที่ยืนยันได้:** เรขาคณิตและพื้นผิวทุกชิ้นในเกมถูกสร้างจากโปรแกรม ไม่ได้ปั้นด้วยโปรแกรมสามมิติ — ตรวจแล้วไม่พบไฟล์ .blend, .obj หรือ .dae ในโปรเจกต์เลย ส่วนไฟล์ .fbx ที่พบหกไฟล์อยู่ในโฟลเดอร์ Assets/Character Models/ ทั้งหมดและมีวันที่ 20 มีนาคม 2562 ซึ่งเป็นชุดโมเดลที่นำเข้ามา ไม่ใช่งานของโครงงาน

– **ข้อแลกเปลี่ยนที่ตามมา:** การแก้เรขาคณิตด้วยมือใน Editor จะถูกเขียนทับเมื่อสั่งสร้างใหม่ครั้งถัดไป การแก้รูปทรงจึงต้องแก้ที่โปรแกรมสร้าง ไม่ใช่ที่วัตถุในฉาก ซึ่งเป็นข้อจำกัดของกระบวนการทำงาน ไม่ใช่ข้อบกพร่องของโปรแกรม

[วางรูปที่ 4.11 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-11.png

รูปที่ 4.11 ผังบ้านทั้งห้าห้องมองจากด้านบนใน Scene View

4.2 ผลการวิเคราะห์โครงสร้างซอฟต์แวร์

หัวข้อนี้รายงานผลการวัดคุณสมบัติเชิงโครงสร้างของโปรแกรมที่เขียนขึ้นตามทฤษฎีการออกแบบเชิงโมดูลในบทที่ 2 ผลทุกข้อเป็นการวัดจากไฟล์และระบุวิธีวัดกำกับไว้เพื่อให้ตรวจซ้ำได้

4.2.1 ผลการแบ่งโมดูล

โปรแกรมถูกแบ่งเป็นเก้ากลุ่มตามหน้าที่ โดยหกกลุ่มมีขอบเขตบังคับระดับการคอมไพล์ด้วยไฟล์ Assembly Definition [23] และสามกลุ่มไม่มี ดังตารางที่ 4.5

ตารางที่ 4.5 โมดูลทั้งเก้ากลุ่มและขนาดของแต่ละกลุ่ม

โมดูล	โฟลเดอร์	Assembly	ไฟล์	บรรทัด	หน้าที่เดียวที่รับผิดชอบ
Items	Scripts/Items/	EscapeRoom.Items	2	37	นิยามว่าสิ่งของหนึ่งชิ้นคืออะไร และผู้เล่นถืออะไรอยู่
Interaction	Scripts/Interaction/	EscapeRoom.Interaction	11	373	สัญญากลางของการโต้ตอบ และกลไกเปิด–ปิด–ล็อก
Clock	Scripts/Clock/	EscapeRoom.Clock	23	1,334	นาฬิกาทุกชนิด การเก็บเวลา การแสดงเวลา และกลไกที่ทำให้เดิน
Puzzle	Scripts/Puzzle/	EscapeRoom.Puzzle	8	384	ปริศนาเฉพาะเรื่อง คือรหัสตัวเลขและกลไกแสง
World	Scripts/World/	EscapeRoom.World	3	162	สภาพแวดล้อมที่ไม่เกี่ยวกับปริศนา และผังบ้านในรูปข้อมูล
Player	Scripts/Player/	*Assembly-CSharp*	10	830	ร่างผู้เล่น การอ่านอินพุต การเดิน การมอง การเล็ง การยกของขึ้นดู
UI	Scripts/UI/	*Assembly-CSharp*	3	216	การแสดงผลบนจอ
Game	Scripts/Game/	*Assembly-CSharp*	3	145	การเริ่มเกม การจบเกม และบรรยากาศแสง
Editor tooling	Scripts/Editor/	*Assembly-CSharp-Editor*	11	5,772	สร้างเรขาคณิต พื้นผิว และ prefab จากโปรแกรม
(ชุดทดสอบ)	Tests/PlayMode/	EscapeRoom.Tests	3	393	—

– **ผลที่พบ:** โครงงานนี้ **ไม่ใช้ namespace เลยทั้งโปรเจกต์** ทุกคลาสอยู่ใน global namespace และทุกไฟล์ .asmdef ตั้ง "rootNamespace": ""

– **ผลเชิงวิศวกรรม:** การแบ่งโมดูลที่นี่ไม่ได้ใช้ namespace เป็นเครื่องมือ แต่ใช้ Assembly Definition แทน ซึ่งเข้มกว่าในแง่การบังคับ — namespace เพียงจัดกลุ่มชื่อและข้ามได้ด้วยคำสั่ง using ขณะที่ Assembly Definition **ปิดกั้นการอ้างถึงในระดับการคอมไพล์** [23] ข้อเสียที่ต้องยอมรับคือหากนำ asset จากภายนอกเข้ามาแล้วมีคลาสชื่อซ้ำ โปรเจกต์จะคอมไพล์ไม่ผ่านทันที

4.2.2 ผลการวัดการพึ่งพาระหว่างโมดูล

วิธีวัด: สำหรับแต่ละโฟลเดอร์โมดูล ลบคอมเมนต์ออกจากทุกบรรทัดก่อน แล้วค้นหาชื่อชนิดข้อมูลทุกตัวที่ประกาศในโมดูลอื่น หากพบถือว่ามีการพึ่งพาจริงในโปรแกรม การตัดคอมเมนต์ออกก่อนเป็นขั้นตอนที่จำเป็น เพราะพบกรณีที่ชื่อชนิดข้อมูลของอีกโมดูลปรากฏในคอมเมนต์เท่านั้น

ตารางที่ 4.6 การพึ่งพาระหว่างโมดูลที่วัดได้จากโปรแกรม

จากโมดูล	ไปโมดูล	ตัวอย่างหลักฐาน
Items	*ไม่พึ่งโมดูลใด*	—
World	*ไม่พึ่งโมดูลใด*	—
Interaction	Items	IActor.cs:25, KeyLock.cs:11,26, PickupItem.cs:8
Clock	Items	ClockPartSocket.cs:13,43
Clock	Interaction	ChimeRelease.cs:20,35,41, ClockPartSocket.cs:10 และอีก 15 จุด
Puzzle	Interaction	CodeLock.cs:12,13,29,37 และอีก 7 จุด
Player	Interaction	InspectView.cs:57,61,72,83 และอีก 20 จุด
Player	Items	HeldSandglass.cs:13,14,27, PlayerInteractor.cs:12 และอีก 4 จุด
Player	UI	InspectView.cs:25, PlayerInteractor.cs:31,33,40
Player	Clock	HeldSandglass.cs:12
UI	Player	InventoryUI.cs:18,19,63,70

จากโมดูล	ไปโมดูล	ตัวอย่างหลักฐาน
UI	Items	InventoryUI.cs:20,29,42,50 และอีก 3 จุด
UI	Interaction	InventoryUI.cs:31,56, PagePanel.cs:21
Game	Player	GameManager.cs:17,45, TitleScreen.cs:12,22
Game	Interaction	GameManager.cs:15



รูปที่ 4.12 แผนภาพการพึ่งพาระหว่างโมดูลทั้งแปดกลุ่ม

ผลที่สำคัญสามข้อ

1. **โมดูล Clock และ Puzzle ไม่อ้างถึงโมดูล Player หรือ UI แม้แต่ครั้งเดียว** ยืนยันด้วยการค้นหาชื่อ PlayerInteractor, PlayerHud, PlayerModeController, InspectView, InputReader และ InventoryUI ในโฟลเดอร์ Clock/ และ Puzzle/ ได้ผลเป็นศูนย์

นี่คือผลที่วัดได้ของการสร้างส่วนต่อประสาน IActor ในขั้นที่ 3 — นาฬิกาและปริศนาถูกสั่งงานผ่านสัญญาสามเมธอด (Say, OfferBag, Take) ไม่ใช่ผ่านคลาสของผู้เล่น ผลคือโมดูลทั้งสองคอมไพล์และทดสอบได้โดยไม่ต้องมีผู้เล่นอยู่เลย ซึ่งเป็นสิ่งที่ชุดทดสอบในหัวข้อ 4.3.1 ใช้จริง

2. **พบการพึ่งพา Player ⟶ Clock หนึ่งจุด** คือ HeldSandglass.cs:12 ที่ประกาศ [SerializeField] private Sandglass glass;

สาเหตุคือคลาส Sandglass จงใจไม่ implement ส่วนต่อประสานใดเลย เพราะ prop นี้มี PickupItem อยู่แล้ว การใส่ IInteractable เพิ่มจะทำให้กำกวมว่าปุ่ม E ไปถึงตัวไหน (คอมเมนต์ที่

Sandglass.cs:10-11) จึงต้องเรียกเมธอดบนคลาสรูปธรรมโดยตรง เป็นเหตุผลที่สมเหตุสมผลในระดับการเล่น แต่ผลข้างเคียงทางโครงสร้างคือชั้น Player ทะลุไปรู้จักคลาสรูปธรรมในโมดูล Clock **เอกสารประกอบภายในโปรเจกต์ไม่ได้บันทึกการพึ่งพาเส้นนี้ไว้** จึงเป็นหนึ่งในสี่จุดที่เอกสารไม่ตรงกับโปรแกรม (หัวข้อ 4.4.2)

3. **โมดูล EscapeRoom.World ไม่มีการพึ่งพาเข้าหรือออกเลย** จึงย้ายไปใช้ในโปรเจกต์อื่นได้ทันทีโดยไม่ต้องแก้อะไร เป็นโมดูลเดียวในโครงงานที่มีคุณสมบัตินี้

4.2.3 ผลการเทียบขอบเขตที่บังคับไว้กับที่โปรแกรมใช้จริง

เนื่องจากไฟล์ Assembly Definition เป็นตัวประกาศว่าโมดูลหนึ่งได้รับอนุญาตให้อ้างถึงโมดูลใดได้ จึงนำสิ่งที่ประกาศไว้มาเทียบกับสิ่งที่โปรแกรมใช้จริง ผลแสดงในตารางที่ 4.7

ตารางที่ 4.7 การเทียบขอบเขตที่ประกาศไว้กับที่โปรแกรมใช้จริง

Assembly	ประกาศให้อ้างถึงได้	โปรแกรมใช้จริง	ผล
EscapeRoom.Items	*ไม่มี*	—	ตรง
EscapeRoom.Interaction	Items	Items	ตรง
EscapeRoom.Clock	Items, Interaction	Items, Interaction	ตรง
EscapeRoom.World	*ไม่มี*	—	ตรง
EscapeRoom.Puzzle	Items, Interaction, **Clock**	Interaction เท่านั้น	**ประกาศเกินสองรายการ**
EscapeRoom.Tests	Items, Interaction, Clock, **Puzzle**	Items, Interaction, Clock	**ประกาศเกินหนึ่งรายการ**

– **ผลที่พบ:** EscapeRoom.Puzzle.asmdef ประกาศว่าอ้าง EscapeRoom.Items และ EscapeRoom.Clock ได้ แต่โปรแกรมในโฟลเดอร์ Puzzle/ ไม่ได้ใช้ชนิดข้อมูลจากสองโมดูลนั้นเลย กรณีเดียวที่พบคำว่า IClockDisplay อยู่ในคอมเมนต์ที่ ICodeInput.cs:5

– **ผลกระทบ:** ไม่ใช่ข้อบกพร่องและไม่กระทบการทำงาน แต่ทำให้ขอบเขตโมดูลหลวมกว่าที่โปรแกรมต้องการจริง หากในอนาคตมีผู้เขียนให้ Horologium ไปอ่าน ClockMovement โดยตรง คอมไพเลอร์จะไม่แจ้งเตือน

เพราะการอ้างถึงเปิดไว้แล้ว การตัดรายการที่ไม่ใช้ออกจะทำให้ไฟล์ Assembly Definition เป็นเอกสารที่บังคับใช้ได้ของการพึ่งพาที่เกิดขึ้นจริง ซึ่งเป็นเหตุผลที่เลือกใช้เครื่องมือนี้ตั้งแต่ต้น

– **เป็นข้อเสนอแนะที่แก้ได้ทันที** โดยลบสองรายการออกจากไฟล์ .asmdef และไม่ต้องแก้โปรแกรมเลย

สำหรับสาเหตุที่โมดูล Player, UI และ Game ไม่มีไฟล์ Assembly Definition มีข้อจำกัดทางเทคนิคบังคับอยู่สองข้อ

1. **พบวงจรการพึ่งพา Player ⇄ UI** (ตารางที่ 4.6) และ Unity ห้ามการอ้างถึงกันเป็นวงจรระหว่าง assembly เด็ดขาด [23] จึงใส่ไฟล์ Assembly Definition ให้สองโมดูลนี้ไม่ได้โดยไม่ตัดวงจรก่อน

2. ไฟล์ PlayerControls.cs ที่ Unity สร้างให้จากค่าอินพุตอยู่ที่ Assets/ ไม่ได้อยู่ใน Scripts/ หากสร้าง assembly สำหรับโมดูล Player คลาสในนั้นจะมองไม่เห็นไฟล์นี้เพราะอยู่ต่าง assembly

และเนื่องจากกฎของ Unity คือ predefined assembly อ้างถึง Assembly Definition ได้ แต่ Assembly Definition อ้างถึง predefined assembly ไม่ได้ [23] การวางชั้นล่างไว้ในไฟล์ Assembly Definition และปล่อยชั้นบนไว้ใน Assembly-CSharp จึงเป็นการ **บังคับทิศทางการพึ่งพาให้เป็นทางเดียวโดยอัตโนมัติ** — การเขียน GetComponent<PlayerMovement>() ลงในคลาส ClockDial จะคอมไพล์ไม่ผ่าน ผลข้างเคียงของการเลือกโครงแบบนี้คือชุดทดสอบไม่สามารถครอบสามโมดูลชั้นบนได้ ซึ่งรายงานไว้ในหัวข้อ 4.3.2

4.2.4 ผลการใช้ส่วนต่อประสานและการนำกลับมาใช้

ตารางที่ 4.8 ส่วนต่อประสานทั้งสิบสองรายการและจำนวนคลาสที่นำไปใช้

ส่วนต่อประสาน	โมดูล	จำนวน implementation	รายชื่อคลาส
IInteractable	Interaction	9	Openable, PickupItem, ClockWinder, ClockSetter, WeightChain, ClockPartSocket, TestRun, CodeWheel,

ส่วนต่อประสาน	โมดูล	จำนวน implementation	รายชื่อคลาส
			Prism
ILock	Interaction	6	KeyLock, SealedLock, TimeLock, RunningLock, CodeLock, StarLock
IClockPart	Clock	3	Mainspring, Pendulum, TestRun
IItemReceiver	Interaction	2	KeyLock, ClockPartSocket
IClockDisplay	Clock	2	ClockDial, DigitalReadout
IActor	Interaction	1	PlayerInteractor (และ RecordingActor ในชุดทดสอบ)
IInspectable	Interaction	1	Inspectable
IReadable	Interaction	1	WrittenPage
IPowerStore	Clock	1	Mainspring
ICodeInput	Puzzle	1	CodeWheel
IInHandOnly	Interaction	1	TestRun (เป็น marker interface ไม่มีเมธอด)
ITimedTest	Interaction	1	TestRun
รวม		**29 ความสัมพันธ์**	จาก **22 คลาส**

วิธีนับ: นับ "ความสัมพันธ์" คือหนึ่งคลาส implement หนึ่งส่วนต่อประสาน หากหนึ่งคลาส implement สามส่วนต่อประสาน (เช่น TestRun) นับเป็นสามความสัมพันธ์ จึงได้ 29 ความสัมพันธ์จาก 22 คลาสที่ implement อย่างน้อยหนึ่งรายการ

การนำกลับมาใช้ที่วัดได้แสดงในตารางที่ 4.9 โดยเลือกเฉพาะกรณีที่ยืนยันได้จากไฟล์ ไม่ใช่กรณีที่เพียงดูเหมือนจะนำกลับมาใช้ได้

ตารางที่ 4.9 หลักฐานการนำกลับมาใช้ที่วัดได้

สิ่งที่นำกลับมาใช้	ใช้ที่ไหน	จำนวน	หลักฐาน
ClockPartSocket	ช่องใส่ลูกตุ้มและตุ้มถ่วงขององค์ 4 **และ** ช่องเสียบปริซึมขององค์ 5	5 instance	ค้น GUID ของ ClockPartSocket.cs พบใน Sundial.prefab 3 ครั้ง และใน Main.unity 2 ครั้ง รวม 5 ตรงกับที่ Unity รายงาน
CodeLock + CodeWheel	องก์ 1, 3 และ 4	CodeLock 3, CodeWheel 11	find_gameobjects ยืนยันเมื่อ 6 พ.ย. 2569
Prefabs/Clocks/ClockDial.prefab	หน้าปัดของนาฬิกาทุกเรือน	46 instance	find_gameobjects ด้วย component ClockDial
Prefabs/Clocks/Parts/Mainspring.prefab	แหล่งพลังของนาฬิกา	45 instance	find_gameobjects ด้วย component Mainspring
Prefabs/House/DoorLeaf.prefab	บานประตูภายในทั้งสี่บาน	4 บาน	นับวัตถุลูกของ House/Doors ที่ใช้ prefab นี้

กรณี ClockPartSocket เป็นหลักฐานที่หนักที่สุด เพราะเป็น **การนำกลับมาใช้ข้ามโมดูลและข้ามองก์โดยไม่แก้โปรแกรมเลย** คลาสนี้อยู่ในโมดูล Clock และเขียนขึ้นเพื่อใส่ลูกตุ้มเข้านาฬิกา โดยรับค่า ItemData กับรายการวัตถุที่จะเปิด แล้วเปิดวัตถุนั้นเมื่อใส่ของถูก (ClockPartSocket.cs:43-53) สิ่งที่ถูกเปิดคือคลาส Pendulum (ซึ่งเป็น IClockPart ในโมดูล Clock) ในกรณีหนึ่ง และคลาส Prism (ซึ่งเป็น IInteractable ในโมดูล Puzzle) ในอีกกรณี โดยที่ ClockPartSocket ไม่รู้ความต่างระหว่างสองสิ่งนั้นเลย

ผลอีกข้อหนึ่งคือ **ชนิดของนาฬิกาเป็นไฟล์ข้อมูล ไม่ใช่เมธอดในโปรแกรม** คลาส ClockRecipe เก็บตัวเรือนเปล่าหนึ่งชิ้นกับรายการสิ่งที่ประกอบเข้าไป โดยมี **รายการเดียว ไม่แยกว่าอะไรคือส่วนแสดงเวลาและอะไรคือชิ้นส่วนกลไก** (ClockRecipe.cs:37) และ ClockMovement แยกเองตอน Awake ด้วยการถามว่าแต่ละชิ้น implement ส่วนต่อประสานใด (ClockMovement.cs:112,116) ไฟล์ Assets/Recipes/PlankClock.asset คือหลักฐานว่าใช้งานได้จริง — เป็นนาฬิกาที่ประกอบจากไฟล์ข้อมูลเพียงไฟล์เดียว ไม่มีเมธอดในโปรแกรมรองรับเฉพาะ และมีชุดทดสอบสามกรณียืนยันว่าทำงานเหมือนนาฬิกาที่เขียนด้วยโปรแกรม

[วางรูปที่ 4.13 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-13.png

รูปที่ 4.13 ไฟล์ PlankClock.asset ใน Inspector ให้เห็นรายการ Fittings ที่ไม่แยกส่วนแสดงเวลาจากชิ้นส่วนกลไก

4.2.5 ผลการตรวจรูปแบบการเขียนโปรแกรมเชิงปริมาณ

ผลการนับรูปแบบการเขียนโปรแกรมที่เป็นตัวชี้วัดของการพึ่งพาแบบต่าง ๆ แสดงดังตารางที่ 4.10 ค่าที่เป็นศูนย์ในตารางนี้มีความหมายทางวิศวกรรมมากกว่าค่าที่ไม่เป็นศูนย์ เพราะเป็นรูปแบบที่สร้างการพึ่งพาแบบมองไม่เห็นหรือการพึ่งพาข้ามทั้งระบบ

ตารางที่ 4.10 ผลการนับรูปแบบการเขียนโปรแกรมในโปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์

รูปแบบ	จำนวนที่พบ	ความหมายเมื่อพบ
abstract	0	ไม่มีคลาสฐานที่ต้องสืบทอด
virtual	0	ไม่มีเมธอดที่ออกแบบให้เขียนทับ
override	0	ไม่มีการเขียนทับเมธอดของคลาสแม่
คลาสที่สืบทอดจาก MonoBehaviour โดยตรง	51	ทุกคลาสอยู่ระดับเดียวกัน ไม่มีลำดับชั้นให้ไล่
Singleton (static … Instance)	0	ไม่มีวัตถุที่ทุกคลาสเรียกถึงได้

รูปแบบ	จำนวนที่พบ	ความหมายเมื่อพบ
FindObjectOfType / FindAnyObjectByType	**0**	ไม่มีการค้นหาวัตถุด้วยชนิดตอนทำงาน
GameObject.Find	**0**	ไม่มีการค้นหาวัตถุด้วยชื่อเป็นข้อความ
SendMessage	**0**	ไม่มีการเรียกเมธอดด้วยชื่อเป็นข้อความ
DontDestroyOnLoad	**0**	ไม่มีวัตถุที่อยู่ข้ามฉาก
UnityEvent	**0**	ไม่มีการต่อเหตุการณ์ผ่าน Inspector
StartCoroutine	**0**	ไม่มีการทำงานแบบ coroutine
event Action	**5**	การแจ้งเหตุการณ์แบบ Observer (3 จุดใช้งานจริง 2 จุดประกาศในส่วนต่อประสาน)
[SerializeField]	**215** ใน 53 ไฟล์	การอ้างอิงที่ตั้งค่าผ่าน Inspector
GetComponent และตระกูล	**43**	การค้นหาผู้ร่วมงานผ่านส่วนต่อประสาน

– **ผลเชิงวิศวกรรม:** ชุดค่าศูนย์นี้ยืนยันว่าโครงงานไม่ได้ใช้รูปแบบที่สร้างการพึ่งพาแบบ global เลย ซึ่งรวมถึง Singleton ที่ทุกคลาสเรียกถึงได้ การค้นหาวัตถุด้วยชื่อหรือชนิดตอนทำงาน และการเรียกเมธอดด้วยชื่อเป็นข้อความ รูปแบบทั้งสามทำให้การพึ่งพาไม่ปรากฏในลายเซ็นของเมธอดและคอมไพเลอร์ตรวจไม่ได้ [1]

– **ผลสำคัญของการไม่มี inheritance:** การที่ abstract, virtual และ override เป็นศูนย์ทั้งหมด ขณะที่มีคลาส 51 คลาสและส่วนต่อประสาน 12 รายการ หมายความว่าความหลากหลาย (polymorphism) ในโครงงานนี้เกิดจากการ implement ส่วนต่อประสานเท่านั้น ไม่ได้เกิดจากการสืบทอด ซึ่งเป็นรูปแบบ composition over inheritance ตามที่อธิบายในบทที่ 2 [5], [8]

– **จุดที่ยังเป็นการพึ่งพาแบบแน่น:** ค่า 215 ของ [SerializeField] คือการอ้างอิงไปยัง **คลาสรูปธรรม** ที่ลากต่อใน Inspector ซึ่งมองจากตัวโปรแกรมไม่เห็นว่าอะไรต่อกับอะไร ต้องเปิด Unity ดู รายงานเป็นข้อจำกัดในหัวข้อ 4.4.2

4.2.6 ผลการประเมินความสามารถในการขยาย

ประเมินตามหลัก Open–Closed Principle [8] คือหากเพิ่มสิ่งใหม่ได้ด้วยการสร้างไฟล์ใหม่เพียงอย่างเดียว ไม่ต้องแก้ไฟล์เดิม ถือว่าผ่าน ผลการประเมินเก้าสถานการณ์แสดงในตารางที่ 4.11

ตารางที่ 4.11 ผลการประเมินความสามารถในการขยายเก้าสถานการณ์

ต้องการเพิ่ม	ต้องเขียนไฟล์ใหม่	ต้องแก้ไฟล์เดิม	ผล
วิธีแสดงเวลาแบบใหม่	1 คลาส implement `IClockDisplay`	ไม่ต้อง	**ผ่าน** (มีชุดทดสอบยืนยัน)
ชิ้นส่วนนาฬิกาแบบใหม่	1 คลาส implement `IClockPart`	ไม่ต้อง	**ผ่าน**
แหล่งพลังแบบใหม่	1 คลาส implement `IPowerStore`	ไม่ต้อง	**ผ่าน**
กลอนแบบใหม่	1 คลาส implement `ILock`	ไม่ต้อง	**ผ่าน**
วัตถุที่โต้ตอบได้แบบใหม่	1 คลาส implement `IInteractable`	ไม่ต้อง	**ผ่าน**
ชนิดของนาฬิกาแบบใหม่	ไม่ต้องเขียนโปรแกรมเลย สร้างไฟล์ `ClockRecipe.asset`	ไม่ต้อง	**ผ่าน** (มีชุดทดสอบยืนยัน)
ปริศนารหัสที่มีห้าหลัก	ไม่ต้องเขียนโปรแกรม เพิ่ม `CodeWheel` และเพิ่มสมาชิกในรายการ	ไม่ต้อง	**ผ่าน**
อุปกรณ์ใส่รหัสแบบใหม่ เช่น แป้นกดตัวเลข	1 คลาส implement `ICodeInput`	**ต้องแก้ `CodeLock`**	**ไม่ผ่าน**
วิธีมองวัตถุแบบใหม่ นอกจาก Hold, Turn, Zoom	—	**ต้องเพิ่มค่าใน enum และแก้ `InspectView` หลายจุด**	**ไม่ผ่าน**

ผลคือผ่านเจ็ดจากเก้าสถานการณ์ และสองข้อที่ไม่ผ่านมีสาเหตุต่างกัน จึงต้องแยกรายงาน

ข้อที่ไม่ผ่านข้อแรก เป็นข้อจำกัดที่ยังแก้ไม่สำเร็จ ฟิลด์ที่รับอุปกรณ์ใส่รหัสใน `CodeLock` ประกาศเป็นชนิด `CodeWheel[]` ซึ่งเป็นคลาสรูปธรรม ไม่ใช่ `ICodeInput[]`

```
// Assets/Scripts/Puzzle/CodeLock.cs:15-17
[SerializeField]
[Tooltip("Left to right, as the player reads them. Anything
implementing ICodeInput.")]
private CodeWheel[] wheels;          // ชนิดเป็น CodeWheel ไม่ใช่
ICodeInput
```

```
// Assets/Scripts/Puzzle/CodeLock.cs:65-70
for (int i = 0; i < wheels.Length; i++)
{
    ICodeInput input = wheels[i];     // แปลงขึ้นเป็นส่วนต่อประสานตอนใช้
    if (input.Digit != combination[i]) return;
}
```

การเขียนคลาสใหม่ที่ implement ICodeInput จึงใส่ลงในรายการนี้ไม่ได้ ต้องแก้ชนิดของฟิลด์ใน CodeLock.cs ก่อน และการเปลี่ยนเป็น ICodeInput[] ตรง ๆ ก็ทำไม่ได้ เพราะ **Unity ไม่สามารถบันทึกค่าของฟิลด์ที่เป็นชนิดส่วนต่อประสานได้** [21] ทางแก้ที่ใช้ได้มีสามทาง คือเก็บเป็น MonoBehaviour[] แล้วแปลงชนิดตอนใช้ ใช้ attribute [SerializeReference] หรือค้นหาด้วย GetComponentsInChildren<ICodeInput>() แบบเดียวกับที่โมดูล Clock ทำ

ข้อสังเกตที่สำคัญคือ **โมดูล Clock แก้ปัญหานี้แล้วด้วยวิธีที่สาม แต่โมดูล Puzzle ยังไม่ได้แก้** จึงเป็นกรณีที่หลักการออกแบบของโครงงานยังไม่ถูกใช้อย่างทั่วถึง และคอมเมนต์ในโปรแกรมที่ระบุว่าเปลี่ยนอุปกรณ์ใส่รหัสได้โดยไม่แก้โปรแกรมนั้น **ยังไม่เป็นจริงตามสภาพของโปรแกรมปัจจุบัน** รายงานไว้ตามข้อเท็จจริง

ข้อที่ไม่ผ่านข้อที่สอง เป็นข้อแลกเปลี่ยนที่เลือกไว้โดยรู้ตัว โครงงานใช้ enum กับคำสั่ง switch อยู่สี่จุด ดังนี้

enum	จำนวนค่า	ผู้อ่านค่า	ผลเมื่อเพิ่มค่าใหม่
InspectStyle	3	InspectView (8 จุด)	ต้องแก้ InspectView
ClockCondition	5	ClockMovement.ConditionRate (:83-95)	ต้องแก้ ClockMovement
PlayerMode	5	PlayerModeController.SwitchTo (:60-74)	ต้องแก้ PlayerModeController
RoomBrightness	3	RoomLighting.LevelFor (:74-81)	ต้องแก้ RoomLighting

เกณฑ์ที่ใช้ตัดสินว่าจะใช้ enum หรือส่วนต่อประสานปรากฏสม่ำเสมอทั้งโครงงาน คือ **มิติที่รู้ค่าครบแล้วและคาดว่าจะไม่เพิ่ม ใช้ enum กับ switch ส่วนมิติที่คาดว่าจะเพิ่มเรื่อย ๆ ใช้ส่วนต่อประสาน** ซึ่งเป็นหลักที่ถูกต้องในวิศวกรรมซอฟต์แวร์ ไม่ใช่การเลี่ยงการออกแบบ มิติที่เลือกใช้ส่วนต่อประสานคือสิ่งที่แสดงเวลา ชิ้นส่วน กลอน และวัตถุที่โต้ตอบได้ ซึ่งทั้งสี่มิติมีจำนวน implementation เพิ่มขึ้นจริงระหว่างการพัฒนา (ตารางที่ 4.8) ขณะที่สี่ enum ข้างต้นมีจำนวนค่าคงเดิมตั้งแต่ถูกสร้าง

4.3 ผลการทดสอบ

4.3.1 ผลการทดสอบด้วยชุดทดสอบอัตโนมัติ

โครงงานมีชุดทดสอบ **15 กรณีในสามไฟล์** เขียนด้วย NUnit [26] และรันผ่าน Unity Test Framework รุ่น 1.6.0 [25] ทั้งหมดเป็น **PlayMode test** คือรันในสภาพแวดล้อมการทำงานจริงของเอนจิน ใช้ attribute `[UnityTest]` กับเมธอดที่คืนค่า `IEnumerator` และสั่ง `yield return null` เพื่อให้เฟรมเดินไปข้างหน้า ซึ่งจำเป็นเพราะสิ่งที่ทดสอบคือพฤติกรรมที่เกิดขึ้นข้ามเฟรม เช่น นาฬิกาเดินไปแล้วกี่นาที

ตารางที่ 4.12 ชุดทดสอบทั้ง 15 กรณีและสิ่งที่แต่ละกรณียืนยัน

#	ไฟล์	สิ่งที่กรณีนั้นยืนยัน
1	`ActorSeamTests.cs`	ตู้ที่ล็อกอยู่จะเรียก `IActor.OfferBag` และไม่เปิด
2	`ActorSeamTests.cs`	`KeyLock` คัดกุญแจผิดออก และเปิดได้เมื่อใส่กุญแจถูก
3	`ActorSeamTests.cs`	**กลอนสองชนิดต่างกันบน `Openable`** **เดียวทำงานร่วมกันได้โดยไม่ต้องมีคลาสใหม่** ข้ามสามโมดูล
4	`ActorSeamTests.cs`	`SealedLock` ปฏิเสธการกดทุกครั้ง เปิดได้จากภายนอกเท่านั้น และเลิกขัดขวางหลังเปิดแล้ว
5	`ActorSeamTests.cs`	**องก์ที่สี่ทั้งองก์ทำงานได้โดยไม่เปิดฉากและไม่มีผู้เล่น** คือเติมพลัง นาฬิกาเดิน ตีระฆัง แล้วผนังเปิดเมื่อตีจบ ไม่ใช่เปิดทันที
6	`ActorSeamTests.cs`	`ClockPartSocket` ซึ่งอยู่ในโมดูลที่ไม่รู้ว่ามีผู้เล่น ถูกสั่งงานผ่านสัญญาเดียวกันได้

#	ไฟล์	สิ่งที่กรณีนั้นยืนยัน
7	ClockSeamTests.cs	**คลาสแสดงเวลาที่เขียนขึ้นภายหลัง ยังได้รับเวลาจาก ClockMovement**
8	ClockSeamTests.cs	ส่วนแสดงเวลาสองตัวบนกลไกเดียวแสดงเวลาตรงกัน
9	ClockSeamTests.cs	กลไกที่ไม่มีชิ้นส่วนใดเลยเดินตลอดไป คือการไม่มีชิ้นส่วนไม่สามารถยับยั้งได้
10	ClockSeamTests.cs	แหล่งพลังที่ว่างเปล่าทำให้นาฬิกาหยุด
11	ClockSeamTests.cs	การเติมพลังทำให้นาฬิกาเดิน โดยกลไกไม่รู้ว่าใครเติม
12	ClockSeamTests.cs	**ชิ้นส่วนที่ยังปิดการทำงานอยู่ยังยับยั้งได้** ซึ่งเป็นรากของกลไกองก์ที่สี่
13	RecipeSeamTests.cs	นาฬิกาที่ประกอบจากไฟล์ข้อมูลบอกเวลา 3:25 และไม่เดิน ตามที่ระบุไว้ในไฟล์
14	RecipeSeamTests.cs	ชิ้นส่วนที่ระบุในไฟล์ข้อมูลยับยั้งได้จริง
15	RecipeSeamTests.cs	ส่วนแสดงเวลาที่ระบุในไฟล์ข้อมูลถูกขับเคลื่อนจริง ตรวจด้วยการวัดมุมของเข็ม

ลักษณะเฉพาะของชุดทดสอบนี้ที่ควรระบุคือ

เป็นการทดสอบว่าคุณสมบัติของโครงสร้างเป็นจริงหรือไม่

ไม่ใช่การทดสอบว่าฟังก์ชันคืนค่าถูกต้องหรือไม่ ซึ่งตรงกับนิยามของ architecture fitness function ในบทที่ 2 [18] ตัวอย่างที่ชัดที่สุดคือกรณีที่ 7

```
// Assets/Tests/PlayMode/ClockSeamTests.cs:41-67
// คลาสนี้ประกาศอยู่ในไฟล์ชุดทดสอบ จึงไม่มีอยู่ตอน ClockMovement ถูกคอมไพล์
private class CountingDisplay : MonoBehaviour, IClockDisplay
{
    public int Shown; public float Last;
    public void Show(float minutes) { Shown++; Last = minutes; }
}


[UnityTest]
public IEnumerator
A_display_written_after_the_movement_still_gets_the_time()
{
    GameObject go = Clock(out ClockMovement movement);
    var display = go.AddComponent<CountingDisplay>();
    Set(movement, "startHour", 4); Set(movement, "startMinute",
40);
    go.SetActive(true);
    yield return null;
```

```
    Assert.Greater(display.Shown, 0, "the movement never called
Show");
    Assert.AreEqual(4 * 60 + 40, display.Last, 1f, "it was handed
the wrong time");
}
```

กรณีนี้จะล้มทันทีหากมีผู้แก้ ClockMovement ให้ถือฟิลด์ชี้ไปที่หน้าปัดโดยตรงแทนการค้นหาด้วยส่วนต่อประสาน และกรณีที่ 5 จะล้มหากมีผู้ทำให้ผนังลับเปิดได้ด้วยการกดปุ่ม E ชุดทดสอบจึงทำหน้าที่ **ปิดกั้นการถอยกลับของโครงสร้าง** ไม่ใช่เพียงตรวจความถูกต้องของค่า

หลักฐานอีกชิ้นหนึ่งคือคลาสที่ทำหน้าที่แทนผู้เล่นในชุดทดสอบ ซึ่งเป็นคลาส C# ธรรมดาสิบบรรทัด ไม่ใช่ MonoBehaviour และไม่อยู่ในฉากใด

```
// Assets/Tests/PlayMode/ActorSeamTests.cs:18-27
private class RecordingActor : IActor
{
    public readonly List<string> Said = new List<string>();
    public readonly List<IItemReceiver> BagOfferedTo = new
List<IItemReceiver>();
    public readonly List<ItemData> Taken = new List<ItemData>();
    public void Say(string message) => Said.Add(message);
    public void OfferBag(IItemReceiver to) =>
BagOfferedTo.Add(to);
    public void Take(ItemData item) => Taken.Add(item);
}
```

คลาสนี้เปิดกลอน ใส่ชิ้นส่วนนาฬิกา และรับสิ่งของได้ครบทุกอย่างที่ผู้เล่นทำได้ในส่วนที่เกี่ยวกับกลอนและนาฬิกา ซึ่งเป็นการยืนยันเชิงปฏิบัติว่าสัญญา IActor สามเมธอดเพียงพอต่อสิ่งที่ prop ต้องการจากผู้ทำงานกับมัน



รูปที่ 4.14 ผลการรันชุดทดสอบใน Test Runner แสดงทั้ง 15 กรณีและเวลาที่ใช้

> **สถานะของผลการรัน — ต้องระบุตามความจริง ยังไม่มีหลักฐานว่าชุดทดสอบทั้ง 15 กรณีผ่าน**
> เพราะการสำรวจโปรเจกต์ทั้งสองเฟสกำหนดไว้ว่าห้ามรันชุดทดสอบ
> เนื่องจากการรันสร้างและลบไฟล์ในโปรเจกต์ อย่างไรก็ตาม
> **พบหลักฐานว่าชุดทดสอบเคยถูกรันจริงอย่างน้อยหนึ่งครั้ง** คือไฟล์ Assets/InitTestScene855cb578-5e59-4f44-aa4a-008404e0b48a.unity ขนาด 5,704 ไบต์ เวลาแก้ไขล่าสุด **7 กันยายน 2569 เวลา 16:53 น.** เนื้อหาเป็นฉากเปล่า และไม่อยู่ในรายการฉากที่จะถูก build *การตีความ:* ชื่อในรูปแบบ InitTestScene ตามด้วยรหัส GUID เป็นรูปแบบที่ Unity Test Framework สร้างขึ้นเมื่อเริ่มรัน PlayMode test และปกติจะลบทิ้งเมื่อรันเสร็จ
> การที่ไฟล์ค้างอยู่บ่งชี้ว่ามีการรันและการรันนั้นจบลงโดยไฟล์ไม่ถูกเก็บกวาด
> ตำแหน่งในลำดับเวลาสนับสนุนข้อนี้ — แก้ InspectView.cs 16:04 น. จากนั้นไฟล์นี้ 16:53 น. แล้วเขียน ActorSeamTests.cs 17:15 น. **สิ่งที่หลักฐานนี้บอกไม่ได้** คือผลการรัน จำนวนกรณีที่รัน และรันกี่ครั้ง จึงยังต้องรัน Test Runner เองเพื่อเก็บรูปที่ 4.14

4.3.2 ขอบเขตที่ชุดทดสอบไม่ครอบ

ตารางที่ 4.13 ความครอบคลุมของชุดทดสอบแยกตามโมดูล

โมดูล	มีชุดทดสอบครอบ	เหตุผลเมื่อไม่มี
Items	ครอบ (ทางอ้อม ผ่าน ItemData ในสี่กรณี)	—
Interaction	ครอบ (Openable, KeyLock, SealedLock, IActor, ILock, IItemReceiver)	—
Clock	ครอบ (ClockMovement, Mainspring, Pendulum, ClockDial, ClockPartSocket, ChimeRelease, RunningLock)	—
Puzzle	**ไม่ครอบ**	ไม่มีข้อจำกัดทางเทคนิค การอ้างถึงเปิดไว้แล้วในไฟล์ .asmdef แต่ยังไม่ได้เขียน
World	ไม่ครอบ	ไม่มีข้อจำกัดทางเทคนิค
Player	**ไม่ครอบ**	**ใส่ไม่ได้** — อยู่ใน Assembly-CSharp ซึ่ง .asmdef ของชุดทดสอบอ้างถึงไม่ได้ [23]

โมดูล	มีชุดทดสอบครอบ	เหตุผลเมื่อไม่มี
UI	**ไม่ครอบ**	เหตุผลเดียวกัน
Game	**ไม่ครอบ**	เหตุผลเดียวกัน
Editor tooling	ไม่ครอบ	—

หลักฐาน: ชนิดข้อมูลทั้งหมดที่ชุดทดสอบใช้คือ Openable, KeyLock, SealedLock, ClockMovement, Mainspring, Pendulum, ClockDial, ClockPartSocket, ChimeRelease, RunningLock, IPowerStore, IClockDisplay, IActor, IItemReceiver, ItemData และ ClockCondition — ไม่มีชนิดข้อมูลจากโมดูล Puzzle เลยสักรายการ

ผลนี้มีประเด็นที่ลึกกว่าการเขียนชุดทดสอบไม่ครบ คือ **การที่สามโมดูลชั้นบนไม่มีชุดทดสอบเป็นผลโดยตรงจากการเลือกโครง assembly ในขั้นที่ 4** ไม่ใช่การละเลย เพราะกฎของ Unity ห้าม Assembly Definition อ้างถึง predefined assembly [23] การจะทดสอบสามโมดูลนี้ต้องตัดวงจร Player ⇄ UI และย้ายไฟล์ PlayerControls.cs ก่อน ซึ่งเป็นงานที่ใหญ่กว่าการเขียนชุดทดสอบเอง

ส่วนโมดูล Puzzle **ไม่มีข้อจำกัดแบบนั้น** และเป็นช่องว่างที่เติมได้ทันที ซึ่งมีน้ำหนักเพิ่มขึ้นเมื่อพิจารณาร่วมกับข้อผิดพลาดของรหัสองก์ที่หนึ่งในหัวข้อ 4.1.2 — ชุดทดสอบที่ตรวจว่า CodeLock บนตู้โชว์ถือรหัสตรงกับเลขบนนาฬิกาที่ยังเดิน จะจับข้อผิดพลาดนั้นได้ทันทีที่เกิด

4.3.3 ผลการทดสอบกับผู้ใช้

[จากผู้เขียน] มีการให้เพื่อนห้าคนทดลองเล่น ผลที่ได้คือ **ผู้ทดลองเล่นทุกคนเข้าใจปริศนาองก์ที่หนึ่งได้ง่ายที่สุดในบรรดาปริศนาทั้งห้าองก์**

ปริศนาองก์ที่หนึ่งถูกออกแบบให้ผู้เล่นมองผนังด้านที่เหรียญเข็มทิศชี้ไป แล้วกวาดสายตาเรียงลำดับจากซ้ายไปขวาตามธรรมชาติของการอ่าน กลไกของเหรียญถูกตรวจสอบจากโปรแกรมแล้วว่าทำงานตามที่ออกแบบ — Assets/Scripts/World/CompassNeedle.cs:24-35 รับทิศในพิกัดของโลกแล้วฉายลงบนระนาบหน้าเหรียญทุกเฟรมใน LateUpdate ผลคือหยิบเหรียญเดินไปที่ใดหรือหมุนตัวอย่างไรก็ยังชี้ทิศเดิม ค่าที่ตั้งไว้ในฉากคือทิศ +x และเหรียญวางห่างผนังด้าน +x เพียง 1.39 เมตร ซึ่งเป็นผนังที่ใกล้ที่สุด

ข้อจำกัดของผลการทดสอบนี้ ที่ต้องระบุไว้

- จำนวนผู้ทดลองเล่นห้าคน เป็นกลุ่มตัวอย่างขนาดเล็กและไม่ได้สุ่ม

– **ไม่พบบันทึกการทดสอบในโปรเจกต์** ไม่มีแบบสอบถาม ไม่มีบันทึกเวลาที่ใช้แก้แต่ละองก์ และไม่มีวันที่ทำการทดสอบ ผลที่รายงานได้จึงเป็นคำบอกเล่าของผู้เขียนเพียงอย่างเดียว

– ข้อค้นพบที่ได้มีเพียงข้อเดียว คือองก์ที่หนึ่งเข้าใจง่ายที่สุด **ไม่มีข้อมูลว่าองก์อื่นมีปัญหาอย่างไรหรือไม่** ซึ่งเป็นช่องว่างที่สำคัญ เพราะโครงสร้างของเกมมีจุดเสี่ยงที่ระบุได้จากการวิเคราะห์ คือปริซึมสามชิ้นถูกเก็บได้ในสามห้องต่างกันแต่ใช้พร้อมกันในห้องสุดท้าย หากผู้เล่นพลาดเก็บชิ้นใดชิ้นหนึ่งจะติดอยู่ที่ห้องสุดท้ายโดยไม่มีคำใบ้ว่าต้องย้อนกลับไปห้องไหน — **ยังไม่มีข้อมูลว่าเกิดขึ้นกับผู้ทดลองเล่นหรือไม่**

– *Confidence:* กลไกของเหรียญ = ตรวจสอบแล้ว · ผลการทดลองเล่น = คำบอกเล่าของผู้เขียน ยืนยันจากโปรเจกต์ไม่ได้

4.3.4 ผลการตรวจสอบสถานะการคอมไพล์และข้อผิดพลาดที่พบ

ตรวจสอบสถานะของโปรเจกต์ผ่าน Unity MCP เมื่อวันที่ 6 พฤศจิกายน 2569 ผลแสดงในตารางที่ 4.14

ตารางที่ 4.14 สถานะของโปรเจกต์และข้อความที่พบใน Console

รายการ	ผล
ข้อผิดพลาดการคอมไพล์	**ไม่พบ**
ฉากที่เปิดอยู่	Assets/Scenes/Main.unity
รุ่นของ Unity Editor	6000.3.5f2
ข้อยกเว้นขณะทำงาน (exception)	**พบ 1 รายการ** — NullReferenceException
คำเตือน	พบ 1 ข้อความ ซ้ำหลายครั้ง — เรื่องความละเอียดของ shadow map

ข้อผิดพลาดที่หนึ่ง เป็นข้อบกพร่องที่ยังค้างอยู่ในโปรแกรม

```
NullReferenceException: Object reference not set to an instance
of an object
  DigitalReadout.SetDigit (System.Int32 place, System.Int32
value)  (at Assets/Scripts/Clock/DigitalReadout.cs:67)
  DigitalReadout.Show (System.Single minutes)
(at Assets/Scripts/Clock/DigitalReadout.cs:53)
  ClockMovement.ShowTime ()
(at Assets/Scripts/Clock/ClockMovement.cs:158)
  ClockMovement.Awake ()
(at Assets/Scripts/Clock/ClockMovement.cs:128)
```

สาเหตุที่วิเคราะห์ได้จากโปรแกรม มีดังนี้

1. ClockMovement.Awake() ค้นหาส่วนแสดงเวลาทุกตัวที่ประกอบอยู่ด้วย GetComponentsInChildren<IClockDisplay>(true) แล้วเรียก ShowTime() ทันทีในบรรทัดสุดท้ายของ Awake (ClockMovement.cs:128) เพื่อให้นาฬิกาแสดงเวลาตั้งต้นทันทีที่ฉากโหลด

2. DigitalReadout เตรียมอาเรย์ _faces ที่เก็บ material ของแต่ละหลักไว้ใน Awake() ของตัวเอง (DigitalReadout.cs:36-42)

3. ทั้งสองคลาสอยู่บน GameObject **ต่างกัน** — ClockMovement อยู่บน BenchMovement และ DigitalReadout อยู่บน TimingMachine ซึ่งเป็นวัตถุลูก (ตรวจจากไฟล์ Main.unity)

4. **Unity ไม่รับประกันลำดับการเรียก Awake ระหว่าง component ที่อยู่บน GameObject ต่างกัน** [14] และตรวจแล้วว่าโปรเจกต์นี้ **ไม่ได้ตั้งค่า Script Execution Order ไว้เลย** (ไม่มีไฟล์ ProjectSettings/MonoManager.asset และไม่พบรายการ ExecutionOrder ในโฟลเดอร์ ProjectSettings/ ทั้งโฟลเดอร์)

5. ดังนั้นหาก ClockMovement.Awake() ทำงานก่อน DigitalReadout.Awake() อาเรย์ _faces จะยังเป็น null ขณะที่ SetDigit เข้าถึง _faces[place] ที่บรรทัด 67 จึงเกิดข้อยกเว้น

ความสำคัญของข้อบกพร่องนี้ คือเป็น **ปัญหาชนิดเดียวกันกับที่โครงงานแก้ไปแล้วหนึ่งจุด** ในขั้นที่ 8 ปัญหาลำดับการทำงานระหว่าง TestRun กับ ClockMovement ถูกแก้โดยย้ายการนับไปไว้ใน LateUpdate ซึ่งรับประกันว่าทำงานหลัง Update ทุกตัว แต่ลำดับของ Awake ยังไม่ได้รับการจัดการด้วยวิธีใด จึงเป็นข้อบกพร่องประเภท **ลำดับการทำงานของวงจรชีวิตที่ไม่ถูกกำหนด** ที่เหลืออยู่

วัตถุที่ได้รับผลกระทบคือเครื่องจับเวลาบนโต๊ะช่าง ซึ่งเป็นวัตถุชิ้นเดียวในเกมที่มีส่วนแสดงเวลาสองชนิดบนกลไกเดียว และเป็นวัตถุที่ตั้งใจวางไว้เพื่อแสดงคุณสมบัติของส่วนต่อประสาน IClockDisplay โดยตรง

ข้อจำกัดของการรายงานข้อนี้: ข้อความใน Console ยืนยันว่าข้อผิดพลาดเกิดขึ้นจริง และกลไกที่ทำให้เกิดยืนยันได้จากตัวโปรแกรม แต่ **ยังยืนยันไม่ได้ว่าเกิดขึ้นทุกครั้งที่เปิดเกมหรือไม่** เพราะลำดับการเรียก Awake ที่ไม่ถูกกำหนดนั้นในทางปฏิบัติอาจคงที่บนเครื่องเดียวและเปลี่ยนเมื่อย้ายเครื่อง การยืนยันต้องกด Play ซึ่งอยู่นอกขอบเขตของการตรวจสอบนี้

วิธีแก้ที่เสนอ มีสองทาง ทางแรกคือให้ DigitalReadout เตรียม _faces แบบ lazy คือตรวจและเตรียมในครั้งแรกที่ Show ถูกเรียก ซึ่งแก้ที่ต้นเหตุและไม่ขึ้นกับลำดับ ทางที่สองคือตั้ง Script Execution Order ให้ DigitalReadout ทำงานก่อน ClockMovement ซึ่งแก้ได้แต่ผูกการทำงานของสองคลาสไว้กับการตั้งค่าในโปรเจกต์ที่มองจากโปรแกรมไม่เห็น จึงเสนอทางแรก

คำเตือนที่สอง เป็นผลที่ประเมินแล้วและยอมรับไว้ ไม่ใช่ข้อบกพร่อง

```
Reduced additional punctual light shadows resolution by 2 to make
24 shadow maps fit in the
4096x4096 shadow atlas. To avoid this, increase shadow atlas
size, decrease big shadow
resolutions, or reduce the number of shadow maps active in the
same frame
```

แหล่งกำเนิดแสงแบบจุดหนึ่งดวงต้องใช้ shadow map หกใบ ค่า 24 ใบจึงหมายถึงแสงแบบจุดที่สร้างเงาสี่ดวง ซึ่งตรงกับจำนวนที่กำหนดไว้ในโครงงาน URP แก้ปัญหาด้วยการลดความละเอียดลงครึ่งหนึ่งเองโดยไม่หยุดทำงาน และ 4096×4096 เป็นขนาด atlas ที่ใหญ่ที่สุดที่ URP รองรับ [19] คำเตือนนี้จึงทำให้หายไปไม่ได้นอกจากลดจำนวนแสงที่สร้างเงาลงเหลือสองดวง ซึ่งจะทำให้รูปทรงของห้องอ่านไม่ออก เป็น**ข้อแลกเปลี่ยนที่เลือกไว้** ไม่ใช่ปัญหาที่ยังไม่ได้แก้

[วางรูปที่ 4.15 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-4-15.png

รูปที่ 4.15 หน้าต่าง Console ของ Unity แสดงสถานะการคอมไพล์และข้อความที่พบ

4.4 สรุปผลและข้อจำกัดที่พบ

4.4.1 ผลการประเมินคุณภาพของโครงสร้างตามเกณฑ์

ประเมินตามเกณฑ์คุณภาพซอฟต์แวร์ที่อ้างในบทที่ 2 โดยเกณฑ์การพึ่งพาและความเกาะเกี่ยวใช้นิยามของ Stevens, Myers และ Constantine [2]

เกณฑ์การแยกส่วนใช้นิยามของ Parnas [1] และเกณฑ์การบำรุงรักษาและความสามารถในการทดสอบอ้างตาม ISO/IEC 25010 [4] ทุกช่องในตารางที่ 4.15 มีทั้งหลักฐานที่สนับสนุนและข้อจำกัดที่พบ เพื่อไม่ให้การประเมินเอนไปข้างใดข้างหนึ่ง

ตารางที่ 4.15 ผลการประเมินคุณภาพของโครงสร้างแปดเกณฑ์

เกณฑ์	ผลการประเมิน	หลักฐานที่สนับสนุน	ข้อจำกัดที่พบ
การแยกหน้าที่ (Separation of Concerns)	**ดี**	สถานะแยกจากการเคลื่อนไหว (Openable + SwingMotion) · เวลาแยกจากการแสดงเวลา (ClockMovement + IClockDisplay) · ตรรกะแยกจากเสียง (TestRun + RunTicking)	InspectView รวมแปดความรับผิดชอบไว้ในคลาสเดียว
ความเป็นโมดูล (Modularity)	**ดี–ดีมาก**	หกโมดูลมีขอบเขตบังคับระดับการคอมไพล์ · โมดูล Clock และ Puzzle อ้างถึง Player และ UI เป็นศูนย์ · โมดูล World แยกสมบูรณ์	สามโมดูลชั้นบนไม่มีขอบเขตบังคับเพราะมีวงจร · Puzzle.asmdef ประกาศเกินสองรายการ
การพึ่งพา (Coupling)	**ต่ำในชั้นล่าง · ปานกลางถึงสูงในชั้นบน**	สี่จุดเชื่อมใช้การค้นหาผ่านส่วนต่อประสาน · สัญญา IActor มีเพียงสามเมธอด · Singleton และการค้นหาวัตถุตอนทำงานเป็นศูนย์	การอ้างอิงผ่าน Inspector 215 จุดชี้ไปยังคลาสรูปธรรม · CodeLock ↔ CodeWheel และ Horologium ↔ Prism ผูกกันสองทาง · วงจร Player ⇄ UI
ความเกาะเกี่ยว (Cohesion)	**สูงเกือบทั้งหมด**	โปรแกรมส่วนที่ทำงานในเกม 63 ไฟล์ เฉลี่ย 55 บรรทัดต่อไฟล์ · ClockDial 32	InspectView 317 บรรทัด ใหญ่กว่าคลาสอื่นเกือบเท่าตัว

เกณฑ์	ผลการประเมิน	หลักฐานที่สนับสนุน	ข้อจำกัดที่พบ
		บรรทัดทำเรื่องเดียว · Pendulum 61 บรรทัด	
การนำกลับมาใช้ (Reusability)	**ดีมาก มีหลักฐานข้ามองก์**	ClockPartSocket ใช้ทั้งองก์ 4 และ 5 โดยไม่แก้โปรแกรม · ClockDial.prefab หนึ่งไฟล์ใช้ 46 เรือน · ClockRecipe สร้างนาฬิกาใหม่ได้โดยไม่เขียนโปรแกรม	นำกลับมาใช้ได้เฉพาะจุดเชื่อมที่ใช้การค้นหาผ่านส่วนต่อประสาน · ICodeInput ยังนำกลับมาใช้จริงไม่ได้
ความสามารถในการขยาย (Extensibility)	**ดีในมิติที่ออกแบบไว้**	ผ่านเจ็ดจากเก้าสถานการณ์ (ตารางที่ 4.11)	สองสถานการณ์ต้องแก้ไฟล์เดิม — ICodeInput และ enum สี่จุด
ความสามารถในการทดสอบ (Testability)	**ดีเฉพาะชั้นล่าง**	15 กรณีรันได้โดยไม่เปิดฉากและไม่ต้องมีผู้เล่น · RecordingActor สิบบรรทัดทำงานแทนผู้เล่นได้	ไม่มีชุดทดสอบครอบโมดูล Puzzle, Player, UI และ Game
ความสามารถในการบำรุงรักษา (Maintainability)	**ปานกลาง**	ไม่มีลำดับชั้นการสืบทอดให้ไล่ · คอมเมนต์อธิบายเหตุผลไม่ใช่สิ่งที่โปรแกรมทำ · มีเอกสารประกอบภายในโปรเจกต์รวม 174 กิโลไบต์	**ไม่มีระบบควบคุมรุ่นของซอร์สโค้ดเลย** · การพึ่งพา 215 จุดมองจากโปรแกรมไม่เห็น · นโยบายไม่ตรวจค่าว่างทำให้การลืมต่อสายหนึ่งเส้นกลายเป็นข้อผิดพลาดขณะเล่น · เอกสารประกอบไม่ตรงกับโปรแกรมสี่จุด

4.4.2 ข้อจำกัดและความเสี่ยงที่พบ

ตารางที่ 4.16 ข้อจำกัดที่พบ เรียงตามผลกระทบ

#	ข้อจำกัด	ผลกระทบ

#	ข้อจำกัด	ผลกระทบ
1	**ไม่มีระบบควบคุมรุ่นของซอร์สโค้ด** ตรวจแล้วไม่พบโฟลเดอร์ .git	การต่อสายใน Inspector ผิดหรือลบวัตถุพลาดย้อนกลับไม่ได้ และไม่มีประวัติการพัฒนาเป็นหลักฐาน
2	**การอ้างอิงผ่าน Inspector 215 จุด มองจากโปรแกรมไม่เห็น**	ต้องเปิด Unity จึงรู้ว่าอะไรต่อกับอะไร เมื่อรวมกับนโยบายไม่ตรวจค่าว่าง การลืมต่อหนึ่งเส้นทำให้เกมหยุดทำงานขณะเล่น
3	**NullReferenceException ที่ DigitalReadout.cs:67** จากลำดับการเรียก Awake ที่ไม่ถูกกำหนด (หัวข้อ 4.3.4)	เครื่องจับเวลาบนโต๊ะช่างอาจแสดงเวลาไม่ได้ และเป็นข้อบกพร่องที่ยังค้าง
4	**InspectView 317 บรรทัด รับผิดชอบแปดอย่าง**	ความเกาะเกี่ยวต่ำที่สุดในโครงงาน และเป็นจุดเดียวที่ขัดกับหลักหนึ่งคลาสหนึ่งหน้าที่ที่โครงงานตั้งไว้เอง
5	**ICodeInput ยังนำกลับมาใช้จริงไม่ได้**	คอมเมนต์ในโปรแกรมระบุคุณสมบัติที่โปรแกรมปัจจุบันยังไม่มี
6	**ไม่มีชุดทดสอบครอบโมดูล Puzzle, Player, UI และ Game**	โมดูล Puzzle เติมได้ทันที สามโมดูลหลังติดข้อจำกัดของโครง assembly
7	**ไม่มีการจัดการสถานะความก้าวหน้าแบบรวมศูนย์**	ไม่มีการบันทึกและโหลดเกม ไม่มีระบบคำใบ้ และไม่มีคลาสใดตอบได้ว่าผู้เล่นอยู่องก์ที่เท่าไร
8	**วงจรการพึ่งพา Player ⇄ UI**	ใส่ขอบเขตบังคับให้สามโมดูลชั้นบนไม่ได้ และทำให้ทดสอบไม่ได้ด้วย
9	**เอกสารประกอบภายในโปรเจกต์ไม่ตรงกับโปรแกรมสี่จุด**	ต้องอ้างจากไฟล์ ไม่ใช่จากเอกสารภายใน
10	**ภาพถ่ายหน้าจอทั้ง 58 ไฟล์เป็นสถานะก่อน 2 กันยายน**	ไม่มีภาพใดแสดงเกมในสถานะสุดท้าย ต้องถ่ายใหม่ทั้งหมด

สำหรับข้อที่ 9 รายละเอียดของทั้งสี่จุดแสดงในตารางที่ 4.17 เพื่อให้ผู้อ่านรายงานตรวจซ้ำได้ว่าตัวเลขใดในรายงานนี้มาจากที่ใด

ตารางที่ 4.17 จุดที่เอกสารประกอบภายในโปรเจกต์ไม่ตรงกับไฟล์จริง

#	เอกสารภายในโปรเจกต์ระบุว่า	ตรวจจากไฟล์จริงได้ว่า	ตัวเลขที่รายงานนี้ใช้
1	การพึ่งพาของโมดูล Puzzle คือ Interaction	ไฟล์ .asmdef ประกาศ Items, Interaction และ	ตารางที่ 4.6 และ 4.7

#	เอกสารภายในโปรเจกต์ระบุว่า	ตรวจจากไฟล์จริงได้ว่า	ตัวเลขที่รายงานนี้ใช้
		Clock แต่โปรแกรมใช้เพียง Interaction	
2	การพึ่งพาของโมดูล Player คือ Items, Interaction และ UI	โปรแกรมมีการพึ่งพา Player ⟶Clock เพิ่มอีกหนึ่งเส้น (HeldSandglass.cs:12)	ตารางที่ 4.6
3	นาฬิกาในฉาก 47 เรือน	นับ component ClockMovement ได้ **46** instance	**46**
4	จำนวน implementation 28 รายการ	นับได้ **29** **ความสัมพันธ์จาก 22** **คลาส** (หากไม่นับ PlayerInteractor : IActor จะได้ 28)	**29 / 22 คลาส** พร้อมระบุวิธีนับ

จุดที่ตรวจแล้วว่าเอกสารภายในโปรเจกต์ **ตรง** กับไฟล์จริง ได้แก่ จำนวนคลาส MonoBehaviour 51 คลาส จำนวนส่วนต่อประสาน 12 รายการ จำนวนชุดทดสอบ 15 กรณี การไม่มีคลาส abstract และการสืบทอด คำตอบของทั้งห้าองก์ และสภาพของนาฬิกาแต่ละเรือนที่ปริศนาใช้

4.4.3 ผลเทียบกับขอบเขตที่กำหนดไว้

ตารางที่ 4.18 ผลเทียบกับขอบเขตที่กำหนดไว้ในหัวข้อ 3.2.1

ขอบเขตที่กำหนดไว้	ผล	หลักฐาน
เดินด้วย WASD ไม่มีการกระโดด ย่อ หรือวิ่ง	**ทำได้**	PlayerMovement.cs มีเพียงการเดินและแรงโน้มถ่วง
ปุ่ม F ดูวัตถุที่ชี้อยู่	**ทำได้**	Inspectable รองรับสามรูปแบบคือ Hold, Turn และ Zoom
ปุ่ม E โต้ตอบ เก็บของ และเปิดกระเป๋า	**ทำได้**	PlayerInteractor ส่งปุ่ม E ให้ IInteractable ที่พบ
ปุ่ม Q ออกจากมุมมองย่อย	**ทำได้**	PlayerModeController สลับชุดการควบคุมสามชุด
ข้อความ "ไม่มีอะไรเกิดขึ้น"	**ทำได้**	InventoryUI.cs:134 ทำงานเมื่อ IItemReceiver.Give

ขอบเขตที่กำหนดไว้	ผล	หลักฐาน
เมื่อเลือกของผิด		คืนค่าเท็จ
ทำเกมทั้งเกม ไม่ใช่เพิ่มด่านในเกมที่มีอยู่	**ทำได้**	ห้าห้อง ห้าองก์ สร้างขึ้นทั้งหมดในโครงงาน
ออกแบบปริศนาเอง เพื่อให้เข้าเกณฑ์ Capstone Project	**ทำได้** [จากผู้เขียน]	ปริศนาทั้งห้าองก์และการจัดวางฉากเป็นงานออกแบบของผู้เขียน
ระยะเวลาหนึ่งสัปดาห์	**เกินกำหนด**	ใช้เวลา 15 วันตามปฏิทิน มีวันที่ทำงานจริง 10 วัน (หัวข้อ 3.1)

สิ่งที่อยู่นอกขอบเขตและไม่ได้ทำ ซึ่งระบุไว้แล้วในหัวข้อ 3.2.1 ได้แก่ การบันทึกและโหลดเกม ระบบคำใบ้ เสียงแบบสามมิติ การ build เป็นไฟล์สำหรับแจกจ่าย และการรองรับหลายภาษา ทั้งห้ารายการยืนยันแล้วว่าไม่มีอยู่ในโปรเจกต์ โดยไม่พบการเรียก PlayerPrefs, JsonUtility หรือการเขียนไฟล์ใด ๆ ไม่พบโฟลเดอร์ Build/ และข้อความทั้งหมดในเกมเป็นภาษาไทยที่ฝังอยู่ในโปรแกรมและในค่าที่ตั้งผ่าน Inspector

บทที่ 5
บทสรุปและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

จากการที่ข้าพเจ้าได้ออกปฏิบัติงานสหกิจศึกษา ณ บริษัท Lumas corp ในตำแหน่ง Game Developer ข้าพเจ้าได้รับมอบหมายโจทย์จากพนักงานที่ปรึกษาให้พัฒนาเกมแนว escape room มุมมองบุคคลที่หนึ่งขึ้นมาทั้งเกม โดยโจทย์กำหนดรูปแบบการควบคุมไว้ให้แล้ว คือเดินด้วยปุ่ม WASD โดยไม่มีการกระโดด ย่อ หรือวิ่ง ใช้ปุ่ม F เพื่อดูวัตถุที่ชี้อยู่ ปุ่ม E เพื่อโต้ตอบและเก็บสิ่งของ และปุ่ม Q เพื่อออกจากมุมมองย่อย ส่วนการออกแบบปริศนาและการจัดวางฉากทั้งหมดเป็นงานที่ข้าพเจ้าออกแบบขึ้นเอง ซึ่งเป็นเงื่อนไขที่ทำให้โครงงานนี้เข้าเกณฑ์ Capstone Project ตามที่ระบุไว้ในโจทย์

ผลการดำเนินงานคือเกมที่เล่นจบได้หนึ่งรอบ ประกอบด้วยห้องห้าห้องเรียงต่อกันเป็นเส้นทางเดียว และปริศนาห้าองก์ที่ต้องแก้เรียงตามลำดับ โดยแต่ละองก์เปิดทางไปสู่องก์ถัดไป ขนาดของงานที่วัดได้คือโปรแกรมภาษา C# จำนวน 78 ไฟล์ รวม 10,604 บรรทัด (นับบรรทัดที่ไม่ว่าง) แบ่งเป็นโปรแกรมส่วนที่ทำงานในเกม 3,481 บรรทัด เครื่องมือสร้างเนื้อหาสำหรับตัวแก้ไข 5,772 บรรทัด และชุดทดสอบ 393 บรรทัด พร้อมด้วย prefab 33 ไฟล์ material 115 ไฟล์ และนาฬิกาในฉากรวม 46 เรือน

สิ่งที่ข้าพเจ้าถือว่าเป็นผลสำคัญที่สุดของโครงงานนี้ไม่ใช่ตัวเกม แต่เป็น **โครงสร้างของซอฟต์แวร์ที่ทำให้เพิ่มสิ่งใหม่ได้โดยไม่ต้องแก้โปรแกรมเดิม** ซึ่งมีผลการวัดรองรับ กล่าวคือโครงงานนี้แบ่งโปรแกรมเป็นเก้าโมดูลตามหน้าที่ โดยหกโมดูลมีขอบเขตบังคับในระดับการคอมไพล์ด้วยไฟล์ Assembly Definition [23] ใช้การประกอบ component แทนการสืบทอดคลาสอย่างสมบูรณ์ — คำสั่ง `abstract`, `virtual` และ `override` เป็นศูนย์ทั้งโครงงาน ขณะที่มีคลาส 51 คลาสและส่วนต่อประสาน 12 รายการ — และใช้การค้นหาผู้ร่วมงานผ่านส่วนต่อประสานในจุดเชื่อมสำคัญสี่จุด ผลที่วัดได้คือการประเมินตามหลัก Open–Closed Principle [8] ผ่านเจ็ดจากเก้าสถานการณ์ และโมดูลที่ทำหน้าที่เรื่องนาฬิกาและปริศนาไม่อ้างถึงคลาสของผู้เล่นแม้แต่ครั้งเดียว จึงคอมไพล์และทดสอบได้โดยไม่ต้องมีผู้เล่นอยู่ในระบบ ซึ่งชุดทดสอบ 15 กรณีของโครงงานนี้ใช้คุณสมบัตินั้นจริง

> **[หมายเหตุของร่าง — ลบก่อนส่ง]** **[ต้องให้ผู้เขียนเขียนเอง]**
> ย่อหน้าสุดท้ายของหัวข้อนี้ควรเป็นความรู้และทักษะที่ได้รับจากการปฏิบัติงาน

> ซึ่งเป็นเรื่องที่อ่านจากโปรเจกต์ไม่ได้ ตัวอย่างรูปเล่มของรุ่นก่อนเขียนไว้ประมาณห้าถึงเจ็ดบรรทัด
> ความรู้ทางเทคนิคที่โครงงานนี้แสดงว่าได้ใช้จริงและใช้อ้างได้มีดังนี้ —
> การออกแบบซอฟต์แวร์เชิงโมดูลและการวัดการพึ่งพาระหว่างโมดูล ·
> การใช้ส่วนต่อประสานเพื่อกลับทิศทางการพึ่งพา · การทดสอบซอฟต์แวร์ด้วย Unity Test Framework และ NUnit · ความถูกต้องของการคำนวณเลขทศนิยมและความเป็นอิสระจากอัตราเฟรม ·
> การสร้างเนื้อหาด้วยโปรแกรม (procedural content generation) · การทำงานกับ Unity Editor ผ่านสคริปต์ ส่วนทักษะที่ไม่เกี่ยวกับเทคนิค เช่น การวางแผนงาน การสื่อสารกับพนักงานที่ปรึกษา หรือการทำเอกสาร **ผู้เขียนต้องเขียนเองทั้งหมด ห้ามให้ผู้ช่วยเดา**

5.2 ปัญหาและแนวทางการแก้ไข

หัวข้อนี้รายงานปัญหาทางวิศวกรรมสามเรื่องที่พบระหว่างและหลังการพัฒนา โดยสองเรื่องแรกแก้ไขแล้ว และเรื่องที่สามยังค้างอยู่ ปัญหาที่เกี่ยวกับการใช้เครื่องมือหรือเรื่องที่ไม่เกี่ยวกับหัวข้อของโครงงานไม่ถูกนำมาเขียนไว้ที่นี่

5.2.1 การนับเวลาของนาฬิกาผูกกับความเร็วของเครื่องที่ใช้เล่น

อาการของปัญหา [จากผู้เขียน]

โปรแกรมชุดแรกนับเวลาโดยอ้างกับรอบการทำงานของหน่วยประมวลผล ผลคือเมื่อนำไปเล่นบนเครื่องที่ต่างกัน **เครื่องที่เร็วทำให้นาฬิกาในเกมเดินเร็ว และเครื่องที่ช้าทำให้นาฬิกาเดินช้า** ทั้งที่นาฬิกาทรายในเกมควรจับเวลาได้สิบวินาทีเท่ากันทุกเครื่อง

ความสำคัญของปัญหานี้ต่อตัวเกม ปัญหานี้ไม่ใช่เพียงเรื่องความสวยงาม แต่ทำให้ **ปริศนาองก์ที่สองใช้เล่นไม่ได้** เพราะกลไกของปริศนานั้นคือการจับเวลานาฬิกาพกสิบสองเรือนเทียบกับนาฬิกาทรายสิบวินาที แล้วหาเรือนที่เดินช้ากว่าเรือนอื่น หากทั้งนาฬิกาพกและนาฬิกาทรายเดินตามความเร็วของเครื่อง การเปรียบเทียบจะไม่ให้ผลที่คงที่ และคำตอบของปริศนาจะเปลี่ยนไปตามเครื่องที่ใช้เล่น ซึ่งหมายความว่าผู้เล่นบางคนจะหาคำตอบไม่ได้เลย

สาเหตุเชิงหลักการ

การนับเวลาโดยอ้างกับรอบการทำงานของเครื่องทำให้ปริมาณที่เพิ่มต่อหนึ่งเฟรมเป็นค่าคงที่ ขณะที่จำนวนเฟรมต่อวินาทีเปลี่ยนไปตามเครื่อง ผลคือเวลาที่นับได้ต่อหนึ่งวินาทีจริงแปรผันตามอัตราเฟรม การแก้ที่ถูกต้องคือให้ปริมาณที่เพิ่มต่อหนึ่งเฟรมเป็นสัดส่วนกับ **เวลาจริงที่เฟรมนั้นใช้ไป** ซึ่งใน Unity คือค่า `Time.deltaTime` [13]

แนวทางการแก้ไขที่ใช้ แก้ด้วยกลไกสี่อย่างที่แยกจากกัน และแต่ละอย่างแก้ปัญหาต่างข้อ ดังตารางที่ 5.1

ตารางที่ 5.1 กลไกสี่อย่างที่ใช้แก้ปัญหาความถูกต้องของการนับเวลา

#	กลไก	หลักฐานในโปรแกรม	แก้ปัญหาข้อใด
1	การนับเวลาทุกจุดใช้ `Time.deltaTime` ซึ่งเป็นเวลาจริงของเฟรมนั้น	`ClockMovement.cs:136`, `Sandglass.cs:107`, `TestRun.cs:77` · ค้นทั้งโปรเจกต์ไม่พบการใช้ `Time.frameCount` เลย	อัตราเฟรมต่างกันแต่เวลาที่นับได้เท่ากัน
2	สะสมเวลาในชนิดข้อมูล `double` ไม่ใช่ `float`	`ClockMovement.cs:50` พร้อมคำอธิบายที่บรรทัด 42–49	ความคลาดเคลื่อนสะสมจากการบวกเลขทศนิยมที่มีขนาดต่างกันมาก
3	เฟรมสุดท้ายของการจับเวลาคิดเป็นเศษส่วนของเฟรม	`TestRun.cs:52` → `Mathf.Clamp01((Length - _ran) / Time.deltaTime)`	เข็มหยุดตรงขีดเดิมเสมอ ไม่เลยขีดไปตามความยาวของเฟรมที่คร่อมเส้นชัย
4	ตัวจับเวลานับใน `LateUpdate` ส่วนนาฬิกาเดินใน `Update`	`TestRun.cs:72-81` พร้อมคำอธิบายที่บรรทัด 68–71	ลำดับการทำงานของ `Update` ระหว่าง component ที่ Unity ไม่กำหนดไว้

กลไกที่สองเป็นข้อที่ลึกที่สุดและไม่ใช่เรื่องของอัตราเฟรมโดยตรง แต่เป็นเรื่องความแม่นยำของเลขทศนิยม คำอธิบายที่บันทึกไว้ในโปรแกรมระบุสาเหตุไว้ชัดเจน คือนาฬิกานับเวลาด้วยการบวกค่าเล็กมากของหนึ่งเฟรมเข้ากับยอดสะสมที่โตขึ้นเรื่อย ๆ และเมื่อสองจำนวนนั้นมีขนาดต่างกันมาก การบวกในชนิด `float` จะปัดเศษของตัวที่เล็กกว่าทิ้ง

ตัวอย่างที่บันทึกไว้คือ ที่เวลา 4:40 ยอดสะสมมีค่า 280 นาที ซึ่งในชนิด float ระยะห่างระหว่างค่าที่แทนได้สองค่าที่ติดกันคือประมาณ 0.00003 นาที ขณะที่นาฬิกาที่เดินช้าเพิ่มค่าเพียง 0.00017 นาทีต่อเฟรม ผลคือราวหนึ่งในห้าของทุกก้าวถูกปัดทิ้ง และ **ปัดทิ้งไม่เท่ากันในนาฬิกาแต่ละเรือน เพราะขนาดของระยะห่างนั้นขึ้นกับเวลาที่นาฬิกาเรือนนั้นกำลังแสดง** ปรากฏการณ์นี้ตรงกับที่อธิบายไว้ในทฤษฎีความแม่นยำของเลขทศนิยม [15] และเป็นผลโดยตรงของการที่มาตรฐาน IEEE 754 กำหนดให้ระยะห่างระหว่างค่าที่แทนได้แปรตามขนาดของเลข [16]

เมื่อเปลี่ยนไปสะสมในชนิด double ระยะห่างดังกล่าวลดลงเหลือราว 6×10^{-14} นาที ซึ่งเล็กกว่าค่าที่เพิ่มต่อเฟรมหลายอันดับขนาด ปัญหาจึงหมดไป ทั้งนี้ **ส่วนที่เปิดให้ภายนอกเรียกยังใช้ float อยู่** เพราะการแปลงชนิดหนึ่งครั้งที่ขอบของคลาสทำให้คลาดเคลื่อนเพียง 0.01 องศาและเกิดเพียงครั้งเดียว สิ่งที่ทำในชนิด float ไม่ได้คือ *การสะสม* ไม่ใช่การส่งค่าออก

กลไกที่สี่เป็นปัญหาเรื่องลำดับการทำงาน ซึ่งเดิมทั้งตัวจับเวลาและตัวนาฬิกานับอยู่ใน Update ทั้งคู่ และ Unity ไม่กำหนดลำดับการเรียก Update ระหว่าง component [14] ผลคือตัวใดทำงานก่อนจะเป็นผู้ตัดสินว่าเฟรมสุดท้ายของการจับเวลาถูกนับหรือถูกข้าม การย้ายตัวจับเวลาไปไว้ใน LateUpdate ซึ่งรับประกันว่าทำงานหลัง Update ทุกตัวเสมอ แก้ปัญหานี้ได้โดยไม่ต้องตั้งค่า Script Execution Order ในโปรเจกต์

[วางรูปที่ 5.1 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-5-01.png

รูปที่ 5.1 ส่วนของโปรแกรมที่แก้ปัญหาการนับเวลา คือการประกาศตัวแปรสะสมเป็น double และการคิดเศษของเฟรมสุดท้าย

ผลหลังแก้ไข คำอธิบายที่บันทึกไว้ในโปรแกรมระบุผลการวัดไว้ว่า นาฬิกาสิบสองเรือนที่เหมือนกันทุกเรือนและถูกจับเวลาด้วยเงื่อนไขเดียวกัน

เดิมเข็มหยุดกระจายอยู่ระหว่าง 59.3 ถึง 61.3 องศา และหลังแก้ไขทุกเรือนหยุดที่ 60.00 องศา ส่วนเรือนที่ตั้งให้เดินช้าหยุดที่ 60.00 องศาเมื่อผ่านไป 25.01 วินาที

> **ข้อจำกัดของหลักฐานที่ต้องระบุ** โครงงานนี้ไม่มีระบบควบคุมรุ่นของซอร์สโค้ด จึงเห็นเฉพาะโปรแกรมชุดที่แก้แล้ว **ไม่สามารถยืนยันรูปแบบของโปรแกรมชุดเดิมได้** อาการของปัญหาจึงเขียนตามคำบอกเล่าของผู้เขียน และใช้โปรแกรมปัจจุบันเป็นหลักฐานของ *วิธีแก้* เท่านั้น ส่วนตัวเลข 59.3–61.3 องศาและ 60.00 องศาเป็น **ผลการวัดที่ผู้เขียนบันทึกไว้ในคำอธิบายของโปรแกรมเอง** ไม่ใช่ผลที่วัดซ้ำในการจัดทำรายงานนี้ หากต้องการให้ตัวเลขนี้เป็นผลที่ตรวจสอบได้ ต้องวัดซ้ำและแคปผล

5.2.2 การพึ่งพาระหว่างโมดูลปริศนากับคลาสของผู้เล่น

อาการของปัญหา วัตถุที่ผู้เล่นโต้ตอบได้ทุกชิ้นในเกม — นาฬิกา ลิ้นชัก วงล้อรหัส — เดิมรับคลาส `PlayerInteractor` ซึ่งเป็นคลาสของผู้เล่นทั้งคลาสเข้ามาเป็นพารามิเตอร์ ผลคือโมดูลที่ทำหน้าที่เรื่องนาฬิกาและปริศนาถูกคอมไพล์ผูกกับคลาสของผู้เล่นของเกมนี้โดยตรง ทั้งที่สิ่งที่วัตถุเหล่านั้นต้องการจากผู้เล่นมีเพียงสามอย่าง

> *แหล่งของอาการ:* คำอธิบายที่บันทึกไว้ในไฟล์ Assets/Scripts/Interaction/IActor.cs:3-6 ระบุว่า *"Every prop in the house used to take a PlayerInteractor"* ซึ่งเป็น **เอกสารที่ผู้เขียนบันทึกไว้ในโปรแกรมเอง** ไม่ใช่ประวัติการแก้ไขที่ตรวจสอบได้ เพราะไม่มีระบบควบคุมรุ่น

ผลเสียของการพึ่งพาแบบนั้น มีสามข้อ ข้อแรกคือโมดูลปริศนาและนาฬิกาจะนำไปใช้ที่อื่นไม่ได้เลย เพราะต้องลากคลาสผู้เล่นของเกมนี้ไปด้วย ข้อที่สองคือการแก้คลาสผู้เล่นจะกระทบการคอมไพล์ของทุกโมดูลที่อ้างถึงมัน และข้อที่สามซึ่งเป็นข้อที่วัดผลได้ชัดที่สุดคือ **เขียนชุดทดสอบไม่ได้** เพราะการทดสอบกลอนประตูหนึ่งบานจะต้องสร้างผู้เล่นทั้งตัวขึ้นมาในฉากก่อน

แนวทางการแก้ไขที่ใช้ สร้างส่วนต่อประสานชื่อ `IActor` ที่มีเพียงสามเมธอด แล้วเปลี่ยนให้วัตถุทุกชิ้นรับส่วนต่อประสานนี้แทนคลาสของผู้เล่น

```
// Assets/Scripts/Interaction/IActor.cs:15-26
public interface IActor
{
    void Say(string message);                  // บอกข้อความกับผู้ที่กำลังทำงานกับวัตถุนี้
    void OfferBag(IItemReceiver to);           // เสนอกระเป๋าให้เลือกของมาใช้กับ to
    void Take(ItemData item);                  // ส่งของให้ผู้นั้นถือไป
}
```

จุดที่สำคัญคือ **ส่วนต่อประสานนี้ถูกวางไว้ในโมดูล Interaction ไม่ใช่โมดูล Player** เพราะโมดูลนาฬิกาและปริศนาอ้างถึงโมดูล Interaction อยู่แล้ว การวางไว้ที่นี่จึงไม่สร้างการพึ่งพาเส้นใหม่ ขณะที่หากวางไว้กับผู้เล่น ปัญหาเดิมจะกลับมาในชื่ออื่น ส่วนคลาส PlayerInteractor ก็ implement ส่วนต่อประสานนี้ด้วยการส่งต่อสามบรรทัด

```
// Assets/Scripts/Player/PlayerInteractor.cs:105-109
public void Say(string message) => hud.ShowMessage(message);
public void OfferBag(IItemReceiver to) => inventoryUI.Open(to);
public void Take(ItemData item) => Inventory.Add(item);
```

หลังจากนั้นได้เพิ่มไฟล์ Assembly Definition ห้าไฟล์ เพื่อ **บังคับไม่ให้การพึ่งพาแบบเดิมกลับมาได้อีกในระดับการคอมไพล์** [23] กล่าวคือการเขียนโปรแกรมในโมดูลนาฬิกาให้อ้างถึงคลาสของผู้เล่นจะคอมไพล์ไม่ผ่านทันที ไม่ใช่เพียงผิดธรรมเนียมที่ตกลงกันไว้

ผลหลังแก้ไข วัดได้สามข้อ

ผลที่วัดได้	ค่า	วิธีวัด
ไฟล์ที่รับ IActor เป็นพารามิเตอร์	17 จุด ใน 18 ไฟล์	ค้นข้อความ IActor actor ในโฟลเดอร์ Scripts/
การอ้างถึง PlayerInteractor ในโฟลเดอร์ Clock/ และ Puzzle/	**0 ครั้ง**	ค้นข้อความในสองโฟลเดอร์
จำนวนบรรทัดที่ PlayerInteractor ต้องเขียนเพื่อรองรับสัญญานี้	**3 บรรทัด**	PlayerInteractor.cs:105-109

และผลที่เป็นรูปธรรมที่สุดคือ **ชุดทดสอบหกกรณีในไฟล์ ActorSeamTests.cs ใช้คลาส C# ธรรมดาสิบบรรทัดที่ไม่ใช่ MonoBehaviour และไม่อยู่ในฉากใด ทำงานแทนผู้เล่นได้ครบ** คือเปิดกลอนประตู ใส่ชิ้นส่วนนาฬิกา และรับสิ่งของ ซึ่งเป็นสิ่งที่ทำไม่ได้เลยก่อนการแก้ไข นี่คือหลักฐานว่าการกลับทิศทางการพึ่งพาเกิดขึ้นจริง ไม่ใช่เพียงการเปลี่ยนชื่อชนิดข้อมูล

ลำดับเวลาของการแก้ไขครั้งนี้ยังให้ข้อสังเกตเชิงกระบวนการที่น่าสนใจ จากเวลาของไฟล์ในระบบ (หัวข้อ 3.3.3 และ 3.3.4) การตัดการพึ่งพาเกิดขึ้นก่อนการเพิ่มไฟล์ Assembly Definition ประมาณสามชั่วโมงครึ่ง และหลังการเพิ่มไฟล์เหล่านั้นไม่มีไฟล์โปรแกรมใดถูกแก้อีกเลยในช่วงเวลาถัดมา ซึ่ง**ตีความได้ว่า**ไฟล์ Assembly Definition ถูกใช้เป็นกลอนล็อกโครงสร้างหลังจากจัดการพึ่งพาให้สะอาดแล้ว ไม่ได้ใช้เป็นเครื่องมือค้นหาปัญหา

5.2.3 ลำดับการเรียก Awake ที่ไม่ถูกกำหนด — ปัญหาที่ยังค้างอยู่

อาการของปัญหา ตรวจสอบ Unity Console เมื่อวันที่ 6 พฤศจิกายน 2569 พบข้อยกเว้นหนึ่งรายการ

```
NullReferenceException: Object reference not set to an instance
of an object
  DigitalReadout.SetDigit (System.Int32 place, System.Int32
value)  (at Assets/Scripts/Clock/DigitalReadout.cs:67)
  DigitalReadout.Show (System.Single minutes)
(at Assets/Scripts/Clock/DigitalReadout.cs:53)
  ClockMovement.ShowTime ()
(at Assets/Scripts/Clock/ClockMovement.cs:158)
  ClockMovement.Awake ()
(at Assets/Scripts/Clock/ClockMovement.cs:128)
```

สาเหตุ วิเคราะห์จากโปรแกรมและจากฉากได้ครบทุกขั้น ดังนี้

1. ClockMovement.Awake() ค้นหาส่วนแสดงเวลาทุกตัวที่ประกอบอยู่ แล้วเรียกให้แสดงเวลาตั้งต้นทันทีในบรรทัดสุดท้ายของ Awake
2. คลาส DigitalReadout เตรียมอาเรย์ที่เก็บ material ของแต่ละหลักไว้ใน Awake() ของตัวเอง ก่อนหน้านั้นอาเรย์ยังเป็นค่าว่าง
3. สองคลาสนี้อยู่บน GameObject **ต่างกัน** คือ ClockMovement อยู่บน BenchMovement และ DigitalReadout อยู่บน TimingMachine ซึ่งเป็นวัตถุลูก
4. **Unity ไม่รับประกันลำดับการเรียก Awake ระหว่าง component ที่อยู่บน GameObject ต่างกัน** [14] และตรวจแล้วว่าโครงงานนี้ไม่ได้ตั้งค่า Script Execution Order ไว้เลย คือไม่มีไฟล์ ProjectSettings/MonoManager.asset และไม่พบรายการ ExecutionOrder ในโฟลเดอร์ ProjectSettings/ ทั้งโฟลเดอร์
5. ดังนั้นหาก ClockMovement.Awake() ทำงานก่อน อาเรย์จะยังว่างขณะถูกเข้าถึง จึงเกิดข้อยกเว้น

[วางรูปที่ 5.2 ที่นี่]
ชื่อไฟล์ที่เตรียมไว้ fig-5-02.png

รูปที่ 5.2 ข้อความข้อยกเว้นและลำดับการเรียกที่ปรากฏใน Unity Console

ข้อสังเกตที่สำคัญที่สุดของปัญหานี้ คือ **เป็นปัญหาชนิดเดียวกับที่โครงงานแก้ไปแล้วหนึ่งจุดในหัวข้อ 5.2.1 กลไกที่สี่** กล่าวคือปัญหาลำดับการทำงานระหว่าง component ที่เอนจินไม่กำหนดไว้ ต่างกันที่จุดนั้นเป็นลำดับของ Update และแก้ได้ด้วยการย้ายไป LateUpdate ขณะที่จุดนี้เป็นลำดับของ Awake ซึ่งไม่มีเมธอดที่รับประกันว่าทำงานหลังสุดให้ย้ายไป จึงต้องแก้ด้วยวิธีอื่น

วัตถุที่ได้รับผลกระทบ คือเครื่องจับเวลาบนโต๊ะช่าง ซึ่งเป็นวัตถุชิ้นเดียวในเกมที่มีส่วนแสดงเวลาสองชนิดบนกลไกเดียว และเป็นวัตถุที่ตั้งใจวางไว้เพื่อแสดงคุณสมบัติของส่วนต่อประสาน IClockDisplay โดยตรง

ข้อจำกัดของการรายงาน ข้อความใน Console ยืนยันว่าข้อผิดพลาดเกิดขึ้นจริง และกลไกที่ทำให้เกิดยืนยันได้จากโปรแกรม แต่ **ยังยืนยันไม่ได้ว่าเกิดทุกครั้งที่เปิดเกมหรือไม่** เพราะลำดับที่เอนจินไม่กำหนดไว้นั้นในทางปฏิบัติอาจคงที่บนเครื่องหนึ่งและเปลี่ยนเมื่อย้ายเครื่อง การยืนยันต้องเคลียร์ Console แล้วเข้าสู่โหมดเล่นหนึ่งครั้ง

แนวทางการแก้ไขที่เสนอ เสนอให้ DigitalReadout เตรียมอาเรย์แบบ lazy คือตรวจและเตรียมในครั้งแรกที่ถูกสั่งให้แสดงเวลา ซึ่งแก้ที่ต้นเหตุและไม่ขึ้นกับลำดับการเรียกเลย ส่วนทางเลือกที่สองคือการตั้งค่า Script Execution Order ให้ DigitalReadout ทำงานก่อน ClockMovement นั้นแก้อาการได้ แต่ผูกการทำงานของสองคลาสไว้กับค่าตั้งในโปรเจกต์ที่มองจากโปรแกรมไม่เห็น ซึ่งขัดกับแนวทางที่โครงงานนี้ใช้มาตลอด จึงเสนอทางแรก

5.3 ข้อเสนอแนะจากการดำเนินโครงงาน

จากการดำเนินโครงงานสหกิจศึกษา มีข้อเสนอแนะดังนี้

1) **ควรใช้ระบบควบคุมรุ่นของซอร์สโค้ดตั้งแต่วันแรกของโครงงาน** ตรวจแล้วว่าโครงงานนี้ไม่มีระบบควบคุมรุ่นเลย ซึ่งเป็นข้อจำกัดที่ส่งผลกว้างที่สุดและกระทบทั้งตัวงานและการจัดทำรายงาน ในด้านตัวงาน โครงงานมีการอ้างอิงที่ตั้งค่าผ่าน Inspector ถึง 215 จุด และโครงงานใช้นโยบายไม่ตรวจค่าว่างโดยเจตนา การต่อสายผิดหรือลบวัตถุพลาดหนึ่งครั้งจึงทำให้เกมหยุดทำงานโดยไม่มีทางย้อนกลับ ในด้านรายงาน การไม่มีประวัติการแก้ไขทำให้ยืนยันรูปแบบของโปรแกรมชุดก่อนการแก้ไขไม่ได้เลย ปัญหาในหัวข้อ

5.2.1 และ 5.2.2 จึงต้องเขียนอาการตามคำบอกเล่าและใช้โปรแกรมปัจจุบันเป็นหลักฐานของวิธีแก้เท่านั้น ซึ่งเป็นหลักฐานที่อ่อนกว่าที่ควรเป็น

2) ควรเขียนชุดทดสอบครอบโมดูล Puzzle โมดูลนี้เป็นโมดูลเดียวที่**ไม่มีข้อจำกัดทางเทคนิคขัดขวางเลย** คือไฟล์ Assembly Definition ของชุดทดสอบประกาศให้อ้างถึงโมดูลนี้ได้อยู่แล้ว แต่ยังไม่มีกรณีทดสอบใดใช้ชนิดข้อมูลจากโมดูลนี้ กรณีทดสอบที่ควรเพิ่มคือตรวจว่า `CodeLock` ปลดเมื่ออุปกรณ์ใส่รหัสทุกตัวตรงกับรหัสที่ตั้งไว้ ตรวจว่า `StampedCombination` ประกอบรหัสจาก `StampedNumber` ได้ถูกลำดับ และตรวจว่า `Horologium` คืนรูปแบบเดิมเมื่อปริซึมทุกชิ้นอยู่ที่คำตอบ

3) ควรแก้ส่วนต่อประสาน `ICodeInput` ให้นำกลับมาใช้ได้จริง ปัจจุบันฟิลด์ที่รับอุปกรณ์ใส่รหัสใน `CodeLock` ประกาศเป็นชนิด `CodeWheel[]` ซึ่งเป็นคลาสรูปธรรม ทำให้คลาสใหม่ที่ implement `ICodeInput` ใส่ลงไปไม่ได้โดยไม่แก้โปรแกรมและคำอธิบายในโปรแกรมที่ระบุว่าเปลี่ยนอุปกรณ์ใส่รหัสได้โดยไม่แก้โปรแกรมนั้นยังไม่เป็นจริง **วิธีแก้มีอยู่ในโครงงานนี้แล้ว** คือใช้การค้นหาด้วย `GetComponentsInChildren<ICodeInput>()` แบบเดียวกับที่โมดูล Clock ใช้กับ `IClockDisplay` ซึ่งเป็นแนวทางที่พิสูจน์แล้วด้วยชุดทดสอบว่าใช้ได้ การแก้จุดนี้ทำให้หลักการออกแบบของโครงงานถูกใช้อย่างทั่วถึง ไม่ใช่ใช้เฉพาะในโมดูลเดียว

4) ควรแก้ข้อบกพร่องเรื่องลำดับการเรียก `Awake` ตามที่เสนอในหัวข้อ 5.2.3 เพราะเป็นข้อบกพร่องที่ยังค้างอยู่
และกระทบวัตถุที่ตั้งใจวางไว้เพื่อแสดงคุณสมบัติสำคัญของโครงสร้าง

5) ควรตัดรายการอ้างอิงที่ไม่ได้ใช้ออกจากไฟล์ Assembly Definition ตรวจพบว่า `EscapeRoom.Puzzle.asmdef` ประกาศให้อ้างถึงสองโมดูลที่โปรแกรมไม่ได้ใช้ และ `EscapeRoom.Tests.asmdef` ประกาศเกินอีกหนึ่งรายการ ซึ่งทำให้ขอบเขตโมดูลหลวมกว่าที่โปรแกรมต้องการจริง
และทำให้การพึ่งพาที่ไม่ควรเกิดสามารถเกิดได้โดยคอมไพเลอร์ไม่แจ้งเตือน
การตัดรายการเหล่านี้ออกใช้เวลาไม่กี่นาที ไม่ต้องแก้โปรแกรมเลย และทำให้ไฟล์ Assembly Definition เป็นเอกสารที่บังคับใช้ได้ของการพึ่งพาที่เกิดขึ้นจริง ซึ่งเป็นเหตุผลที่เลือกใช้เครื่องมือนี้ตั้งแต่ต้น

6) ควรแยกความรับผิดชอบของคลาส `InspectView` คลาสนี้มี 317 บรรทัดและรับผิดชอบงานแปดอย่าง เป็นคลาสที่มีความเกาะเกี่ยวต่ำที่สุดในโครงงาน และเป็นจุดเดียวที่ขัดกับหลักหนึ่งคลาสหนึ่งหน้าที่ที่โครงงานตั้งไว้เอง
ส่วนที่แยกออกได้สะอาดที่สุดคือการคำนวณตำแหน่งและระยะของวัตถุที่ยกขึ้นมา ซึ่งเป็น static method อยู่แล้วสามตัว และส่วนที่จัดการนาฬิกาทราย ซึ่งเกี่ยวกับปริศนาองก์ที่สองเท่านั้น

7) ควรตัดวงจรการพึ่งพาระหว่างโมดูล Player และ UI
วงจรนี้เป็นเหตุทางเทคนิคที่ทำให้ใส่ขอบเขตบังคับให้สามโมดูลชั้นบนไม่ได้
และทำให้เขียนชุดทดสอบครอบสามโมดูลนั้นไม่ได้ด้วย เพราะกฎของ Unity ห้าม Assembly Definition อ้างถึง predefined assembly [23] การตัดวงจรทำได้โดยให้โมดูล UI ไม่รู้จักโมดูล Player เลย แล้วให้โมดูล Player ป้อนข้อมูลที่จำเป็นเข้าไปผ่านส่วนต่อประสาน ซึ่งเป็นวิธีเดียวกับที่โครงงานนี้ใช้แก้ปัญหาในหัวข้อ 5.2.2 สำเร็จมาแล้ว ข้อนี้เป็นงานที่ใหญ่ที่สุดในรายการและควรทำหลังข้ออื่น

8) ควรเพิ่มการจัดการสถานะความก้าวหน้าของผู้เล่น
ปัจจุบันความก้าวหน้ากระจายเป็นตัวแปรอยู่ในกลอนแต่ละตัว
ไม่มีคลาสใดตอบได้ว่าผู้เล่นอยู่องก์ที่เท่าไร ผลคือเกมไม่มีการบันทึกและโหลด และไม่สามารถทำระบบคำใบ้ได้ ซึ่งมีผลต่อการเล่นจริงเพราะโครงสร้างของเกมมีจุดเสี่ยงอยู่ คือปริซึมสามชิ้นเก็บได้ในสามห้องต่างกันแต่ใช้พร้อมกันในห้องสุดท้าย
หากผู้เล่นพลาดเก็บชิ้นใดชิ้นหนึ่งจะติดอยู่ที่ห้องสุดท้ายโดยไม่มีคำใบ้ว่าต้องย้อนกลับไปห้องไหน

9) ควรทำการทดสอบกับผู้ใช้อย่างเป็นระบบและบันทึกผลไว้
การทดสอบที่มีคือการให้เพื่อนห้าคนทดลองเล่น
และได้ข้อค้นพบเพียงข้อเดียวคือปริศนาองก์ที่หนึ่งเข้าใจง่ายที่สุด
โดยไม่มีบันทึกการทดสอบใดในโครงงาน ไม่มีแบบสอบถาม และไม่มีบันทึกเวลาที่ใช้แก้แต่ละองก์
หากบันทึกเวลาที่ผู้เล่นแต่ละคนใช้ในแต่ละองก์และจุดที่ติด
จะได้ข้อมูลที่ชี้ได้ว่าปริศนาองก์ใดยากเกินไปหรือคำใบ้ใดไม่ชัด
ซึ่งเป็นข้อมูลที่โครงงานประเภทนี้ควรมีและปัจจุบันยังไม่มี

10) ควรเก็บหลักฐานภาพของผลงานทันทีเมื่องานแต่ละส่วนเสร็จ
ตรวจพบว่าภาพถ่ายหน้าจอทั้ง 58 ไฟล์ที่เก็บไว้ในโครงงานมีวันที่อยู่ก่อนวันที่ 2 กันยายน 2569 ทั้งหมด ซึ่งเป็นช่วงก่อนงานในหกวันทำการสุดท้ายของการพัฒนา ผลคือ**ไม่มีภาพใดในโครงงานที่แสดงเกมในสถานะสุดท้าย**
และภาพประกอบรายงานทุกภาพต้องถ่ายใหม่ทั้งหมด
การถ่ายภาพเก็บไว้เมื่องานแต่ละส่วนเสร็จจะทำให้มีหลักฐานของพัฒนาการของงานด้วย
ซึ่งเป็นสิ่งที่รายงานสหกิจศึกษาต้องการ

11) ควร build เกมเป็นไฟล์สำหรับแจกจ่ายและทดสอบบนไฟล์นั้น
ตรวจแล้วว่าโครงงานยังไม่เคย build เลย ไม่พบโฟลเดอร์ผลลัพธ์ใด ๆ ซึ่งหมายความว่าโครงงานได้รับการทดสอบเฉพาะในตัวแก้ไขเท่านั้น
และ**ยังไม่เคยทดสอบในสภาพแวดล้อมที่ผู้เล่นจริงจะใช้** ข้อนี้มีน้ำหนักเป็นพิเศษกับโครงงานนี้

เพราะปัญหาหลักที่รายงานในหัวข้อ 5.2.1 คือปัญหาที่อาการแสดงออกต่างกันตามเครื่องที่ใช้เล่น การยืนยันว่าแก้สำเร็จแล้วจริงจึงควรทำบนไฟล์ที่ build แล้ว และทดสอบบนเครื่องที่มีความเร็วต่างกันอย่างน้อยสองเครื่อง

ลำดับความสำคัญของข้อเสนอแนะทั้งสิบเอ็ดข้อ โดยพิจารณาจากผลกระทบเทียบกับความยากในการทำ แสดงดังตารางที่ 5.2

ตารางที่ 5.2 ลำดับความสำคัญของข้อเสนอแนะ

ลำดับ	ข้อเสนอแนะ	ผลกระทบ	ความยาก
1	ข้อ 1 ใช้ระบบควบคุมรุ่น	สูงที่สุด ป้องกันความเสียหายที่ย้อนไม่ได้	ต่ำ
2	ข้อ 5 ตัดรายการอ้างอิงที่ไม่ใช้ออกจาก Assembly Definition	ปานกลาง	ต่ำที่สุด ไม่ต้องแก้โปรแกรม
3	ข้อ 4 แก้ข้อบกพร่องเรื่องลำดับ Awake	สูง เป็นข้อบกพร่องที่ยังค้าง	ต่ำ แก้หนึ่งคลาส
4	ข้อ 2 เขียนชุดทดสอบครอบโมดูล Puzzle	สูง จับข้อผิดพลาดชนิดที่เคยเกิดแล้ว	ต่ำ ไม่มีข้อจำกัดขัดขวาง
5	ข้อ 3 แก้ ICodeInput ให้นำกลับมาใช้ได้จริง	ปานกลาง ทำให้หลักการออกแบบทั่วถึง	ต่ำ มีแบบอย่างในโครงงานแล้ว
6	ข้อ 11 build และทดสอบบนเครื่องที่ต่างกัน	สูง ยืนยันว่าปัญหาหลักแก้สำเร็จจริง	ปานกลาง
7	ข้อ 10 เก็บหลักฐานภาพเมื่องานเสร็จ	ปานกลาง กระทบรายงานมากกว่าตัวงาน	ต่ำ
8	ข้อ 9 ทดสอบกับผู้ใช้อย่างเป็นระบบ	สูงต่อคุณภาพการเล่น	ปานกลาง ต้องหาผู้ทดลองเล่น
9	ข้อ 6 แยกความรับผิดชอบของ InspectView	ปานกลาง	ปานกลาง
10	ข้อ 8 เพิ่มการจัดการสถานะความก้าวหน้า	ปานกลาง เปิดทางให้คุณสมบัติใหม่	สูง
11	ข้อ 7 ตัดวงจรการพึ่งพา Player ⇄ UI	สูงต่อโครงสร้าง	สูงที่สุด

เอกสารอ้างอิง

[1] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, Dec. 1972.

[2] W. P. Stevens, G. J. Myers, and L. L. Constantine, "Structured design," *IBM Systems Journal*, vol. 13, no. 2, pp. 115–139, 1974.

[3] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018.

[4] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, International Organization for Standardization, Geneva, Switzerland, 2011.

[5] R. Nystrom, *Game Programming Patterns*. Genever Benning, 2014.

[6] J. Gregory, *Game Engine Architecture*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2018.

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[8] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River, NJ, USA: Prentice Hall, 2002.

[9] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[10] B. Meyer, *Object-Oriented Software Construction*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.

[11] E. W. Dijkstra, "On the role of scientific thought," in *Selected Writings on Computing: A Personal Perspective*. New York, NY, USA: Springer-Verlag, 1982, pp. 60–66.

[12] Unity Technologies, "ScriptableObject data assets," *Unity 6.3 User Manual*. [Online]. Available: https://docs.unity.com/en-us/engine/6000.3/manual/scripting/get-started/fundamental-unity-types/class-scriptable-object. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[13] Unity Technologies, "deltaTime," *Unity Scripting API Reference*. [Online]. Available: https://docs.unity.com/en-us/engine/current/script-reference/unityengine/time/deltatime. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[14] Unity Technologies, "Event function execution order," *Unity 6.3 User Manual*. [Online]. Available: https://docs.unity.com/en-us/engine/6000.3/manual/scripting/managing-update-order/execution-order. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[15] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, Mar. 1991.

[16] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, Jul. 2019.

[17] G. Meszaros, *xUnit Test Patterns: Refactoring Test Code*. Boston, MA, USA: Addison-Wesley, 2007.

[18] N. Ford, R. Parsons, and P. Kua, *Building Evolutionary Architectures: Support Constant Change*. Sebastopol, CA, USA: O'Reilly Media, 2017.

[19] Unity Technologies, "Introduction to the Universal Render Pipeline," *Unity 6.3 User Manual*. [Online]. Available: https://docs.unity.com/en-us/engine/6000.3/manual/render-pipelines/universal-render-pipeline/urp-introduction. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[20] *C# Language Specification*, Standard ECMA-334, 6th ed., ECMA International, Geneva, Switzerland, Jun. 2022.

[21] Unity Technologies, "MonoBehaviour," *Unity Scripting API Reference*. [Online]. Available: https://docs.unity.com/en-us/engine/current/script-reference/unityengine/monobehaviour. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[22] Unity Technologies, "Prefabs," *Unity 6.3 User Manual*. [Online]. Available: https://docs.unity.com/en-us/engine/6000.3/manual/working-with-gameobjects/prefabs. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[23] Unity Technologies, "Referencing assemblies," *Unity 6.3 User Manual*. [Online]. Available: https://docs.unity.com/en-us/engine/6000.3/manual/programming-environment/script-compilation/assembly-definition-files/assembly-definitions-referencing. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[24] Unity Technologies, "Input System," *Input System Package Documentation*, version 1.17.0. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.17/manual/index.html. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[25] Unity Technologies, "Unity Test Framework," *Test Framework Package Documentation*, version 1.6.0. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.test-framework@1.6/manual/index.html. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[26] NUnit Project, "NUnit — a unit-testing framework for all .NET languages." [Online]. Available: https://nunit.org. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

[27] M. Hendrikx, S. Meijer, J. Van Der Velden, and A. Iosup, "Procedural content generation for games: A survey," *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 9, no. 1, pp. 1–22, Feb. 2013.

[28] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Cham, Switzerland: Springer, 2016.

[29] Model Context Protocol, "What is the Model Context Protocol (MCP)?" [Online]. Available: https://modelcontextprotocol.io. [สืบค้นวันที่ 6 พฤศจิกายน 2569].

ภาคผนวก

ภาคผนวก ก.

รายงานปฏิบัติงานประจำสัปดาห์

[หมายเหตุของร่าง — ลบก่อนส่ง] **แทรกรายงานปฏิบัติงานประจำสัปดาห์ที่นี่** — รูปเล่มที่ผ่านการอนุมัติแล้วซึ่งใช้เทียบ แนบแบบฟอร์มของ ศูนย์สหกิจศึกษาและจัดหางาน สถาบันเทคโนโลยีไทย-ญี่ปุ่น ไว้ **27 หน้า หนึ่งหน้าต่อหนึ่งสัปดาห์** เรียงตามลำดับสัปดาห์ และหน้าในภาคผนวกเหล่านั้น **ไม่แสดงเลขหน้า** เอกสารชุดนี้เป็นสิ่งที่ผู้เขียนส่งศูนย์สหกิจศึกษาอยู่แล้วทุกสัปดาห์ ให้รวบรวมไฟล์หรือสำเนาที่เซ็นแล้วมาแทรกแทนหน้านี้ ถ้าช่วงฝึกงานคือกรกฎาคมถึงตุลาคม จะได้ประมาณ 16 ถึง 18 สัปดาห์ ถ้าสาขาวิชากำหนดให้มีภาคผนวกอื่นด้วย เช่น ภาคผนวก ข. สำหรับตัวอย่างโปรแกรมหรือคู่มือการใช้งาน ให้เพิ่มหน้าคั่นแบบเดียวกันนี้ต่อท้าย

ประวัติผู้จัดทำโครงงาน

ชื่อ-นามสกุล [.....]

วันเดือนปีเกิด [.....]

ประวัติการศึกษา

ระดับมัธยมศึกษา มัธยมศึกษาตอนต้น พ.ศ. [.....]
[ชื่อโรงเรียน]

มัธยมศึกษาตอนปลาย พ.ศ. [.....]
[ชื่อโรงเรียน]

ระดับอุดมศึกษา วิศวกรรมศาสตรบัณฑิต สาขาวิศวกรรม[.....]
สถาบันเทคโนโลยีไทย-ญี่ปุ่น พ.ศ. [.....]

สถานที่ฝึกงานสหกิจศึกษา บริษัท Lumas corp

ตำแหน่ง นักพัฒนาเกม (Game Developer)

ทุนการศึกษา [- ไม่มี -]

ประวัติการฝึกอบรม [- ไม่มี -]